

Часть I

Основные средства

Здесь описываются встроенные типы языка C++ и средства построения программ на их основе. Представлено C-подмножество языка C++ и его дополнительные возможности для поддержки традиционных стилей программирования. Также обсуждаются базовые средства составления программ на C++ из логических и физических частей.

Главы

4. Типы и объявления
5. Указатели, массивы и структуры
6. Выражения и операторы
7. Функции
8. Пространства имен и исключения
9. Исходные файлы и программы

Типы и объявления

Не соглашайтесь ни на что, кроме совершенства!

— Аноним

Совершенство достигается только к моменту краха.

— К.Н. Паркинсон

Типы — фундаментальные типы — логические значения — символы — символьные литералы — целые — целые литералы — типы с плавающей запятой — литералы с плавающей запятой — размеры — **void** — перечисления — объявления — имена — область видимости — инициализация — объекты — **typedef** — советы — упражнения.

4.1. Типы

Рассмотрим формулу

$x = y + f(2);$

Чтобы это выражение могло использоваться в программе на C++, имена **x**, **y** и **f** должны быть *объявлены* надлежащим образом. То есть программист должен указать, что некие сущности, поименованные **x**, **y** и **f**, реально существуют, и что они имеют типы, для которых операции = (присваивание), + (сложение) и () (вызов функции) имеют точный смысл.

Каждое имя (*идентификатор*) в программе на C++ *ассоциируется с некоторым типом*. Именно тип определяет, какие операции можно выполнять с этим именем (то есть с именованной сущностью), и как интерпретируются эти операции. Например, следующие объявления

```
float x;           // x - переменная с плавающей запятой
int y=7;          // y - целая переменная с начальным значением 7
float f(int);      // f - функция с аргументом целого типа,
                  // возвращающая число с плавающей запятой
```

делают наш текущий пример осмысленным. Поскольку у объявлена переменная целого типа, то ей можно присваивать значения, ее можно использовать в арифметических выражениях и т.д. С другой стороны, *f* объявлена как функция, принимающая целый аргумент, так что ее можно вызывать с подходящим аргументом.

В данной главе рассматриваются фундаментальные типы (§4.1.1) и объявления (§4.9). Примеры в ней просто иллюстрируют свойства языка; они не предназначены для демонстрации чего-либо практически полезного. Изошренные и реалистичные примеры будут приводиться в последующих главах, когда большая часть языка C++ будет уже рассмотрена. Здесь же перечисляются базовые элементы, из которых состоят все программы на языке C++. Вы должны освоить эти элементы плюс терминологию и сопутствующий им несложный синтаксис, чтобы выполнять законченные проекты на C++, и особенно для того, чтобы иметь возможность разбираться в чужих кодах. Тем не менее, для изучения последующих глав глубокого понимания всех рассмотренных в настоящей главе вопросов не требуется. Достаточно бегло рассмотреть основные концепции, а за необходимыми деталями вернуться позднее.

4.1.1. Фундаментальные типы

Язык C++ обладает набором *фундаментальных типов*, соответствующих *базовым принципам организации компьютерной памяти* и самым общим способам хранения данных:

- §4.2 Логический тип (*bool*)
- §4.3 Символьные типы (такие как *char*)
- §4.4 Целые типы (такие как *int*)
- §4.5 Типы с плавающей запятой (такие как *double*)

Дополнительно, пользователь может объявить

- §4.8 Перечислимые типы для представления конечного набора значений (*enum*)

Также имеется

- §4.7 Тип *void*, указывающий на отсутствие информации

Поверх перечисленных типов можно строить следующие типы:

- §5.1 Указательные типы (такие как *int**)
- §5.2 Массивы (такие как *char[]*)
- §5.5 Ссылочные типы (такие как *double&*)
- §5.7 Структуры данных и классы (глава 10)

Логический, символьные и целые типы называются *интегральными типами*. Интегральные типы и типы с плавающей запятой в совокупности называются *арифметическими типами*. Перечисления и классы (глава 10) называются *типами, определяемыми пользователем*, так как их нужно определить до момента использования, что контрастирует с фундаментальными типами, не требующими никаких объявлений. Поэтому их еще называют *встроенными типами* (*built-in types*).

Интегральные типы и типы с плавающей запятой *могут иметь различные размеры*, предоставляя тем самым программисту выбор объема занимаемой ими памяти, точности представления и диапазона возможных значений (§4.6). Общая идея со-

стоит в том, что на компьютере символы хранятся в виде набора байт, целые числа хранятся и обрабатываются как наборы машинных слов, числа с плавающей запятой располагаются в характерных для них объемах памяти (соответствующих специализированным регистрам процессора), и на все эти сущности можно ссылаться по адресам. Фундаментальные типы языка C++ вместе с указателями и массивами предоставляют программисту отражение этого машинного уровня в манере, относительно независимой от конкретной реализации.

В большинстве приложений можно обойтись типом **bool** для логики, типом **char** для символов, типом **int** для целых чисел и типом **double** — для дробных (чисел с плавающей запятой). Остальные фундаментальные типы являются вариациями, предназначенными для оптимизации и решения специальных задач, и их лучше до поры игнорировать. Но, разумеется, их нужно знать для чтения существующего стороннего кода на C и C++.

4.2. Логический тип

Переменные логического типа **bool** могут принимать лишь два значения — **true** (истина) или **false** (ложь). Логические переменные служат для хранения результатов логических операций. Например:

```
void f(int a, int b)
{
    bool bl = a==b;    // = есть присваивание, == проверка на равенство;
    // ...
}
```

Если у целых переменных **a** и **b** значения одинаковые, то **bl** будет равно **true**; в противном случае **bl** станет равным **false**.

Часто тип **bool** служит типом возврата функций, проверяющих выполнение некоторого условия (предикатные функции, или просто *предикаты*). Например:

```
bool is_open (File* ) ;
bool greater (int a, int b) {return a>b; }
```

По определению, при преобразовании к типу **int** значению **true** сопоставляется **1**, а значению **false** — **0**. В обратном направлении целые значения неявно преобразуются в логические следующим образом: ненулевые значения трактуются как **true**, а **0** как **false**. Например:

```
bool b=7;           // bool(7) есть true, так что b инициализируется значением true
int i= true;        // int(true) есть 1, так что i инициализируется значением 1
```

В арифметических и логических выражениях логические значения сначала преобразуются в целые, после чего выражения и вычисляются. Если результат вычислений требуется преобразовать обратно в тип **bool**, то **0** преобразуется в **false**, а ненулевые значения — в **true**:

```
void g ()
{
    bool a=true;
    bool b=true;
```

```

bool x=a+b;    // a+b равно 2, так что x становится true
bool y=a|b;    // a|b равно 1, так что y становится true
}

```

Указатель можно неявно преобразовывать к типу **bool** (§C.6.2.5). Ненулевой указатель преобразуется в **true**; указатель с нулевым значением — в **false**.

4.3. Символьные типы

В переменной типа **char** может храниться код символа, соответствующий некоторой стандартной кодировке. Например:

```
char ch = 'a' ;
```

Практически повсеместно под тип **char** отводится *8 бит памяти*, так что переменные этого типа могут хранить *один из 256 символов*. В типичном случае кодировка символов является одним из вариантов стандарта ISO-646, например ASCII, что соответствует символам стандартной клавиатуры. Много проблем проистекает из-за того, что набор этот стандартизован не полностью (§C.3).

Серьезные различия имеют место для кодировок, поддерживающих разные национальные языки, и даже для разных кодировок одного и того же языка. Здесь мы, однако, интересуемся лишь тем, как это влияет на язык C++. Сложная и интересная проблема создания программ, поддерживающих несколько национальных языков и/или кодировок, в целом выходит за рамки настоящей книги, хотя и упоминается в отдельных ее местах (§20.2, §21.7 и §C.3.3).

Можно с уверенностью полагать, что любая кодировка содержит десятичные цифры, 26 букв английского алфавита и стандартные знаки пунктуации. Неправильно полагать, что каждая 8-битная кодировка содержит не более 127 символов (большинство национальных кодировок содержат 255 символов), что нет алфавитных символов, отличных от букв английского языка (многие Европейские языки содержат дополнительные символы), что алфавитные символы имеют смежные коды (кодировка EBCDIC оставляет зазор между символами 'i' и 'j'), или что в обязательном порядке присутствуют все символы, необходимые для записи конструкций языка C++ (в некоторых национальных кодировках отсутствуют символы {} [] \; см. §C.3.1). Лучше вообще избегать привязки к конкретным представлениям объектов. Это касается и символов.

Каждому символу сопоставляется целочисленное значение. Например, символу 'b' в рамках кодировки ASCII соответствует число 98. Вот программа, которая ответит на вопрос о числовом коде любого символа, который вы введете с клавиатуры:

```

#include <iostream>

int main ()
{
    char c;
    std::cin >> c;
    std::cout << "the value of '" << c << "' is " << int(c) << '\n';
}

```

Выражение *int(c)* дает целочисленное значение для символа *c*. В связи с преобразованием *char* в *int* возникает вопрос: тип *char* знаковый или беззнаковый? Ведь в зависимости от этого 256 восьмибитных значений могут трактоваться как числа от 0 до 255, или как числа от -128 до +127. К сожалению, точный ответ на вопрос *зависит от конкретной реализации* (§C.2, §C.3.4). В языке C++ два типа дают точные ответы на указанный вопрос: тип *unsigned char* соответствует значениям от 0 до 255, а тип *signed char* — значениям от -128 до +127. К счастью, различия касаются символов с кодами вне диапазона от 0 до 127, в который попадают все общеупотребительные символы.

Если тип *char* используется для хранения значений вне диапазона 0 — 127, то возможны проблемы с переносимостью. Ознакомьтесь с разделом §C.3.4, если у вас в программе используются сразу несколько символьных типов или если вы храните целые значения в типе *char*.

Тип *wchar_t* специально предназначен для хранения более широкого диапазона кодов, характерных, например, для кодировки Unicode. Размер этого типа данных зависит от реализации и всегда достаточен для хранения самого широкого символьного набора, поддерживающего ту или иную национальную специфику (см. §21.7 и §C.3.3). Странное имя этого типа досталось от языка C. В языке C этот тип не является встроенным, а определяется с помощью оператора *typedef* (§4.9.7). Суффикс *_t* специально применяется с целью четкого указания на то, что *тип определен с помощью typedef*.

Подчеркнем, что символьные типы являются интегральными (§4.1.1), так что к ним можно применять арифметические и побитовые логические операции (§6.2).

4.3.1. Символьные литералы

Символьным литералом (*символьной константой*; *character literal* или *character constant*) называется символ, заключенный в *апострофы* (одиночные кавычки), например, *'a'* или *'0'*. Типом символьного литерала является *char*. Символьные литералы на самом деле есть просто символьные константы, именующие целочисленные значения из текущих кодировок символов, используемых на компьютере. Например, если программа выполняется на компьютере с ASCII-кодировкой, то *'0'* есть 48. Применение символьных литералов вместо десятичных обозначений делает программы более переносимыми. Ряд символьных литералов в своей записи используют специальный символ ** (backslash — обратная косая черта или escape-символ) и имеют стандартные названия, например, *"\n"* это символ перевода строки, а *"\t"* — символ табуляции. Более подробно такие символьные литералы рассматриваются в §C.3.2.

Символьные литералы для расширенных наборов символов имеют вид *L'ab'*, где количество символов между апострофами и их смысл зависят от конкретных реализаций, чтобы соответствовать типу *wchar_t*. Естественно, что такие литералы имеют тип *wchar_t*.

4.4. Целые типы

Как и тип *char*, каждый целый тип представим в трех формах: «просто» *int*, *signed int* и *unsigned int*. Кроме того, целые могут быть трех размеров: *short int*, «просто» *int* и *long int*. Вместо *long int* можно просто писать *long*. Аналогично, *short* есть синоним для *short int*, *unsigned* — для *unsigned int*, а *signed* — для *signed int*.

Беззнаковые (*unsigned*) целые типы идеальны для трактовки блоков памяти как битовых массивов. Использование *unsigned* вместо *int* с целью заполучить лишний бит для представления целых положительных значений почти всегда является неудачным решением. А попытки гарантировать положительность числовых значений объявлением целой переменной с модификатором *unsigned* опровергаются правилами неявных преобразований типов (§C.6.1, §C.6.2.1).

В отличие от типа «просто» *char*, типы «просто» *int* всегда знаковые (*signed*). Явное применение модификатора *signed* лишь подчеркивает этот факт.

4.4.1. Целые литералы

Целые литералы представимы в четырех внешних обликах: десятичном, восьмеричном, шестнадцатеричном и символьном (§A.3). Чаще всего используются десятичные литералы, которые выглядят вполне ожидаемым образом

0 1234 976 12345678901234567890

Компилятор обязан выдавать предупреждение, если величина литерала выходит за допустимые границы.

Литерал, начинающийся с *0x*, является шестнадцатеричным (по основанию 16) литералом. Если же литерал начинается с нуля, но символ *x* за ним не следует, то это восьмеричный (по основанию 8) литерал. например:

<i>decimal</i> (десятичный) :		2	63	83
<i>octal</i> (восьмеричный) :	0	02	077	0123
<i>hexadecimal</i> (шестнадцатеричный) :	0x0	0x2	0x3f	0x53

Буквы *a*, *b*, *c*, *d*, *e* и *f* (или их эквиваленты в верхнем регистре) используются для обозначения чисел *10*, *11*, *12*, *13*, *14* и *15*, соответственно. Восьмеричные и шестнадцатеричные литералы используются преимущественно для представления битовых последовательностей. Использование таких литералов для обозначения обычных чисел чревато сюрпризами. Например, на аппаратных платформах, у которых тип *int* реализуется 16-битным словом (с представлением отрицательных целых в дополнительном коде), *0xffff* есть отрицательное десятичное число *-1*, а на платформах с большим числом бит — это уже *65535*. Суффикс *U* явным образом обозначает беззнаковое (*unsigned*) целое, а суффикс *L* — длинное (*long*) целое. Например, типом литерала *3* является *int*, типом *3U* — *unsigned int*, а типом литерала *3L* — тип *long int*. Если суффикс отсутствует, компилятор самостоятельно приписывает литералу подходящий тип, который он выбирает, сопоставляя значение литерала и машинозависимые размеры целых типов (§C.4).

Целесообразно ограничить использование неочевидных констант хорошо прокомментированными объявлениями с ключевым словом *const* (§5.4) или инициализаторами перечислений (§4.8).

4.5. Типы с плавающей запятой

Типы с плавающей запятой соответствуют числам с плавающей запятой. Как и целые типы, они могут быть *трех размеров*: **float** (одинарная точность), **double** (двойная точность) и **long double** (расширенная точность).

Точный смысл каждого типа зависит от реализации. Выбор оптимальной точности в конкретных задачах требует изрядных знаний в области вычислений с плавающей запятой. Если у вас их нет, то проконсультируйтесь со специалистом, или основательно изучите предмет, или используйте тип **double** наудачу.

4.5.1. Литералы с плавающей запятой

По умолчанию *литералы с плавающей запятой* (*floating-point literal*) имеют тип **double**. Опять-таки, компилятор обязан выдавать предупреждения в случаях, когда значение литерала не соответствует машинным размерам для типов с плавающей запятой. Приведем примеры литералов с плавающей запятой:

```
1.23 .23 0.23 1. 1.0 1.2e10 1.23e-15
```

Пробельные символы в составе обозначений литералов с плавающей запятой *не допускаются*. Например, **65.43 e-21** не является литералом, а скорее это набор из четырех отдельных лексем (вызывающих ошибку компиляции):

```
65.43 e - 21
```

Если вам нужен литерал с плавающей запятой, имеющий тип **float**, следует использовать *суффиксы f* или **F**:

```
3.14159265f 2.0f 2.997925F 2.9e-3f
```

Для придания литералу типа **long double** следует использовать суффиксы **l** или **L**:

```
3.14159265L 2.0L 2.997925L 2.9e-3L
```

4.6. Размеры

Некоторые аспекты фундаментальных типов языка C++, например, размер типа **int**, зависят от конкретных реализаций (§C.2). Я всегда обращаю внимание на подобного рода зависимости и рекомендую по возможности устранять их, или хотя бы минимизировать последствия. Почему вообще по этому поводу стоит беспокоиться? Людям, программирующим на разных платформах и применяющим разные компиляторы, ответ на вопрос очевиден: иначе придется тратить кучу времени, чтобы то и дело отыскивать трудноуловимые и крайне неочевидные ошибки. Люди же, ограниченные единственной системной платформой и единственным компилятором, чувствуют себя свободными от этой проблемы, утверждая, что «язык — это то, что реализует мой компилятор». Я считаю эту точку зрения узкой и недалёкой. Действительно, ведь если программа добивается успеха на какой-то платформе, то ее с огромной долей вероятности будут портировать на иные аппаратно-программные платформы, и кому-то все равно придется возиться с устранением имеющихся несовместимостей. Кроме того, часто программы приходится компилировать разными компиляторами на одной и той же системной платформе, или,

например, не исключена ситуация, когда новые версии одного и того же компилятора по-разному реализуют те или иные аспекты языка C++. Намного проще при первоначальном написании программного кода заранее выявить все источники системных несовместимостей и ограничить их влияние, чем потом разыскивать их в хитросплетениях готовой программы.

Ограничить влияние нюансов языка, зависящих от конкретной реализации, относительно несложно. Намного труднее ограничить влияние системнозависимых библиотек на переносимость программ. Везде, где только возможно, стоит использовать средства стандартных библиотек.

Несколько целых типов, беззнаковые типы и типы со знаком, нескольких типов с плавающей запятой — все это предназначено для того, чтобы программист мог *выжать максимум из аппаратных возможностей конкретного компьютера*. Для разных типов компьютеров объем требуемой памяти, время доступа к ней и скорость вычислений существенно зависят от выбора фундаментального типа. Если вам знакома машинная архитектура, то несложно выбрать, например, подходящий целый тип для конкретной переменной. Значительно труднее писать истинно переносимый низкоуровневый программный код.

Размеры программных объектов в C++ кратны размеру типа **char**, так что фактически по определению размер типа **char** равен 1. Размер любого объекта или типа можно узнать с помощью операции **sizeof** (§6.2). Вот что гарантируется относительно размеров фундаментальных типов:

$$\begin{aligned} 1 &\equiv \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \\ 1 &\leq \text{sizeof}(\text{bool}) \leq \text{sizeof}(\text{long}) \\ \text{sizeof}(\text{char}) &\leq \text{sizeof}(\text{wchar_t}) \leq \text{sizeof}(\text{long}) \\ \text{sizeof}(\text{float}) &\leq \text{sizeof}(\text{double}) \leq \text{sizeof}(\text{long double}) \\ \text{sizeof}(N) &\equiv \text{sizeof}(\text{signed } N) \equiv \text{sizeof}(\text{unsigned } N) \end{aligned}$$

Здесь N может быть **char**, **short int**, **int** или **long int**. Кроме того, гарантируется, что под **char** отводится минимум 8 бит, под **short** или **int** — минимум 16 бит, под **long** — минимум 32 бита. Тип **char** достаточен для представления всего набора символов, присущих машинной архитектуре.

Представим графически типовой набор фундаментальных типов, а также конкретный экземпляр текстовой строки:

char:	'a'
bool:	1
short:	756
int:	100000000
int*:	&c1
double:	1234567e34
char[14]:	Hello, world!\0

В сопоставимом масштабе (0.2 дюйма на байт) мегабайт памяти растянется вправо на три мили (около 5 километров).

Размер типа ***char*** выбирается так, чтобы он был оптимальным для хранения и манипулирования символами на данном типе компьютеров; обычно это 8 бит (один байт). Аналогично, размер типа ***int*** выбирается с целью оптимального хранения и вычислений с целыми числами; обычно это 4-байтовое слово (32 бита). Неразумно требовать большего, однако ж, машины с 32-битным типом `char` существуют.

Машинозависимые аспекты фундаментальных типов представлены в файле `<limits>` (§22.2). Например:

```
#include <limits>
#include <iostream>

int main ()
{
    std::cout<<"largest float=="<<std::numeric_limits<float>::max ()
    << " , char is signed == " << std::numeric_limits<char>::is_signed<<' \n' ;
}
```

Разные фундаментальные типы можно свободно *смешивать в выражениях* (например, в присваиваниях). При этом, *по-возможности*, величины преобразуются так, чтобы *не терялась информация* (§C.6).

Если значение *v* может быть точно представлено в переменной типа *T*, то преобразование *v* к типу *T* не ведет к потере информации, и нет проблем. Преобразования с потерей информации лучше всего просто не допускать (§C.6.2.6).

Для успешного выполнения больших программных проектов и для понимания практически важного стороннего кода требуется точное понимание процесса *неявного преобразования типов (implicit conversion)*. Но для чтения последующих глав в этом нет необходимости.

4.7. Тип `void`

Синтаксически тип ***void*** относится к фундаментальным типам, но использовать его можно лишь как часть более сложных типов, ибо объектов типа ***void*** *не существует*. Этот тип используется либо для информирования о том, что у функции нет возврата, либо в качестве базового типа указателей на объекты неизвестных типов. Например:

```
void x;           // ошибка: не существует объектов типа void
void& r;          // ошибка: не существует ссылок на void
void f();         // функция f не возвращает значения (§7.3)
void* pv;         // указатель на объект неизвестного типа (§5.6)
```

Объявляя функцию, вы обязаны указать тип ее возврата. С логической точки зрения кажется, что в случае отсутствия возврата можно было бы просто опустить тип возврата. Это, однако, сделало бы синтаксис C++ менее последовательным (Приложение А) и конфликтовало бы с синтаксисом языка C. Поэтому ***void*** используется в качестве «псевдотипа возврата», означающего, что фактически возврата нет.

4.8. Перечисления

Перечисление (enumeration) является *типом*, содержащим *набор значений*, определяемых пользователем (программистом). После определения перечисление используется почти так же, как и целые типы.

Именованные целые константы служат *элементами перечислений*. Например, следующее объявление

```
enum {ASM, AUTO, BREAK};
```

определяет в качестве элементов перечисления три целые константы, которым неявно присваиваются целые значения. *По умолчанию*, целые значения присваиваются элементам перечисления в возрастающем порядке, *начиная с нуля*, так что *ASM == 0, AUTO == 1 и BREAK == 2*. Перечисление *может иметь имя*, например:

```
enum keyword { ASM, AUTO, BREAK};
```

Каждое перечисление является отдельным типом. *Типом элемента перечисления* является *тип самого перечисления*. Например, тип *AUTO* есть *keyword*.

Если переменная объявляется с типом *keyword* (тип перечисления), а не просто *int*, то это дает пользователю и компилятору дополнительные сведения о предполагаемом характере использования этой переменной. Например:

```
void f(keyword key)
{
    switch (key)
    {
        case ASM:
            // некоторые действия
            break;
        case BREAK:
            // некоторые действия
            break;
    }
}
```

Здесь компилятор может выдать предупреждение о том, что используются лишь два из трех элементов перечисления.

Элементы перечисления можно явно инициализировать *константными выражениями* (§C.5) интегрального типа (§4.1.1). Когда все элементы перечисления неотрицательные, *диапазон их значений* равен $[0; 2^k - 1]$, где 2^k есть наименьшая степень двойки, для которой все элементы перечисления попадают в указанный диапазон. При наличии отрицательных элементов этот диапазон равен $[-2^k; 2^k - 1]$. Диапазон определяется *минимальным количеством бит, требуемых для представления всех значений элементов* (отрицательные значения — в дополнительном коде). Например:

```
enum e1 {dark, light};           // диапазон 0:1
enum e2 {a = 3, b = 9};         // диапазон 0:15
enum e3 {min = -10, max = 1000000}; // диапазон -1048576:1048575
```

Значение интегрального типа может быть явно преобразовано к типу перечисления. Если при этом исходное значение не попадает в диапазон перечисления, то результат преобразования не определен. Например:


```
enum flag {x=1, y=2, z=4, e=8 };      // диапазон 0:15
flag f1=5;                            // ошибка типа: 5 не принадлежит к типу flag
flag f2=flag(5);                      // ok: flag(5) принадлежит типу flag и входит в его диапазон
flag f3=flag(z|e);                   // ok: flag(12) принадлежит типу flag и входит в его диапазон
flag f4=flag(99);                    // не определено: 99 не входит в диапазон типа flag
```

Последнее присваивание наглядно показывает, почему не допускаются неявные преобразования целых в перечисления — просто большинство целых значений не имеют представления в отдельно взятом конкретном перечислении.

Понятие диапазона перечисления в языке C++ отличается от понятия перечисления в языках типа *Pascal*. Однако в языке C и затем в языке C++ исторически закрепились необходимость точного определения диапазона перечислений (не совпадающего со множеством значений элементов перечисления) для корректного манипулирования битами.

Под перечисления (точнее, под переменные этого типа) отводится столько же памяти (результат операции *sizeof*), что и под интегральный тип, способный содержать весь диапазон перечисления. При этом указанный объем памяти не больше, чем *sizeof(int)* при условии, что все элементы перечисления представимы типами *int* или *unsigned int*. Например, *sizeof(e1)* может быть равен 1 или 4, но не 8, на компьютере, для которого *sizeof(int) == 4*.

При выполнении арифметических операций перечисления автоматически (неявно) преобразуются в целые (§6.2). Перечисления относятся к определяемым пользователем типам, так что для них можно задавать свои собственные операции вроде ++ или << (§11.2.3).

4.9. Объявления

Прежде, чем имя (идентификатор) будет использоваться в программе, оно должно быть объявлено. То есть должен быть специфицирован тип, говорящий компилятору о том, к какого рода сущностям относится поименованный программный объект. Вот примеры, иллюстрирующие разнообразие возможных объявлений:

```
char ch;
string s;
int count=1;
const double pi=3.1415926535897932385;
extern int error_number;
const char* name = "Njal";
const char* season[] = { "spring", "summer", "fall", "winter" };

struct Date {int d, m, y; };
int day(Date* p) {return p->d; }
double sqrt(double);
template<class T> T abs(T a) {return a<0 ? -a : a; }

typedef complex<short> Point;
struct User;
enum Beer {Carlsberg, Tuborg, Thor};
namespace NS { int a; }
```

Как видно из представленных примеров, объявления могут делать нечто большее, чем просто связывать имя с типом. Большинство из данных *объявлений* (*declarations*) являются еще и *определениями* (*definitions*), так как они помимо имени и типа определяют (создают) еще и сущности, на которые имена ссылаются. Для имени `ch` этой объективной сущностью является блок памяти определенного размера, выделяемый под переменную с именем **ch**. Для имени `day` таковой сущностью является полностью определенная функция. Для константы **pi** — это число **3.1415926535897932385**. Имя **Date** связывается с новым типом. Для **Point** объективной сущностью служит тип **complex<short>** — имя **Point** становится синонимом указанного типа. Из всех представленных выше объявлений только

```
double sqrt(double);
extern int error_number;
struct User;
```

не являются определениями: объявленные здесь имена ссылаются на сущности, которые должны быть определены в рамках иных объявлений. В некотором ином объявлении должно быть определено тело (код) функции **sqrt()**, где-то в другом месте должна быть выделена память под целую переменную **error_number**, в рамках иного объявления имени типа **User** должно быть точно специфицировано, что есть этот тип на самом деле, и где-то должен быть определен тип **complex<short>**. Например:

```
double sqrt(double d) { /* ... */ }
int error_number = 1;
struct User { /* ... */ };
```

В программах на C++ для каждой именованной сущности должно быть ровно одно определение (о действии директивы **#include** см. §9.2.3). Объявлений же может быть сколько угодно. При этом все объявления некоторой сущности должны согласовываться по типу этой сущности. Поэтому фрагмент кода

```
int count;
int count; // ошибка: redefinition (повторное определение)
extern int error_number;
extern short error_number; // ошибка: type mismatch (несоответствие типов)
```

содержит две ошибки, а следующий фрагмент

```
extern int error_number;
extern int error_number;

typedef complex<short> Point;
typedef complex<short> Point;
```

ошибок не содержит.

Некоторые определения снабжают определяемые сущности «значением». Например:

```
struct Date { int d, m, y; };
int day(Date* p) { return p->d; }
const double pi = 3.1415926535897932385;
```

Для типов, шаблонов, функций и констант такое «значение» постоянно. Для неконстантных типов данных начальное значение может быть позднее изменено. Например:

```
void f()
{
    int count=1;
    const char* name="Bjarne"; // name указывает на константу (§5.4.1)
    //
    count=2;
    name="Marian";
}
```

Только в двух определениях из представленных в начале данного раздела не задаются значения:

```
char ch;
string s;
```

Вопрос о том, как и когда переменным присваиваются значения по умолчанию, рассмотрен в §4.9.5 и §10.4.2. Любое объявление, в котором значение задается, является определением.

4.9.1. Структура объявления

Объявление состоит из четырех частей: необязательного «спецификатора», базового типа, *декларатора* (*declarator*) и необязательного *инициализирующего выражения* (*инициализатора* — *initializer*). За исключением определений функции и пространства имен объявление заканчивается точкой с запятой. Например:

```
char* kings[]={ "Antigonus", "Seleucus", "Ptolemy" };
```

Здесь *char* является базовым типом, декларатором является **kings[]*, а инициализирующее выражение есть *= { ... }*. Стоящие в начале объявления ключевые слова, вроде *virtual* (§2.5.5, §12.2.6) или *extern* (§9.2), служат *спецификаторами* (*specifiers*), задающими (специфицирующими) *дополнительные атрибуты*, отличные от типа.

Деклараторы состоят из объявляемого имени и, возможно, дополнительных знаков операций. Наиболее употребительными в деклараторах являются следующие знаки операций (§A.7.1):

*	указатель	<i>prefix</i>
*const	константный указатель	<i>prefix</i>
&	ссылка	<i>prefix</i>
[]	массив	<i>postfix</i>
()	функция	<i>postfix</i>

Их использование не вызывало бы дополнительных сложностей, будь они все как один *префиксными* (*prefix*), или же *постфиксными* (*postfix*). Однако *, [] и () разрабатывались так, чтобы теснее отражать свою роль в построении выражений (§6.2). В результате, операция * является префиксной, а [] и () — постфиксными. В деклараторах связь со знаками постфиксных операций более тесная, чем с префиксными. Как следствие, **kings[]* есть массив указателей на что угодно, а в случае объявления «указателя на функцию» приходится в обязательном порядке использовать группирующие круглые скобки (см. примеры в §5.1). За более подробными грамматическими деталями отсылаем к Приложению А.

Отметим, что тип является неотъемлемой частью объявлений. Например:

```

const c=7;                                // error: нет типа
gt(int a, int b) {return (a>b) ? a : b;}    // error: нет типа возвращаемого значения
unsigned ui;                               // ok: 'unsigned' есть mun 'unsigned int'
long li;                                   // ok: 'long' есть mun 'long int'

```

Ранние версии языков С и С++ отличались от текущего стандарта С++ тем, что первые два из перечисленных примеров являлись в них допустимыми, так как по умолчанию предполагался тип `int` (§В.2). Это *правило «неявного int»* служило *источником недоразумений* и приводило к трудноуловимым ошибкам.

4.9.2. Объявление нескольких имен

В одном операторе объявления можно сосредоточить сразу несколько объявляемых имен. В этом случае объявление содержит список деклараторов, разделенных запятыми. Например, можно объявить две целые переменные следующим образом:

```
int x, y;           // int x; int y;
```

Важно отметить, что *действие знаков операций относится исключительно к ближайшему имени*, и не распространяется на весь список имен в объявлении. Например:

```

int* p, y;          // int* p; int y; (не int* y;)
int x, *q;           // int x; int* q;
int v[10], *pv;      // int v[10]; int* pv;

```

В принципе, такие конструкции лишь затуманивают смысл программы, и их лучше избегать.

4.9.3. Имена

Имя (идентификатор) состоит из последовательности букв и цифр. Первым символом должна быть буква. Знак `_` (знак подчеркивания) считается буквой. В языке С++ нет ограничений на количество символов в имени. Однако некоторые части системной реализации (например, компоновщик — `linker`) находятся вне компетенции разработчиков компилятора, и эти вот части, к сожалению, могут налагать определенные ограничения на имена. Среды выполнения (`run-time environments`) также могут расширять или ограничивать применимые в именах символы. Расширения (например, символ `$`) приводят к непереносимым программам. Ключевые слова языка С++ (Приложение А), например, `new` или `int`, не могут использоваться в качестве имен определяемых пользователем сущностей. Вот примеры допустимых имен:

```

hello      this_is_a_most_unusually_long_name
DEFINED    foO      bAr      u_name      HorseSense
var0       var1     CLASS    _class     _

```

А вот примеры последовательностей символов, которые именами быть не могут:

```

012      a fool      $sys      lass 3var
pay.due  foo~bar    .name     if

```

Имена, начинающиеся со знака подчеркивания, обычно резервируются для специальных средств реализации и сред выполнения, так что их не стоит использовать в обычных прикладных программах.

Компилятор, читая программу, пытается для фиксации имен использовать *наиболее длинные последовательности символов*. Таким образом, последовательность символом **var10** составляет единое имя, а не имя **var** с последующим числом **10**. Аналогично, **elseif** — это одно имя, а не ключевое слово **else**, за которым следует ключевое слово **if**.

Символы верхнего и нижнего регистров различаются, так что имена **Count** и **count** — разные. Однако создавать имена, отличающиеся лишь регистром их символов, не слишком разумно. И вообще, лучше избегать создания похожих друг на друга имен. Например, буква **o** в верхнем регистре (**O**) и нуль (**0**) выглядят похожими; это же характерно для буквы **l** и единицы (**1**). Следовательно, выбор идентификаторов **l0**, **lO**, **l1** и **lI** вряд ли можно признать удачным.

Имена с широкой областью видимости должны быть достаточно длинными и «самоговорящими», например, **vector**, **Window_with_border** или **Department_number**. В то же время код становится чище и прозрачнее, если именам с узкой областью видимости присваиваются короткие и привычные имена вроде **x**, **i** и **p**. Классы (глава 10) и пространства имен (§8.2) помогают ограничивать области видимости. Полезно бывает часто используемые имена делать более короткими, а более длинные — резервировать для редко используемых сущностей. Целесообразно выбирать имена так, чтобы они отражали саму сущность, а не детали ее представления. Например, имя **phone_book** лучше, чем **number_list**, несмотря на то, что телефонные номера и будут, вероятно, храниться в контейнере **list** (§3.7). Выбор хороших имен — это искусство.

Старайтесь придерживаться единого стиля именования. Например, имена пользовательских типов из нестандартной библиотеки начинайте с заглавной буквы, а имена, не относящиеся к типам — с прописной (например, **Shape** и **current_token**). Имена макросов составляйте из одних лишь букв верхнего регистра (если уж нужен макрос, то пишите, например, **НАСК**), а отдельные смысловые части идентификаторов разделяйте знаками подчеркивания. И все-таки, на практике единого стиля достигнуть трудно, так как программы часто составляются из фрагментов, полученных из разных источников, применяющих разумные, но разные системы именования. Будьте последовательны в выборе сокращений и составных имен.

4.9.4. Область видимости

Объявление вводит имя в *область видимости (scope)*; это значит, что имя можно использовать лишь в ограниченной части программы. Для имени, объявленного в теле функции (его часто называют *локальным*), область видимости простирается от точки объявления до конца содержащего это объявление блока. *Блоком* называется фрагмент кода, ограниченного с двух сторон фигурными скобками (открывающей и закрывающей).

Имя называется *глобальным*, если оно объявлено вне функций, классов (глава 10) и пространств имен (§8.2). Область видимости глобального имени простирается от точки объявления до конца файла, содержащего это объявление. Объявление имени внутри блока скрывает совпадающее имя, объявленное во внешнем объемлющем блоке, или глобальное имя. Таким образом, внутри блока имя можно переопределить с тем, чтобы оно ссылалось на новую сущность. По выходе из блока имя восстанавливает свой прежний смысл. Например:

```

int x;           // глобальная переменная x

void f()
{
    int x;       // локальная x скрывает глобальную x
    x = 1;       // присваивание локальной x

    {
        int x;   // скрывает первую локальную x
        x = 2;   // присваивание второй локальной x
    }

    x = 3;       // присваивание первой локальной x
}

int* p = &x;    // взять адрес глобальной x

```

Скрытие имен неизбежно в больших программах. В то же время, программист может и не заметить, что какое-то имя оказалось скрыто. Из-за того, что такие ошибки редки и нерегулярны, их трудно обнаруживать. Следовательно, *сокрытие имен желательно минимизировать*. Хотите проблем — используйте *i* или *x* в качестве глобальных имен, или в качестве локальных в функциях большого размера.

Доступ к скрытому глобальному имени осуществляется с помощью операции *::* (операция разрешения области видимости — *scope resolution operator*). Например:

```

int x;

void f2()
{
    int x=1;       // скрывает глобальную x
    : :x=2;        // присваивание глобальной x
    x=2;           // присваивание локальной x
    // ...
}

```

Не существует способа обращения к скрытому локальному имени.

Область видимости имени начинается в *точке объявления*, точнее, *сразу после декларатора*, но *перед инициализирующим выражением*. Из этого следует, что имя можно использовать в качестве инициализатора самого себя. Например:

```

int x;

void f3()
{
    int x = x;     // извращение: инициализируем x ее собственным
                  // (непроинициализированным) значением
}

```

Это не запрещено, просто глупо. Хороший компилятор предупредит о попытке чтения неинициализированной переменной (§5.9[9]).

В принципе, в блоке можно обращаться с помощью одного и того же имени к разным переменным, даже не используя операции *::*. Вот пример на эту тему:

```

int x=11;

void f4 ()
{
    int y = x;           // используется глобальная x: y=11
    int x = 22;
    y = x;               // используется локальная x: y=22
}

```

Считается, что имена аргументов функции объявлены в самом внешнем блоке, относящемся к функции, так что следующий пример

```

void f5 (int x)
{
    int x;               // error (ошибка)
}

```

компилируется с ошибкой, ибо имя *x* дважды определено в одной и той же области видимости. Отнесение такой ситуации к ошибочным позволяет избегать коварных ошибок.

4.9.5. Инициализация

Если инициализирующее выражение присутствует в объявлении, то оно задает начальное значение объекта. Если же оно отсутствует, то глобальные объекты (§4.9.4), объекты, объявленные в пространстве имен (§8.2) и статические локальные объекты (§7.1.2, §10.2.4) (совокупно называются *статическими объектами* — *static objects*) автоматически инициализируются нулевым значением соответствующего типа. Например:

```

int a;                 // означает int a = 0;
double d;              // означает double d = 0.0;

```

Локальные переменные (иногда называемые также *автоматическими объектами* — *automatic objects*) и объекты, создаваемые в свободной памяти (*динамические объекты* или объекты, созданные на куче — *heap objects*), по умолчанию не инициализируются. Например:

```

void f()
{
    int x;              // x не имеет точно определенного значения
    // ...
}

```

Элементы массивов и члены структур инициализируются по умолчанию или не инициализируются в зависимости от того, являются ли соответствующие объекты статическими. Для пользовательских типов можно самостоятельно определить инициализацию по умолчанию (§10.4.2).

Сложные объекты требуют более одного инициализирующего значения. Для инициализации массивов (§5.2.1) и структур (§5.7) в стиле языка C используются *списки инициализирующих значений* (*initializer lists*), ограниченные фигурными скобками. Для инициализации пользовательских типов применяются конструкторы, которые используют обычный функциональный синтаксис, когда через запятую перечисляются их аргументы (§2.5.2, §10.2.3).

Обратите внимание на то, что в объявлениях пустая пара круглых скобок `()` всегда означает функцию (§7.1). Например:

```
int a[] = {1, 2}; // инициализатор массива
Point z(1, 2);    // инициализация в функциональном стиле (вызов конструктора)
int f();          // объявление функции
```

4.9.6. Объекты и леводопустимые выражения (lvalue)

Разрешено выделять память под неименованные объекты и использовать их, и можно осуществлять присваивания странно выглядящим выражениям (например, `*p[a+10]=7`). В результате возникает потребность в названии для «чего-то в памяти». Так мы приходим к фундаментальному определению объекта: *объект* — это непрерывный блок памяти. Соответственно, леводопустимое выражение (*lvalue*) — это выражение, ссылающееся на объект. Исходным смыслом *lvalue* (от английского *left value*) является «нечто, что может стоять в левой части операции присваивания». На самом деле, не каждое *lvalue* может стоять в левой части операции присваивания, ибо некоторые *lvalue* ссылаются на константы (§5.5). Те *lvalue*, которые были объявлены без модификаторов `const`, называются *модифицируемыми lvalue*. Приведенное простое и низкоуровневое понятие объекта не следует путать с высокоуровневыми понятиями классовых объектов и объектов полиморфных типов (§15.4.3).

Если программист не указал ничего иного (§7.1.2, §10.4.8), объявленный в функции объект создается в точке его определения и существует до момента выхода его имени из области видимости (§10.4.4). Такие объекты принято называть автоматическими объектами. Объекты, объявленные глобально или в пространствах имен, или с модификатором `static` в функциях или классах, создаются и инициализируются в работающей программе лишь однажды и живут до тех пор, пока программа не завершит работу (§10.4.9). Такие объекты принято называть статическими. Элементы массивов и нестатические члены классов и структур живут ровно столько, сколько живут содержащие их объекты.

Используя операции *new* и *delete*, можно самостоятельно управлять временем жизни объектов (§6.2.6).

4.9.7. Ключевое слово `typedef`

Объявление, начинающееся с ключевого слова *typedef*, определяет новое имя для *типа*, а не новую переменную какого-либо существующего типа. Например:

```
typedef char* Pchar;
Pchar p1, p2;        // p1 и p2 типа char*
char* p3=p1;
```

Вводимые таким образом имена часто являются короткими синонимами для типов с неудобоваримыми именами. Например, можно ввести для типа *unsigned char* (особенно при его частом использовании) более короткий синоним, *uchar*:

```
typedef unsigned char uchar;
```

Часто *typedef* используется для того, чтобы ограничить прямое обращение к целевому типу единственным местом в программе. Например:


```
typedef int int32;  
typedef short int16;
```

Если теперь использовать тип *int32* всюду, где нужны большие целые, то программу будет легко портировать на системную платформу, для которой *sizeof(int)* равен 2. Действительно, нужно лишь дать новое определение для *int32*:

```
typedef long int32;
```

Хорошо это или плохо, но с помощью *typedef* вводятся лишь новые синонимы существующих типов, а не сами новые типы. Следовательно, эти синонимы можно использовать параллельно именам существующих типов. Если же требуются новые типы, имеющие схожую семантику и/или представление, то следует присмотреться к перечислениям (§4.8) и классам (глава 10).

4.10. Советы

1. Сужайте область видимости имен; §4.9.4.
2. Не используйте одно и то же имя и в текущей области видимости, и в объемлющей области видимости; §4.9.4.
3. Ограничивайте объявление одним именем; §4.9.2.
4. Присваивайте короткие имена локальным и часто используемым переменным; глобальным и редко используемым — длинные; §4.9.3.
5. Избегайте похожих имен; §4.9.3.
6. Придерживайтесь единого стиля именования; §4.9.3.
7. Внимательно выбирайте имена таким образом, чтобы они отражали смысл, а не представление; §4.9.3.
8. Применяйте *typedef* для получения осмысленных синонимов встроенного типа в случаях, когда встроенный тип для представления переменных может измениться; §4.9.7.
9. Синонимы типов задавайте с помощью *typedef*; новые типы вводите с помощью перечислений и классов; §4.9.7.
10. Помните, что любое объявление должно содержать тип явно (никаких «неявных *int*»); §4.9.1.
11. Без необходимости не полагайтесь на конкретные числовые коды символов; §4.3.1, §С.6.2.1.
12. Не полагайтесь на конкретные размеры целых; §4.6.
13. Без необходимости не полагайтесь на диапазон значений чисел с плавающей запятой; §4.6.
14. Предпочитайте тип *int* типам *short int* и *long int*; §4.6.
15. Предпочитайте тип *double* типам *float* и *long double*; §4.5.
16. Предпочитайте тип *char* типам *signed char* и *unsigned char*; §С.3.4.
17. Избегайте необязательных предположений о размерах объектов; §4.6.
18. Избегайте арифметики беззнаковых типов; §4.4.

19. Относитесь с подозрением к преобразованиям из *signed* в *unsigned* и обратным преобразованиям; §С.6.2.6.
20. Относитесь с подозрением к преобразованиям из типов с плавающей запятой в целые; §С.6.2.6.
21. Относитесь с подозрением к преобразованиям из более широких типов в более узкие (например, из *int* в *char*); §С.6.2.6.

4.11. Упражнения

1. (*2) Запустите программу «Hello, world!» (§3.2). Если эта программа не компилируется, обратитесь к §В.3.1.
2. (*1) Для каждого объявления из §4.9 выполните следующее: если объявление не является определением, переделайте его в определение; если же объявление является определением, напишите для него соответствующее объявление, не являющееся одновременно и определением.
3. (*1.5) Напишите программу, которая выводит размеры фундаментальных типов, нескольких типов указателей и нескольких перечислений по вашему выбору. Используйте операцию *sizeof*.
4. (*1.5) Напишите программу, которая выводит буквы '*a*' — '*z*' и цифры '*0*' — '*9*' и их десятичные коды. Повторите все для иных символов, имеющих зрительные образы. Выведите числовые коды в шестнадцатеричном виде.
5. (*2) Каковы на вашей машине минимальные и максимальные значения для следующих типов: *char*, *short*, *int*, *long*, *float*, *double*, *long double* и *unsigned*?
6. (*1) Какова максимальная длина локальных имен на вашей машине? Какова максимальная длина для внешних имен на вашей машине? Есть ли ограничения на символы, которые можно использовать в именах?
7. (*2) Нарисуйте граф для фундаментальных типов, поддерживаемых любой стандартной реализацией (стрелка указывает направление от первого типа ко второму, если все значения первого типа представимы переменными второго типа). Нарисуйте тот же граф, но для вашей любимой реализации.

Указатели, массивы и структуры

*Возвышенное и нелепое
часто столь тесно переплетены,
что их трудно рассматривать порознь.
— Том Пэйн*

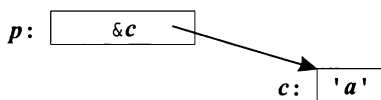
Указатели — нуль — массивы — строковые литералы — константы — указатели и константы — ссылки — **void*** — структуры данных — советы — упражнения.

5.1. Указатели

Для заданного типа T , тип T^* является «указателем на T ». Это означает, что переменные типа T^* содержат адреса объектов типа T . Например:

```
char c = 'a';
char* p = &c;    // p содержит адрес c
```

или в графической форме:



К сожалению, указатели на массивы и указатели на функции требуют более сложных конструкций объявления:

```
int* pi;           // указатель на int
char** ppc;        // указатель на указатель на char
int* ap[15];       // массив из 15 указателей на int
int (*fp)(char*);  // указатель на функцию с аргументом char* и возвратом int
int* f(char*);     // функция с аргументом char* и возвратом "указатель на int"
```

Подробное объяснение этого синтаксиса дано в §4.9.1, а полное изложение грамматики — в Приложении А.

Основной операцией над указателями является *разыменование* (*dereferencing*), то есть обращение к объекту, на который показывает указатель. Эту операцию также называют *косвенным обращением* (*indirection*). Унарная операция разыменования обозначается знаком *, применяемым префиксным образом по отношению к операнду. Например:

```
char c = 'a' ;
char* p = &c;      // p содержит адрес переменной c
char c2 = *p;      // c2 == 'a'
```

Указатель *p* показывает на переменную *c*, значением которой служит 'a'. Таким образом, значение выражения **p* (присваиваемое *c2*) есть 'a'.

Допускается ряд арифметических операций над указателями на элементы массивов (§5.3). Указатели на функции могут быть чрезвычайно полезны; они обсуждаются в §7.7.

Указатели задуманы с целью непосредственного использования механизмов адресации памяти, реализуемых на машинном уровне. Большинство машин умеют адресовать отдельные байты. А те, что не могут, умеют извлекать байты из машинных слов. В то же время, редкие машины в состоянии адресовать отдельные биты. Следовательно, наименьший объект, который можно независимо разместить в памяти и которым можно манипулировать через встроенные указатели, есть объект типа *char*. Стоит отметить, что тип *bool* требует по меньшей мере столько же памяти, что и тип *char* (§4.6). Для более компактного хранения малых объектов, можно использовать побитовые логические операции (§6.2.4) или битовые поля в структурах (§C.8.1).

5.1.1. Нуль

Нуль (0) имеет тип *int*. Благодаря стандартным преобразованиям (§C.6.2.3), нуль можно использовать в качестве константы любого интегрального типа (§4.1.1), типа с плавающей запятой, указателя или указателя на член класса. Тип нуля определяется по контексту. Указатель со значением 0 называется *нулевым указателем* (*null pointer*) и обычно (но не всегда) представим на машинном уровне в виде последовательности нулевых битов соответствующей длины.

Не существует объектов в памяти с адресом нуль. В результате, нуль служит *литералом указательного типа*, означающим, что указатель с таким значением не ссылается ни на какой объект.

В языке C было принято использовать макроконстанту *NULL* для обозначения нулевых указателей. Из-за более плотного контроля типов в языке C++, использование простого 0 вместо так или иначе определенной макроконстанты *NULL* приводит к меньшим проблемам. Если вы чувствуете, что вам все же нужен *NULL*, воспользуйтесь

```
const int NULL = 0;
```

Модификатор *const* (§5.4) предотвращает случайное изменение *NULL* и гарантирует, что *NULL* можно использовать всюду, где требуется константа.

5.2. Массивы

Для заданного типа T , тип $T[size]$ является «массивом из $size$ элементов типа T ». Индексация (нумерация) элементов требует индексов от 0 и до $size-1$. Например:

```
float v[3];           // массив из трех элементов типа float: v[0], v[1], v[2]
char* a[32];         // массив из 32 указателей на char: a[0] .. a[31]
```

Количество элементов массива (размер массива) должно задаваться константным выражением (§C.5). Если же нужен массив переменного размера, воспользуйтесь типом **vector** (§3.7.1, §16.3). Например:

```
void f(int i)
{
    int v1[i];           // ошибка: размер массива не константное выражение
    vector<int> v2(i);    // ok (все нормально)
}
```

Многомерные массивы определяются как массивы массивов. Вот соответствующий пример:

```
int d2[10][20];        // d2 есть массив из 10 массивов по 20 целых в каждом
```

Если здесь, как это принято во многих других языках программирования, отделять запятой различные размеры массива, то возникнет ошибка компиляции, ибо *запятая* в языке C++ обозначает *операцию следования* (*sequencing operator*, §6.2.2), и которая неприменима в константных выражениях (§C.5). Можете проверить это на следующем примере:

```
int bad[5, 2];         // error: запятая недопустима в константных выражениях
```

Многомерные массивы рассматриваются в §C.7. В коде высокого уровня их лучше вообще избегать¹.

5.2.1. Инициализация массивов

Массив можно проинициализировать списком значений. Например:

```
int v1[] = {1, 2, 3, 4};
char v2[] = {'a', 'b', 'c', 0};
```

Когда массив объявляется без явного указания размера, но со списком инициализирующих значений, *размер вычисляется по количеству этих значений*. Следовательно, $v1$ и $v2$ имеют типы $int[4]$ и $char[4]$, соответственно. Когда размер массива указан явно, а в инициализирующем списке присутствует большее количество значений, то возникает ошибка компиляции. Например:

```
char v3[2] = {'a', 'b', 0}; // error: слишком много инициализаторов
char v4[3] = {'a', 'b', 0}; // ok
```

Если же в инициализирующем списке значений не хватает, то «оставшимся не у дел» элементам массива присваивается 0 . Например:

¹ Трудно согласиться с данным тезисом автора — разве матричная математика не относится к высокоуровневому коду? — *Прим. ред.*

```
int v5[8] = {1, 2, 3, 4};
```

Эквивалентно

```
int v5[] = {1, 2, 3, 4, 0, 0, 0, 0};
```

Заметим, что *присваивания списком значений для массивов не существует*:

```
void f()
{
    v4 = {'c', 'd', 0}; // error: такое присваивание для массивов недопустимо
}
```

Если нужно подобное присваивание, воспользуйтесь стандартными типами **vector** (§16.3) или **valarray** (§22.4).

Массив символов удобно инициализировать строковыми литералами (§5.2.2).

5.2.2. Строковые литералы

Строковым литералом (string literal) называется последовательность символов, заключенная в двойные кавычки.

```
"this is a string"
```

Строковые литералы содержат *на один символ больше*, чем это кажется на первый взгляд: они оканчиваются нулевым символом '0', числовое значение которого равно нулю. Например:

```
sizeof("Bohr") == 5
```

Тип строкового литерала есть «массив соответствующего количества константных символов», так что литерал "**Bohr**" имеет тип **const char[5]**.

*Строковый литерал допускается присваивать указателю типа char**, так как в более ранних версиях языков C и C++ типом строковых литералов был **char***. Благодаря этому допущению миллионы строк ранее написанного на C и C++ кода остаются синтаксически корректными. Тем не менее, попытка модификации строкового литерала через такой указатель ошибочна:

```
void f()
{
    char* p = "Plato";
    p[4] = 'e'; // error: присваивание константе; результат не определен
}
```

Такого рода ошибки обычно не выявляются до стадии выполнения программы, и к тому же, разные реализации по-разному реагируют на нарушение этого правила (см. также §B.2.3). Очевидность константного характера строковых литералов позволяет компиляторам оптимизировать как методы хранения этих литералов, так и методы доступа к ним.

Если нам нужна строка с гарантированной возможностью модификации, то нужно скопировать символы в массив:

```
void f()
{
    char p[] = "Zeno"; // p есть массив из 5 char
}
```

```
p[0] = 'R';      // ok
}
```

Память под строковые литералы выделяется *статически*, поэтому их возврат из функций *безопасен*. Например:

```
const char* error_message(int i)
{
    //...
    return "range error";
}
```

Память, содержащая строку *"range error"*, никуда не денется после выхода из функции *error_message()*.

От конкретной реализации компилятора зависит, будут ли два одинаковых строковых литерала сосредоточены в одном и том же месте памяти (§С.1). Например:

```
const char* p = "Heraclitus";
const char* q = "Heraclitus";

void g()
{
    if(p==q) cout<<"one!\n"; // результат зависит от конкретной реализации
}
```

Обратите внимание на то, что для указателей операция *==* сравнивает адреса (значения указателей), а не адресуемые ими величины.

Пустая строка (empty string) записывается в виде *смежных двойных кавычек* (*""*) и имеет тип *const char[1]*.

Синтаксис, использующий обратную косую черту для обозначения специальных символов (§С.3.2), применим и в строковых литералах. Это позволяет вносить в содержимое строковых литералов и двойные кавычки (*"*), и символ обратной косой черты (**). Чаще всего, конечно, используется символ *'\n'* для принудительного перевода строки вывода. Например:

```
cout<<"beep at the end of message\a\n";
```

Символ *'a'* из ASCII-таблицы называется *BEL* (звонок); он приводит к подаче звукового сигнала.

Переходить же на новую строку исходного текста программы при записи содержимого строкового литерала *нельзя*:

```
"this is not a string
but a syntax error"
```

С целью улучшения читаемости текста программы длинные строковые литералы можно разбить на фрагменты, отделяемые друг от друга лишь пробельными символами¹. Например:

```
char alpha[] = "abcdefghijklmnopqrstuvwxy"
               "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

¹ В том числе, и переходом на новую строку текста программы. — Прим. ред.

Компилятор объединяет смежные строковые литералы в один литерал, так что переменную *alpha* можно было бы с тем же успехом инициализировать единственной строкой:

```
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

В состав строк можно включать нулевые символы, но большинство функций и не догадываются о том, что после нулей в них есть что-то еще. Например, строка **"Jens\000Munk"** будет трактоваться как **"Jens"** такими библиотечными функциями, как *strcpy()* или *strlen()* (см. §20.4.1).

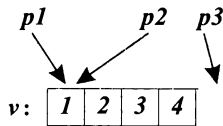
Строки с префиксом **L**, например **L"angst"**, состоят из символов типа *wchar_t* (§4.3, §C.3.3). Тип этих строк есть *const wchar_t[]*.

5.3. Указатели на массивы

В языке C++ указатели и массивы тесно связаны. Имя массива может быть использовано в качестве указателя на его первый элемент. Например:

```
int v[] = {1, 2, 3, 4};
int* p1 = v;           // указатель на начальный элемент (неявное преобразование)
int* p2 = &v[0];       // указатель на начальный элемент
int* p3 = &v[4];        // указатель на элемент, следующий за последним
```

или в графической форме:



Гарантируется возможность настройки указателя на «элемент, расположенный за последним элементом массива». Это важный момент для многих алгоритмов (§2.7.2, §18.3). Однако поскольку такой указатель на самом деле не указывает ни на какой элемент массива, его нельзя (не стоит) использовать ни для записи, ни для чтения. Возможность получения осмысленного адреса «элемента, расположенного перед первым элементом массива», не гарантируется и таких действий следует избегать. В рамках некоторых машинных архитектур память под массивы может выделяться в самом начале адресного пространства, так что адреса «элемента, расположенного перед первым элементом массива» может просто не существовать.

Неявные преобразования имени массива в указатель на его первый элемент широко используются при вызовах функций в C-стиле. Например:

```
extern "C" int strlen(const char*); // из <string.h>

void f()
{
    char v[] = "Annemarie";
    char* p = v;                     // неявное преобразование от char[] к char*
    strlen(p);
    strlen(v);                       // неявное преобразование от char[] к char*
    v = p;                           // error: неверное присваивание массиву
}
```


Здесь в обоих вызовах стандартной библиотечной функции *strlen*() передается одно и то же адресное значение. Заковыка в том, что избежать такого неявного преобразования нельзя, так как невозможно объявить функцию, при вызове которой копировался бы сам массив *v*. К счастью, не существует обратного (явного и неявного) преобразования от указателя к массиву.

Неявное преобразование от массива к указателю, имеющее место при вызове функции, означает, что при этом *размер массива теряется*. Ясно, что, тем не менее, функция должна каким-то образом узнать размер массива, иначе невозможно будет выполнить осмысленные действия над ним. Как и другие функции из стандартной библиотеки языка C, получающие указатель на символьные строки, функция *strlen*() полагается на *терминальный ноль* в качестве *индикатора конца строки*; *strlen(p)* возвращает количество символов в строке вплоть до, но не считая терминального *нуля*. Все это достаточно низкоуровневые детали. Типы *vector* (§16.3) и *string* (глава 20) из стандартной библиотеки не страдают этими проблемами.

5.3.1. Доступ к элементам массивов

Эффективный и элегантный доступ к элементам массивов (и другим структурам данных) является ключевым фактором для многих алгоритмов (§3.8, глава 18). Такой доступ можно осуществлять либо через указатель на начало массива и индекс элемента, либо непосредственно по указателю на элемент. Например, следующий код, применяющий индексы для прохода по символам строки

```
void fi(char v[])
{
    for(int i = 0; v[i] != 0; i++)
        // используется v[i]
}
```

эквивалентен коду, применяющему с той же целью указатель:

```
void fp(char v[])
{
    for(char* p = v; *p != 0; p++)
        // используется *p;
}
```

Унарная операция разыменования (операция ***) действует так, что **p* есть символ, адресуемый указателем *p*, а операция *++* *инкрементирует значение указателя* с тем, чтобы он показывал на *следующий элемент массива*.

Не существует фундаментальных причин, по которым одна из этих версий кода была бы эффективнее другой. Современные компиляторы должны превращать оба варианта в одинаковые фрагменты машинного кода (§5.9[8]). Программисты должны осуществлять выбор между ними, исходя из логических или эстетических соображений.

Результат воздействия на указатели арифметических операций *+*, *-*, *++* и *--* зависит от типа объектов, на которые указатель ссылается. Когда арифметическая операция применяется к указателю *p* типа *T**, предполагается, что *p* указывает на элемент массива объектов типа *T*; *p+1* указывает на следующий элемент этого массива, а *p-1* — на предыдущий элемент. Это подразумевает, что адресное (целочисленное)

значение выражения $p+1$ на `sizeof(T)` больше, чем величина указателя p . Например, результатом выполнения кода

```
#include <iostream>

int main ()
{
    int vi[10];
    short vs[10];

    std::cout<< &vi[0] <<' ' <<&vi[1] <<' \n';
    std::cout<< &vs[0] <<' ' <<&vs[1] <<' \n';
}
```

будет

```
0x7fffaef0 0x7fffaef4
0x7fffaedc 0x7fffaede
```

при использовании принятой по умолчанию шестнадцатеричной формы записи значений указателей. Из данного примера видно, что в моей реализации `sizeof(short)` есть 2, а `sizeof(int)` есть 4.

Вычитание указателей друг из друга определено для случая, когда оба указателя показывают на элементы одного и того же массива (язык, правда, не предоставляет быстрых способов проверки этого факта). Результатом вычитания указателей будет количество элементов массива (целое число), расположенных между указуемыми элементами. К указателю можно прибавить или вычесть целое; в обоих случаях результатом будет указатель. При этом, если полученный таким образом указатель не указывает на один из элементов того же массива (или на элемент, расположенный за последним элементом массива), то результат его применения не определен. Например:

```
void f()
{
    int v1[10];
    int v2[10];

    int i1 = &v1[5] - &v1[3]; // i1 = 2
    int i2 = &v1[5] - &v2[3]; // результат не определен

    int* p1 = v2+2;           // p1 = &v2[2];
    int* p2 = v2-2;           // *p2 не определен
}
```

Обычно, в использовании более-менее сложной арифметики указателей необходимости нет, и ее лучше избегать. *Сложение указателей не имеет смысла вообще*; в итоге оно запрещено.

Массивы не являются самоописываемыми сущностями, так как их размеры не хранятся вместе с ними. Это предполагает, что для успешного перемещения по элементам массива нужно так или иначе получить его размер. Вот пример на эту тему:

```
void fp(char v[], unsigned int size)
{
    for(int i=0; i<size; i++)
        // используется v[i]

    const int N = 7;
    char v2[N];
    for(int i=0; i<N; i++)
        // используется v2[i]
}
```

Заметим, что большинство реализующих язык C++ программных средств не предоставляет никаких способов контроля границ массива. Такая концепция массивов весьма низкоуровневая. Более интеллектуальные концепции массивов можно реализовать с помощью классов (§3.7.1).

5.4. Константы

В языке C++ реализована концепция определяемых пользователем констант, явно отражающая тот факт, что значение нельзя изменять непосредственно. Это полезно во многих отношениях. Например, существуют объекты, которые не требуют изменений после их инициализации; использование символических констант обеспечивает более удобное сопровождение кода, чем в случае непосредственного применения литералов в коде программы; указатели часто используются только для чтения, а не для записи; большинство параметров функций предназначены лишь для чтения, а не для перезаписи.

Ключевое слово **const** используется для объявления константного объекта. Так как константным объектам присваивать значения нельзя, то они в *обязательном порядке* должны инициализироваться. Например:

```
const int model = 90;           // model является константой
const int v[] = {1, 2, 3, 4}; // v[i] являются константами
const int x;                   // error: нет инициализатора
```

Объявление объекта константным гарантирует, что значение объекта не изменится в его области видимости:

```
void f()
{
    model = 200;                // error
    v[2]++;                     // error
}
```

Заметим, что ключевое слово **const** модифицирует именно *тип*, то есть оно ограничивает способы использования объекта, а не его размещение в памяти. Например:

```
void g(const X* p)
{
    // здесь нельзя модифицировать *p
}
```

```
void h ()
{
    X val;           // val можно изменять
    g (&val) ;
    // ...
}
```

В зависимости от своего «интеллекта», конкретные компиляторы могут по-разному использовать факт константности объекта. Например, инициализатор константного объекта часто (но не всегда) является константным выражением (§C.5); если это так, то его можно вычислить во время компиляции. Далее, если компилятор может выявить все случаи использования константы, то он может не выделять под нее память. Например:

```
const int c1 = 1;
const int c2 = 2;
const int c3 = my_f(3) ; // значение c3 не известно в момент компиляции
extern const int c4;      // значение c4 не известно в момент компиляции
const int* p = &c2;      // под c2 нужно выделить память
```

Так как компилятор знает значения переменных *c1* и *c2*, он может использовать их в константных выражениях. Поскольку значения для *c3* и *c4* во время компиляции неизвестны (исходя лишь из информации в данной единице компиляции; см. §9.1), для них требуется выделить память. Адрес переменной *c2* вычисляется (и где-то, наверное, используется), и для нее память выделять тоже нужно. Самым простым и часто встречающимся случаем является ситуация, когда значение константы во время компиляции известно и память под нее не выделяется; *c1* служит примером такого случая. Ключевое слово **extern** указывает, что переменная *c4* определена где-то в другом месте программы (§9.2).

Как правило, для массива констант память всегда выделяется, так как компилятор не в состоянии выявить, какие именно элементы массива используются в выражениях. Впрочем, на многих машинах и в этом случае можно достичь увеличения эффективности за счет помещения таких массивов в области памяти с атрибутом «только для чтения».

Константы часто используются в качестве размера массивов и в качестве меток case-ветвей оператора **switch**. Например:

```
const int a = 42;
const int b = 99;
const int max = 128;

int v[max] ;

void f(int i)
{
    switch (i)
    {
        case a :
            // ...
        case b :
```

В подобного рода случаях часто альтернативой константам служат перечисления (§4.8).

Как ключевое слово `const` применяется к функциям-членам классов, рассказывается в §10.2.6 и §10.2.7.

Символические константы следует использовать систематически, дабы избежать появления «магических чисел» в программном коде. Если такое число обозначает, например, размер массива, то будучи многократно продублированным непосредственно в коде, оно вызовет серьезные затруднения при модификации программы, так как потребуются выявить и правильно изменить каждое его вхождение. Использование символических констант локализует информацию. Часто числовые константы означают некоторые предположения о работе программы. Например, число **4** может означать количество байт в целом типе, **128** — емкость буфера ввода, а **6.24** — обменный курс датской кроны по отношению к доллару США. Тому, кто впоследствии сопровождает программу, понять смысл числовых литералов бывает довольно трудно. Часто на такие числа не обращают должного внимания при переносе программ в иные среды, или при иных модификациях программ, нарушающих первоначальные предположения, что в итоге приводит к весьма неприятным ошибкам. Хорошо прокомментированные символические константы сводят указанные проблемы к минимуму.

5.4.1. Указатели и константы

В операциях с указателями участвуют сразу два объекта: указатель и объект, на который он ссылается. *Помещение ключевого слова `const` в начало объявления делает константным объект, а не указатель.* Для объявления указателя константой нужно применить декларатор ****const*** вместо просто звездочки. Например:

```
void f1(char* p)
{
    char s[] = "Gorm";

    const char* pc = s;           // указатель на константу
    pc[3] = 'g';                 // error: pc указывает на константу
    pc = p;                      // ok

    char* const cp = s;          // константный указатель
    cp[3] = 'a';                 // ok
    cp = p;                      // error: cp есть константа

    const char* const cpc = s;   // константный указатель на константу
    cpc[3] = 'a';               // error: cpc указывает на константу
    cpc = p;                    // error: cpc есть константа
}
```

Подчеркнем, что константой указатель делает именно декларатор ****const***. Никакого декларатора ***const**** не существует, так что ключевое слово ***const***, расположенное перед звездочкой, относится к базовому типу. Например:

```
char* const cp;                 // константный указатель на char
char const* pc;                 // указатель на константу типа char
const char* pc2;               // указатель на константу типа char
```

Некоторые люди находят удобным читать такие объявления справа налево. Например, «*cp* это константный указатель на *char*», или «*pc2* есть указатель на *char* константный».

Объект может быть константным при обращении к нему через некоторый указатель, и обычной переменной при иных способах доступа. Это широко используется в контексте параметров функций. Если у функции параметр-указатель объявлен константным, то функция лишается возможности изменять адресуемое им значение. Например:

```
char* strcpy (char* p, const char* q) ; // не может модифицировать *q
```

Вы можете присвоить адрес переменной указателю на константу, поскольку от такого присваивания не будет никакого вреда. Противоположное неверно: нельзя присвоить адрес константы обычному (неконстантному) указателю, ибо в противном случае с его помощью можно было бы изменить значение константы. Например:

```
void f4 ()
{
    int a = 1;
    const int c = 2;
    const int* p1 = &c;           // ok
    const int* p2 = &a;           // ok

    int* p3 = &c;                 // error: инициализация переменной типа int*
                                // значением типа const int*
    *p3 = 7;                     // попытка изменить значение константы c
}
```

Снять ограничения на применение указателей на константы можно с помощью явного приведения типа (§10.2.7.1 и §15.4.2.1).

5.5. Ссылки

Ссылка (*reference*) является альтернативным именем объекта. Ссылки чаще всего используются для объявления параметров и возвращаемых значений у функций вообще, и у перегруженных операций (глава 11) в частности. Обозначение *X&* означает ссылку на *X* (*reference to X*). Например:

```
void f ()
{
    int i = 1;
    int& r = i;                 // r и i теперь ссылаются на одно и то же целое
    int x = r;                  // x = 1
    r = 2;                      // i = 2
}
```

Чтобы гарантировать привязанность ссылки к некоторому объекту, мы обязаны ее инициализировать. Например:

```
int i = 1;
int& r1 = i;                 // ok: r1 инициализировано
```

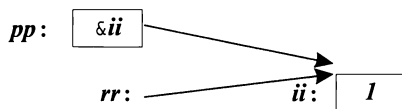
```
int& r2;                // error: отсутствует инициализатор
extern int& rr3;         // ok: r3 инициализируется в другом месте
```

Инициализация ссылок это не то же самое, что присваивание. Несмотря на обманчивую видимость, *ни одна операция не выполняется над самой ссылкой*. Наприме-

```
void g ()
{
    int ii = 0;
    int& rr = ii;
    rr++;                // ii увеличивается на 1
    int* pp = &rr;       // pp указывает на ii
}
```

Здесь все выражения допустимы, но в результате `rr++` увеличивается не ссылка `rr`, а адресуемая этой ссылкой переменная `ii` целого типа. Следовательно, *значение ссылки никогда не меняется после инициализации*; она всегда ссылается на объект, с которым была проинициализирована. Чтобы получить указатель на объект, с которым связана ссылка `rr`, можно использовать выражение `&rr`.

Очевидной гипотетической реализацией ссылки может служить (константный) указатель, который автоматически (неявно) разыменуете при использовании. Большого вреда в такой интерпретации нет, особенно если всегда помнить, что над ссылками нельзя выполнять операции так же, как над указателями:



В ряде случаев компилятор может так оптимизировать код, что в момент исполнения программы ей не будет соответствовать никакого объекта в памяти.

Инициализация ссылок тривиальна, когда инициализатор представляет из себя *lvalue* (объект, адрес которого можно получить; §4.9.6). Инициализатором для типа `T&` должен быть *lvalue* типа `T`.

Инициализатор для типа `const T&` не обязан следовать этому правилу и даже может вообще не иметь тип `T`. В таких случаях:

1. Если необходимо, выполняется неявное преобразование к типу `T` (§C.6).
2. Результирующее значение помещается во временную переменную типа `T`.
3. Временная переменная используется в качестве инициализатора.

Рассмотрим пример:

```
double& d = 1;          // error: требуется lvalue
const double& cdr = 1;   // ok
```

Можно интерпретировать последнюю инициализацию следующим образом:

```
double temp = double(1); //создаем временную переменную с нужным значением
const double& cdr = temp; //затем используем ее для инициализации cdr
```

Временная переменная, созданная для хранения инициализатора ссылки, существует до тех пор, пока ссылка не выйдет из своей области видимости.

Видно, что ссылки на константы и ссылки на переменные серьезно различаются, так как введение временных переменных в последнем случае приводило бы к неприятным ошибкам: изменение исходной переменной влечет за собой изменение быстро исчезающей временной переменной. В случае ссылок на константы таких проблем нет. Ссылки на константы находят широкое применение в качестве аргументов функций (§11.6).

Ссылки можно использовать для объявления аргументов функций с тем, чтобы функции могли изменять значения передаваемых им объектов. Например:

```
void increment (int& aa) { aa++; }

void f()
{
    int x = 1;
    incremen (x);    // x = 2
}
```

Семантика передачи аргументов аналогична инициализации, так что при вызове функции **increment**() аргумент **aa** становится другим именем для **x**. Для улучшения читаемости программ лучше избегать функций, изменяющих свои аргументы. Предпочтительнее использовать явным образом возврат функции или объявить аргумент указателем:

```
int next (int p) { return p+1; }
void incr (int* p) { (*p)++; }

void g ()
{
    int x = 1;
    incremen (x);    // x = 2
    x = next (x);    // x = 3
    incr (&x);       // x = 4
}
```

Внешний вид вызова **increment (x)** не дает читателю и намек на то, что значение **x** будет изменено, в отличие от **x = next (x)** и **incr (&x)**. Так что ссылочные аргументы уместно использовать лишь в функциях, чьи имена прозрачно намекают на возможность модификации аргументов.

Ссылки также применяются для определения функций, которые можно использовать и в правой, и в левой частях операции присваивания. Как всегда, наиболее интересные примеры, демонстрирующие столь хитрое поведение функций, сосредоточены в рамках нетривиальных пользовательских типов. Давайте для примера определим сейчас простой ассоциативный массив. Сначала задаем структуру **Pair** следующим образом:

```
struct Pair
{
    string name;
    double val;
};
```

Общая идея заключается в том, что строка имеет ассоциированное с ней числовое значение с плавающей запятой. Несложно определить функцию **value()**, кото-

рая поддерживает структуру данных, содержащую ровно один элемент типа *Pair* для каждой уникальной строки, переданной этой функции. Чтобы не затягивать рассмотрение вопроса, приведем очень простую (и неэффективную) реализацию:

```
vector<Pair> pairs;

double& value (const string& s)
/*
  поддерживает набор пар Pair:
  ищет строку s; если найдена, возвращает соответствующее значение;
  в противном случае создает новый Pair и возвращает 0. См. также §11.8.
*/
{
  for (int i = 0; i < pairs.size(); i++)
    if (s == pairs[i].name) return pairs[i].val;

  Pair p = {s, 0};
  pairs.push_back(p); // добавить Pair в конец (§3.7.3)
  return pairs[pairs.size() - 1].val;
}
```

Здесь речь идет о построении массива чисел, индексируемых строками. Для заданного строкового аргумента функция *value()* находит соответствующий числовой объект (именно объект, а не его значение) и возвращает ссылку на него. Вот пример использования функции *value()*:

```
int main () // посчитать кол-во вхождений каждого слова во входном потоке
{
  string buf;
  while (cin >> buf) value(buf)++;
  for (vector<Pair>::const_iterator p = pairs.begin(); p != pairs.end(); ++p)
    cout << p->name << ": " << p->val << '\n';
}
```

Здесь в каждой итерации цикла *while* из стандартного потока ввода *cin* читается одно слово и записывается в строковый буфер *buf()*, после чего обновляется значение связанного с этим словом счетчика. В конце печатается результирующая таблица введенных слов вместе с числами их повторов. Например, если на входе были слова

```
aa bb bb aa aa bb aa aa
```

то программа выдаст следующую статистическую информацию:

```
aa: 5
bb: 3
```

Несложно переделать представленное решение в настоящий ассоциативный массив на базе классового шаблона с перегруженной операцией *[]* (§11.8). Еще проще воспользоваться типом *map* из стандартной библиотеки (§17.4.1).

5.6. Тип `void*`

Переменной типа `void*` можно присвоить значение указателя любого типа; одной переменной типа `void*` можно присваивать значение другой переменной этого типа, а также сравнивать их между собой на равенство (или неравенство); тип `void*` можно явным образом преобразовывать в указатели иных типов. Другие операции с типом `void*` опасны, так как компилятор не знает типа адресуемых объектов, и поэтому такие операции вызывают ошибку компиляции. Чтобы воспользоваться указателем типа `void*`, его нужно явно преобразовать к иным типам указателей. Например:

```
void f(int* pi)
{
    void* pv = pi;           // ok: неявное преобразование из int* в void*
    *pv;                     // error: нельзя разыменовать void*
    pv++;                     // error: нельзя инкрементировать void*
                              // (неизвестен размер указуемого объекта)

    int* pi2 = static_cast<int*>(pv); // явное преобразование в int*

    double* pd1 = pv;         // error
    double* pd2 = pi;         // error
    double* pd3 = static_cast<double*>(pv); // небезопасно
}
```

В общем случае небезопасно использовать указатель, приведенный к типу, отличному от типа адресуемого объекта. К примеру, под тип `double` на многих машинах отводится 8 байт. Тогда работа с указателем `pd3` будет непредсказуемой, так как `pi` указывает на блок памяти, выделенный под `int` (обычно 4 байта). Операция `static_cast` для явного преобразования типов указателей *внутренне опасна* и внешне выглядит «безобразно», но так и было задумано, чтобы своим внешним видом она напоминала о реальной опасности преобразования.

В основном, тип `void*` применяется в параметрах функций, которым не позволено делать предположения о типе адресуемых объектов, и для возврата из функций «бестиповых» объектов. Чтобы воспользоваться таким объектом, нужно явно преобразовать тип указателя.

Функции, использующие `void*`, типичны для самых нижних слоев системного обеспечения, где ведется работа с аппаратными ресурсами компьютера. Например:

```
void* my_alloc(size_t n); // выделить n байт из моей специальной кучи
```

К наличию `void*` на более высоких уровнях программного обеспечения следует относиться с подозрением — скорее всего это следствие ошибок проектирования. Если `void*` используется для оптимизации, то его лучше скрыть за фасадом безопасного интерфейса (§13.5, §24.4.2).

Указатели на функции (§7.7) и указатели на члены классов (§15.5) не могут присваиваться переменным типа `void*`.

5.7. Структуры

Массив — это агрегат (набор) элементов одинакового типа. А *структура* — это *агрегат элементов* (почти) *произвольных типов*. Например:

```
struct address
{
    char* name;           // "Jim Dandy"
    long int number;      // 61
    char* street;         // "South St"
    char* town;           // "New Providence"
    char state[2];        // 'N' 'J'
    long zip;             // 7974
};
```

Здесь определяется *новый тип* с именем **address**, состоящий из полей, которые необходимо заполнить для отправки письма. Обратите внимание на *точку с запятой в конце определения*. Для языка C++ это довольно редкий случай, когда после закрывающей фигурной скобки нужно ставить точку с запятой, так что люди часто забывают о ней, а это вызывает *ошибку компиляции*.

Переменные типа **address** можно объявлять точно так же, как и другие переменные, а индивидуальные поля (*члены* — *members*) достижимы с помощью операции (*операция точка* — *dot operator*). Например:

```
void f()
{
    address jd;
    jd.name = "Jim Dandy";
    jd.number = 61;
}
```

Конструкция, использованная нами ранее для инициализации массивов (§5.2.1) применима и для инициализации переменных структурных типов. Например:

```
address jd = {"Jim Dandy", 61, "South St",
             "New Providence", {'N', 'J'}, 7974};
```

Впрочем, конструкторы обычно подходят лучше (§10.2.3). Заметьте, что **jd.state** нельзя инициализировать строкой **"NJ"**. Строки оканчиваются терминальным символом **'\0'**. Следовательно, **"NJ"** содержит три символа — на один больше, чем можно разместить в **jd.state**.

К объектам структурных типов часто обращаются с помощью указателей, используя операцию **->** (*операция разыменования указателей на структуры* — *structure pointer dereference operator*). Например:

```
void print_addr(address* p)
{
    cout<< p->name << '\n'
         << p->number << ' ' << p->street << '\n'
         << p->town << '\n'
         << p->state[0] << p->state[1] << ' ' << p->zip << '\n';
}
```

Для указателя p выражение $p \rightarrow m$ эквивалентно выражению $(*p).m$.

Объекты структурных типов можно присваивать, передавать в функции и возвращать из функций. Например:

```
address current;

address set_current (address next)
{
    address prev = current;
    current = next;
    return prev;
}
```

Другие операции, например операции сравнения ($=$ и $!=$), не определены. Однако пользователь (программист) может определить их сам (глава 11).

Размер объекта структурного типа не обязательно равен сумме размеров полей структуры, так как для разных машинных архитектур нужно в обязательном порядке или целесообразно (для эффективности) располагать объекты определенных типов, выровненными по заданным границам. Например, целые часто располагаются на границах машинных слов. При этом говорят, что объекты надлежащим образом выровнены (*aligned*) в памяти компьютера. Это приводит к «дырам» в структурах. Например, для многих машин окажется, что `sizeof(address)` равен 24, а не 22, как можно было бы ожидать. Можно попытаться минимизировать неиспользуемое пространство, отсортировав поля по убыванию их размера. Однако все же лучше упорядочивать поля структур из соображений наглядности, а сортировку по размеру производить лишь в случае жестких требований по оптимизации.

Имя нового типа можно использовать сразу же после его появления, а вовсе не после его полного определения. Например:

```
struct Link
{
    Link* previous;
    Link* successor;
};
```

До окончания полного определения типа нельзя объявлять объекты или поля этого типа. Например:

```
struct No_good
{
    No_good member; // error: рекурсивное определение
};
```

Тут возникает ошибка компиляции, ибо компилятор не знает, сколько памяти нужно отвести под `No_good`. Чтобы позволить двум (и более) структурным типам ссылаться друг на друга, достаточно объявить, что некоторое имя является именем структурного типа. Например:

```
struct List;           // определение будет дано ниже

struct Link
{
    Link* pre;
```

```

    Link* suc;
    List* member_of;
};

struct List
{
    Link* head;
};

```

Без предварительного объявления имени **List** использование этого имени в объявлении структуры **Link** привело бы к ошибке компиляции.

Именем структурного типа можно пользоваться до его полного определения в случаях, когда нет обращений к полям структуры или нет необходимости знать размер типа. Например:

```

class S;           // S - это имя некоторого типа

extern S a;
S f();
void g(S);
S* h(S*);

```

Тем не менее, многим объявлениям требуется полное определение типа:

```

void k(S* p)
{
    S a;           // error: S не определен и его размер неизвестен
    f();           // error: S не определен, а его размер нужен
                  // для возвращаемого значения

    g(a);          // error: S не определен, а его размер нужен для передачи аргумента
    p->m = 7;       // error: S не определен и его члены неизвестны
    S* q = h(p);   // ok: с указателями проблем нет
    q->m = 7;       // error: S не определен и его члены неизвестны
}

```

Структуры являются упрощенными классами (глава 10).

По причинам, уходящим корнями глубоко в предысторию языка C, разрешается совмещать имена структурного типа и иных программных конструкций в рамках одной и той же области видимости. Например:

```

struct stat { /* ... */ };
int stat(char* name, struct stat* buf);

```

В этом случае, просто имя **stat** относится к неструктурным конструкциям, а имя структуры должно предваряться ключевым словом **struct**. Аналогично, требуется использовать ключевые слова **class**, **union** (§C.8.2) и **enum** (§4.8) с целью преодоления возникающих неоднозначностей. Лучше, однако, избегать такого конфликта имен.

5.7.1. Эквивалентность типов

Две структуры являются разными типами, даже если у них поля одинаковые. Например, структуры

```
struct S1 {int a; };  
struct S2 {int a; };
```

являются разными типами, так что следующий код ошибочен:

```
S1 x;  
S2 y = x;           // error: несоответствие типов
```

Структуры разнятся также и от фундаментальных типов:

```
S1 x;  
int i = x;           // error: несоответствие типов
```

Каждая *структура* должна иметь единственное определение в программе (§9.2.3).

5.8. Советы

1. Избегайте нетривиальной арифметики указателей; §5.3.
2. Внимательно отслеживайте потенциальную возможность выхода за границы массива; §5.3.1.
3. Используйте `0` вместо `NULL`; §5.1.1.
4. Используйте типы ***vector*** и ***valarray*** вместо встроенных (в C-стиле) массивов; §5.3.1.
5. Используйте ***string***, а не массивы ***char*** с терминальным нулем; §5.3.
6. Минимизируйте применение простых ссылок для аргументов функций; §5.5.
7. Применяйте ***void**** лишь в низкоуровневом коде; §5.6.
8. Избегайте нетривиальных литералов («магических чисел») в коде — используйте символические макроконстанты; §4.8, §5.4.

5.9. Упражнения

1. (*1) Напишите следующие объявления: указатель на символ, массив из 10 целых, ссылка на массив из 10 целых, указатель на массив строк, указатель на указатель на символ, целая константа, указатель на целую константу, константный указатель на целое. Проинициализируйте все объекты.
2. (*1.5) Каковы на вашей системе ограничения на значения указателей типа ***char****, ***int**** и ***void****? Например, может ли ***int**** иметь нечетное значение (подсказка: подумайте о выравнивании)?
3. (*1) Используйте ***typedef*** для определения типов: ***unsigned char***, ***const unsigned char***, указатель на целое, указатель на указатель на ***char***, указатель на массив ***char***, массив из 7 указателей на ***int***, указатель на массив из 7 указателей на ***int*** и массив из 8 массивов по 7 указателей на целые.

4. (*1) Напишите функцию, которая обменивает значения у двух целых чисел. Используйте аргументы типа *int**. Напишите другой вариант с аргументами типа *int&*.
5. (*1.5) Каков размер массива *str* в следующем примере:

```
char str[] = "a short string";
```


Какова длина строки "a short string"?
6. (*1) Определите функции *f(char)*, *g(char&)* и *h(const char&)*. Вызовите их с аргументами 'a', 49, 3300, *c*, *uc* и *sc*, где *c* — это *char*, *uc* — *unsigned char*, *sc* — *signed char*. Какие из этих вызовов допустимы? В каких вызовах компилятор создаст временные переменные?
7. (*1.5) Определите таблицу имен месяцев и количества дней в них. Выведите эту таблицу. Прodelайте это дважды: первый раз воспользуйтесь массивом элементов типа *char* для названия месяца и массивом типа *int* для количества дней; второй раз примените массив структур, которые содержат и названия месяцев, и число дней.
8. (*2) Выполните тестовые прогоны, чтобы на практике убедиться в эквивалентности кода для случаев итерации по элементам массива с помощью указателей и с помощью индексации (§5.3.1). Если у компилятора есть разные степени оптимизации, убедитесь, влияет ли это (и как) на качество генерируемого машинного кода.
9. (*1.5) Продемонстрируйте пример, когда имеет смысл использовать объявляемое имя в инициализаторе.
10. (*1) Определите массив строк, содержащих названия месяцев. Распечатайте эти строки. Для печати передайте этот массив в функцию.
11. (*2) Читайте последовательность слов из потока ввода. Пусть слово *Quit* служит признаком конца ввода. Печатайте слова в порядке, в каком они были введены, но не допускайте повтора одинаковых слов. Модифицируйте программу так, чтобы перед печатью слова предварительно сортировались.
12. (*2) Напишите функцию, которая подсчитывает количество повторов пар букв в строке типа *string*, а также в символьном массиве с терминальным нулем (строка в C-стиле). Например, пара букв "ab" входит в строку "habaasbaabb" дважды.
13. (*1.5) Определите структуру *Date* для хранения дат. Напишите функции для чтения дат из потока ввода, для вывода дат и для их инициализации.

Выражения и операторы

*Преждевременная оптимизация —
корень всех зол.
— Д. Кнут*

*С другой стороны, мы не можем игнорировать
эффективность.
— Джон Бентли*

Пример с калькулятором — ввод — параметры командной строки — обзор операций — побитовые логические операции — инкремент и декремент — свободная память — явное преобразование типов — свodka операторов — объявления — операторы выбора — объявления в условиях — операторы цикла — пресловутый *goto* — комментарии и отступы — советы — упражнения.

6.1. Калькулятор

Мы рассмотрим операторы и выражения языка C++ на примере программы калькулятора, реализующего четыре стандартных арифметических действия в форме инфиксных операций над числами с плавающей запятой. Пользователь может также определять переменные. Например, если на входе калькулятора имеется

```
r=2.5  
area=pi* r* r
```

(где *pi* — это предопределенная переменная) то в результате работы программа-калькулятор выдаст

```
2.5  
19.635
```

где *2.5* — это результат обработки первой строки, а *19.635* — второй.

Наш калькулятор состоит из четырех основных частей: синтаксического анализатора (или парсера — *parser*), функции ввода, таблицы символов и управляющей

программы (управляющая, ведущая программа — *driver*). По сути дела, эта программа является миниатюрным компилятором, где парсер выполняет синтаксический анализ, функция ввода реализует ввод исходных данных и выполняет лексический анализ, в таблице символов хранится постоянная информация, а управляющая программа осуществляет инициализацию, вывод результатов и обработку ошибок. Мы могли бы добавить к этому калькулятору массу иных возможностей, чтобы он стал более полезным (§6.6[20]), но код и без того уже достаточно велик, и кроме того, новые возможности калькулятора ничего бы нам не добавили в плане изучения языка C++.

6.1.1. Синтаксический анализатор

Вот *формальная грамматика* программы-калькулятора:

```

program :
    END                                // END это конец ввода
    expr_list END

expr_list :
    expression PRINT                  // PRINT это точка с запятой
    expression PRINT expr_list

expression :                          // выражение
    expression + term
    expression - term
    term

term :
    term / primary
    term * primary
    primary

primary :                             // первичное выражение
    NUMBER                             // число
    NAME                                // имя
    NAME = expression
    -primary
    (expression)

```

Иными словами, программа есть последовательность выражений, разделенных точками с запятой. Базовыми элементами выражений служат числа, имена и операции *****, **/**, **+**, **-** (унарная и бинарная), **=**. Объявлять имена до их использования необязательно.

Применяемый нами стиль синтаксического анализа называется *рекурсивным спуском* (*recursive descent*). В языках типа C++, где вызов функций обходится достаточно дешево, этот стиль еще и эффективен. Каждому порождающему правилу грамматики сопоставляется своя функция, вызывающая другие функции. Терминальные символы (например, **END**, **NUMBER**, **+** и **-**) распознаются лексическим анализатором **get_token()**; нетерминальные символы распознаются функциями синтаксического анализа, **expr()**, **term()** и **prim()**. Как только оба операнда (под)выражения распознаны, выражение тут же вычисляется; в настоящем же компиляторе в этот момент скорее генерируется машинный код.

Входные данные для парсера предоставляет функция *get_token()*. Самый последний возврат функции *get_token()* хранится в глобальной переменной *curr_tok*, имеющей тип перечисления *Token_value*:

```
enum Token_value
{
    NAME,          NUMBER,          END,
    PLUS='+',      MINUS='-',        MUL='*',          DIV='/',
    PRINT=';',     ASSIGN='=',      LP='(',           RP=')'
};

Token_value curr_tok=PRINT;
```

Представление лексемы (*tokens*) целочисленным кодом соответствующего символа удобно и эффективно, в том числе и при отладке программы. С этим не возникает никаких проблем, так как в известных мне системах кодировки символов нет печатных символов с однозначными кодами. Я выбрал *PRINT* в качестве начального значения для переменной *curr_tok*, поскольку именно это значение она получает после вычисления выражения и вывода результата. Таким образом, система стартует в нормальном состоянии, что минимизирует потенциальные ошибки и необходимость в специальной инициализации.

Каждая функция синтаксического анализа принимает аргумент типа *bool* (§4.2), указывающий, должна ли функция вызвать *get_token()* для получения очередной лексемы. Каждая такая функция вычисляет «свое выражение» и возвращает вычисленное значение. Функция *expr()* обрабатывает сложение и вычитание. Она состоит из единственного цикла, выполняющего сложение или вычитание *термов* (по-английски *terms* — это члены алгебраических выражений, над которыми выполняются операции сложения/вычитания; сами они представлены в виде произведения/частного выражений):

```
double expr (bool get)           // сложение и вычитание
{
    double left=term (get) ;

    for ( ; ; )                  // "вечно"
        switch (curr_tok)
        {
            case PLUS:
                left += term (true) ;
                break ;
            case MINUS:
                left -= term (true) ;
                break ;
            default:
                return left ;
        }
}
```

Эта функция мало что делает сама по себе. В манере высокоуровневых функций из крупных программ она вызывает другие функции, которые и делают большую часть работы.

Оператор **switch** сравнивает значение выражения в круглых скобках с набором констант. Оператор **break** используется для организации принудительного выхода из оператора **switch**. Константы, стоящие за **case**-метками, должны отличаться друг от друга. Если значение проверяемого выражения не совпадает ни с одной из констант, то осуществляется переход на **default**-ветвь. В общем случае, программист не обязан реализовывать **default**-ветвь в операторе **switch**.

Следует отметить, что выражение вроде $2 - 3 + 4$ вычисляется как $(2 - 3) + 4$, что диктуется определенной выше грамматикой.

Странное обозначение **for**(; ;) задает *бесконечный цикл*. Это вырожденный случай оператора цикла **for** (§4.2); **while**(**true**) — еще один пример на эту тему. В итоге оператор **switch** исполняется раз за разом, пока не встретится что-либо отличное от + или -, после чего срабатывает оператор **return** из **default**-ветви.

Операции присваивания += и -= использованы для выполнения сложения или вычитания. Можно было бы писать **left** = **left** + **term**(**true**) и **left** = **left** - **term**(**true**) с тем же самым конечным результатом. Но выражения **left** += **term**(**true**) и **left** -= **term**(**true**) не только короче, но и четче фиксируют предполагающиеся к выполнению действия. Каждая операция присваивания является отдельной самостоятельной лексемой, и поэтому выражение **a** += 1; ошибочно из-за лишних пробелов между + и =.

Сочетание простых операций присваивания возможно со следующими бинарными (двухоперандными) операциями

+ - * / % & | ^ << >>

так что имеются следующие *составные* (комбинированные) *операции присваивания*:

= += -= *= /= %= &= |= ^= <<= >>=

Здесь % означает операцию целочисленного деления по модулю (то есть нахождение остатка); &, | и ^ — побитовые логические операции «И», «ИЛИ» и «ИСКЛЮЧАЮЩЕЕ ИЛИ»; << и >> это операции битового сдвига влево и вправо; в §6.2 дана сводная информация по всем операциям и их смыслу. Для операндов встроенных типов и любой бинарной (двухоперандной) операции @ выражение **x** @ = **y** равносильно **x** = **x** @ **y** с той лишь разницей, что **x** вычисляется лишь один раз.

В главах 8 и 9 обсуждается модульная организация программ. Нашу же программу-калькулятор мы здесь упорядочиваем так, что объявления даются лишь один раз и до их использования. Единственное исключение касается функции **expr**() , которая вызывает **term**() , которая вызывает **prim**() , которая вызывает **expr**() . Этот замкнутый круг нужно как-то разорвать. Объявление

```
double expr (bool) ;
```

расположенное до определения функции **prim**() , решает проблему.

Функция **term**() обрабатывает умножение и деление аналогично тому, как **expr**() обрабатывает сложение и вычитание:

```
double term (bool get)           // умножение и деление
{
    double left = prim (get) ;

    for ( ; ; )
        switch (curr_tok)
```

```

{
    case MUL :
        left *= prim (true) ;
        break;
    case DIV:
        if (double d = prim (true) )
        {
            left /= d;
            break;
        }
        return error { "divide by 0" } ;
    default :
        return left;
}
}

```

Деление на нуль не определено и обычно приводит к неприятным последствиям. Поэтому мы проверяем делитель на нуль до выполнения операции деления, и вызываем функцию **error**() в случае нулевого значения делителя. Функция **error**() описывается в §6.1.4.

Переменная **d** определяется в программе ровно там, где она требуется, и тут же при этом инициализируется. Областью видимости переменной, определенной в условии, является тело условной конструкции, а значение этой переменной совпадает с величиной условного выражения (§6.3.2.1). Поэтому выражение **left** /= **d** вычисляется тогда и только тогда, когда **d** не равно нулю.

Функция **prim**(), обрабатывающая первичные элементы, похожа на **expr**() и **term**(), но поскольку тут мы забрались ниже по иерархии вызовов некоторая реальная работа уже выполняется (и цикл не нужен):

```

double number_value;
string string_value;

double prim (bool get)           // обработка первичных выражений
{
    if (get) get_token ();
    switch (curr_tok)
    {
        case NUMBER:
        {
            double v = number_value;
            get_token ();
            return v;
        }
        case NAME:
        {
            double& v = table [string_value] ;
            if (get_token () == ASSIGN) v = expr (true) ;
            return v;
        }
        case MINUS:              // унарный минус
            return -prim (true) ;
    }
}

```

```

case LP:
{
    double e = expr (true) ;
    if (curr_tok != RP) return error ("'" ' expected") ;
    get_token () ;           // пропустить ')'
    return e ;
}
default:
    return error ("primary expected") ;
}
}

```

Когда встречается **NUMBER** (то есть целый литерал или литерал с плавающей запятой), возвращается его значение. Вводимое функцией *get_token()* значение помещается в глобальную переменную *number_value*. Обычно глобальная переменная в программе сигнализирует о недостаточно четкой ее структуре — например, проведена некоторая оптимизация. Таков и наш случай. В идеале лексема состоит из двух частей: значения, определяющего тип лексемы (*Token_value* в данной программе), и величины самой лексемы (где это требуется). У нас же имеется единственная переменная *curr_tok*, из-за чего и потребовалась глобальная переменная *number_value* для хранения последнего прочитанного **NUMBER**. Задача исключить эту глобальную переменную предложена ниже в качестве самостоятельного упражнения (§6.6[21]). В действительности, нет жесткой необходимости сохранять значение *number_value* в локальной переменной *v* перед вызовом *get_token()*. Для каждого допустимого ввода наш калькулятор всегда выполняет вычисления с единственным числом, после чего читает новое. Но в случае ошибки сохраненное число и его вывод помогут пользователю.

Аналогично тому, как последнее значение **NUMBER** хранится в переменной *number_value*, строковое представление последнего **NAME** хранится в *string_value*. Прежде, чем обработать имя, калькулятору нужно заглянуть вперед и узнать, как это имя используется — читается или оно участвует в операции присваивания. В любом случае происходит обращение к *таблице символов*, имеющей тип *map* (§3.7.4, §17.4.1):

```
map<string, double> table;
```

Когда таблица символов *table* индексируется строкой, возвращается значение типа *double*, ассоциированное с указанной строкой. Например, если пользователь вводит

```
radius=6378.388;
```

то калькулятор выполняет следующее:

```
double& v = table ["radius"] ;
// ... expr () вычисляет значение, которое будет присвоено ...
v = 6378.388;
```

Ссылка *v* настраивается на числовое значение типа *double*, ассоциированное с именем *radius*, и сохраняет к нему доступ, пока функция *expr()* не выделит число *6378.388* из входного потока символов.

6.1.2. Функция ввода

Очень часто считывание ввода является самой запутанной частью программы. Это объясняется тем, что программе приходится взаимодействовать с человеком — учитывать его причуды, пристрастия и просто случайные ошибки. Попытка заставить человека действовать как машина часто воспринимается им (вполне справедливо) как оскорбление. Задача процедуры низкоуровневого ввода состоит в чтении потока символов и выделения из него более высокоуровневых лексем. Эти лексемы затем подаются на вход еще более высокоуровневым процедурам. В нашей программе низкоуровневый ввод обрабатывается функцией *get_token()*. По счастью, написание процедур низкоуровневого ввода не является каждодневной задачей, ибо во многих системах для этого имеются готовые стандартные процедуры.

Я выбрал двухэтапную тактику построения функции *get_token()*. Сначала я напишу обманчиво простую версию, которая все сложности перекладывает на пользователя программы. Затем я модифицирую ее в окончательный вариант, который выглядит менее элегантно, зато прост в использовании.

Общий план состоит в считывании символа, определении по этому символу типа лексемы и возврате соответствующего *Token_value*.

Начальные операторы функции считывают первый непробельный символ в переменную *ch* и проверяют успешность ввода:

```
Token_value get_token ()
{
    char ch = 0;
    cin >> ch;

    switch (ch)
    {
        case 0:
            return curr_tok = END;    // присваивание и возврат
```

По умолчанию, операция *>>* пропускает пробельные символы (то есть собственно пробел, табуляцию, переход на новую строку и т.д.) и не изменяет значение переменной *ch* в случае неудачи ввода. Отсюда следует, что *ch == 0* означает конец ввода.

Операция присваивания вырабатывает значение, совпадающее с новой величиной переменной, стоящей в левой части операции присваивания. Поэтому я присваиваю *END* переменной *curr_tok* и возвращаю это же значение в рамках единственного оператора. Это улучшает надежность сопровождения программы, так как в случае двух операторов программист может изменить лишь один из них, забыв про другой.

Теперь взглянем на отдельные ветви оператора *switch* до того, как будет представлен весь текст функции. Для символа окончания выражений, то есть символа *' ; '*, круглых скобок и символов операций, выполняемых калькулятором, возвращаются их числовые коды:

```
case ' ; ':
case ' * ':
case ' / ':
case ' + ':
case ' - ':
```

```

case ' ( ' :
case ' ) ' :
case ' = ' :
    return curr_tok=Token_value (ch) ;

```

Для чисел поступаем следующим образом:

```

case ' 0 ' : case ' 1 ' : case ' 2 ' : case ' 3 ' : case ' 4 ' :
case ' 5 ' : case ' 6 ' : case ' 7 ' : case ' 8 ' : case ' 9 ' :
case ' . ' :
    cin.putback (ch) ;
    cin>>number_value;
    return curr_tok=NUMBER;

```

Может быть и не слишком хорошо с точки зрения восприятия располагать *case*-ветви оператора **switch** горизонтально, но уж больно утомительно писать однообразные строки одну под другой. Суть же кода очень проста, так как операция **>>** читать константы с плавающей запятой в переменные типа **double** умеет изначально. Выявив характерный для чисел начальный символ (цифра или точка), возвращают его в поток **cin**, после чего вся константа читается в переменную **number_value**.

Аналогично работаем с именами:

```

default:                // NAME, NAME =, или ошибка
if (isalpha (ch) )
{
    cin.putback (ch) ;
    cin>>string_value;
    return curr_tok=NAME;
}
error ("bad token") ;
return curr_tok=PRINT;

```

Чтобы избежать перечисления всех алфавитных символов в *case*-ветвях, используем стандартную библиотечную функцию **isalpha()** (§20.4.2). Операция **>>** читает символы в строку (в нашем случае, в переменную **string_value**) до тех пор, пока не встретится пробельный символ. Это значит, что во входном тексте пользователь программы должен после имени (и до знака операции) располагать пробельные символы. Это крайне неудобно и мы еще вернемся к этому вопросу в §6.1.3.

И вот, наконец, полный текст функции ввода:

```

Token_value get_token ()
{
    char ch=0;
    cin>>ch;

    switch (ch)
    {
        case 0:
            return curr_tok=END;
        case ' ; ' :
        case ' * ' :
        case ' / ' :

```



```

case '+' :
case '-' :
case '(' :
case ')' :
case '=' :
    return curr_tok=Token_value(ch) ;
case '0' : case '1' : case '2' : case '3' : case '4' :
case '5' : case '6' : case '7' : case '8' : case '9' :
case '.' :
    cin.putback(ch) ;
    cin>>number_value;
    return curr_tok=NUMBER;
default:                                     // NAME, NAME =, или ошибка
    if(isalpha(ch))
    {
        cin.putback(ch) ;
        cin>>string_value;
        return curr_tok=NAME;
    }
    error("bad token") ;
    return curr_tok=PRINT;
}
}

```

Преобразование операций к типу *Token_value* имеет тривиальный смысл, так как элементы этого перечисления, соответствующие операциям, имеют значения, совпадающие с кодировками символов операций (см. также §4.8).

6.1.3. Низкоуровневый ввод

Работа с текущей версией калькулятора имеет ряд неудобств. Например, нужно не забывать про точку с запятой, иначе калькулятор не выведет вычисленного значения. Необходимость же в обязательном порядке проставлять пробелы после имен, вообще просто нонсенс. Действительно, в данной версии калькулятора *x=7* трактуется как идентификатор, а вовсе не набор из идентификатора *x*, операции *=* и числа *7*. Все перечисленные неудобства устраняются путем замены в теле функции *get_token()* простых стандартных операций типориентированного ввода на операции посимвольного ввода.

Начнем с того, что предложим наряду с точкой с запятой использовать и перевод строки для индикации конца выражений:

```

Token_value get_token()
{
    char ch;

    do                                     // пропустить все пробельные символы кроме '\n'
    {
        if(!cin.get(ch)) return curr_tok = END;
    } while (ch != '\n' && isspace(ch)) ;

    switch (ch)

```

```

case ' ' :
case '\n' :
    return curr_tok=PRINT;

```

Здесь использован *оператор цикла do*, который эквивалентен циклическому оператору *while*, за исключением того, что *его тело всегда исполняется хотя бы один раз*. Вызов *cin.get(ch)* читает один символ из стандартного потока ввода в переменную *ch*. По умолчанию, *get()* не выполняет автоматически пропуск пробельных символов так, как это делает операция *>>*. Проверка *if(!cin.get(ch))* выявляет невозможность чтения символа из *cin*; в этом случае возвращается *END* и работа с калькулятором завершается. Операцию *!* (логическое отрицание) приходится использовать из-за того, что *get()* возвращает *true* в случае успешного чтения.

Стандартная библиотечная функция *isspace()* проверяет, является ли символ пробельным (§20.4.2); *isspace(c)* возвращает ненулевое значение для пробельных символов и нуль в противном случае. Функция *isspace()* выполняется быстрее, чем наш альтернативный перебор всех пробельных символов. Имеются также стандартные функции для выявления цифр — *isdigit()* — букв — *isalpha()*, а также цифр или букв — *isalnum()*.

После пропуска пробельных символов следующий за ними символ используется для выявления типа лексемы.

Проблема, вызванная чтением строки операцией *>>* вплоть до пробельных символов, преодолевается последовательным посимвольным чтением до тех пор, пока не встретится символ, отличный от буквы или цифры:

```

default:                // NAME, NAME=, или ошибка
    if (isalpha (ch) )
    {
        string_value=ch;
        while (cin.get(ch) && isalnum (ch) ) string_value.push_back (ch) ;
        cin.putback (ch) ;
        return curr_tok=NAME;
    }
    error ("bad token") ;
    return curr_tok=PRINT;

```

Оба рассмотренных улучшения достигаются модификацией четко ограниченных небольших участков кода. Конструирование программ таким образом, что их последующие модификации могут ограничиваться локальными изменениями кода, является важной целью проектирования.

6.1.4. Обработка ошибок

Из-за того, что наша программа чрезвычайно проста, обработка ошибок для нее не является предметом первостепенного интереса. Наша функция обработки ошибок просто подсчитывает их количество, выводит сообщение и завершает работу:

```

int no_of_errors;

double error (const string& s)
{
    no_of_errors++;

```

```

cerr<< "error: " << s << '\n';
return 1;
}

```

Поток **cerr** — это *небуферизованный выходной поток*, который обычно используется для *вывода сообщений об ошибках* (§21.2.1).

Наша функция возвращает значение по той причине, что обычно ошибки возникают в процессе вычисления выражений, так что следует либо полностью прервать этот процесс, либо вернуть некоторое значение, которое не вызовет осложнений в дальнейшей работе. Для нашего простого калькулятора годится второй подход. Если бы *get_token()* отслеживала номер строки исходного текста, функции *error()* имело бы смысл информировать пользователя о том, где примерно произошла ошибка. Это было бы особенно полезно в случае неинтерактивного использования программы-калькулятора (§6.6[19]).

Часто нужно завершать работу программы при возникновении ошибок, ибо нет разумного продолжения их работы. Это можно сделать вызовом *библиотечной функции exit()*, которая *сначала освобождает ресурсы* (вроде потоков вывода), и *только после этого завершает программу*, возвращая значение, совпадающее с аргументом ее вызова (§9.4.1.1).

Более регулярный метод обработки ошибок основан на работе с исключениями (§8.3, глава 14), но для программы из 150 строк вполне достаточно уже сделанного нами.

6.1.5. Управляющая программа

Теперь нам недостает лишь управляющей программы, которая запускает и оркеструет всю работу. В нашем случае для этого достаточно одной функции *main()*:

```

int main ()
{
    table [ "pi" ] = 3.1415926535897932385;    // вводим predefined имена
    table [ "e" ] = 2.7182818284590452354;

    while (cin)
    {
        get_token ();
        if (curr_tok == END) break;
        if (curr_tok == PRINT) continue;
        cout << expr (false) << '\n';
    }

    return no_of_errors;
}

```

По устоявшемуся соглашению функция *main()* возвращает нуль в случае нормального завершения и ненулевое значение в противном случае (§3.2). Возврат количества ошибок отлично развивает эту идею. Вся инициализация свелась у нас к формированию таблицы predefined имен.

В теле функции *main()* основная работа сосредоточена в цикле, где считывается выражение и выводится результат его вычисления. Последнее выполняется следующей строкой кода:

```
cout<<expr (false) << '\n' ;
```

Аргумент **false** сообщает функции **expr()**, что ей не надо вызывать **get_token()** для выделения лексемы.

Проверка **cin** для каждой итерации цикла гарантирует корректное завершение программы, если с входным потоком возникнут какие-нибудь проблемы, а сравнение с **END** гарантирует корректное завершение цикла, если **get_token()** обнаружит *конец ввода*¹. Оператор **break** осуществляет принудительный выход из ближайшего оператора **switch** или цикла. Сравнение с **PRINT** (то есть с '\n' и ';') освобождает функцию **expr()** от ответственности за обработку пустых выражений. Оператор **continue** означает переход к следующей итерации цикла (по-другому можно сказать, что в самый конец тела цикла), так что код

```
while (cin)
{
    // . .
    if (curr_token==PRINT) continue ;
    cout<< expr (false) << '\n' ;
}
```

эквивалентен

```
while (cin)
{
    // ...
    if (curr_token!=PRINT)
        cout<< expr (false) << '\n' ;
}
```

6.1.6. Заголовочные файлы

Наша программа-калькулятор использует средства стандартной библиотеки, из-за чего в окончательный код нужно включить следующие директивы **#include** препроцессора:

#include <iostream>	// I/O (ввод/вывод)
#include <string>	// string (строки)
#include <map>	// map (ассоциативные массивы)
#include <cctype>	// isalpha(), и т.д.

Все средства из перечисленных здесь заголовочных файлов определяются в пространстве имен **std**, так что имена из этих файлов нужно либо полностью квалифицировать префиксом **std:**, либо заранее внести их в глобальное пространство имен следующим образом:

```
using namespace std;
```

Я выбрал последнее, чтобы не смешивать в одну кучу обсуждение вопросов обработки выражений и концепций модульного построения программ. В главах 8 и 9 обсуждаются способы модульной организации программы-калькулятора с приме-

¹ Это служебный символ end-of-file — он генерируется клавиатурой в ответ на нажатие специальной комбинации клавиш, чаще всего — Ctrl+Z. — *Прим. ред.*

нением сегментирующих средств в виде пространств имен и набора исходных файлов. На многих системах помимо указанных выше заголовочных файлов имеются еще и их эквиваленты с расширением `.h`, и в которых классы, функции и т.д. определяются в глобальном пространстве имен (§9.2.1, §9.2.4, §B.3.1).

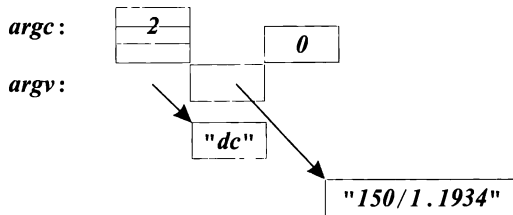
6.1.7. Аргументы командной строки

После того как я написал и оттестировал программу-калькулятор, я обнаружил, что пользоваться ею не всегда удобно: нужно ее запустить, затем ввести выражение, и в конце концов, завершить работу программы. Чаще всего я использовал калькулятор для вычисления единственного выражения. Если бы это выражение можно было представить параметром командной строки, то удалось бы сэкономить несколько нажатий клавиш.

Программа начинает свою работу с функции `main()` (§3.2, §9.4). Ей передаются два аргумента, первый из которых задает общее количество аргументов командной строки (обычно обозначается `argc`), а второй — это массив аргументов (обычно обозначаемый `argv`). Эти аргументы передаются функции `main()` оболочкой, ответственной за запуск программы; если оболочка не может передать аргументы, то `argc` устанавливается в нуль. Так как соглашения о вызове функции `main()` взяты из языка C, то используется массив строк в стиле C (`char* argv[argc+1]`). Имя программы (как оно записано в командной строке) передается с помощью `argv[0]`. Список аргументов имеет завершающий нуль, то есть `argv[argc] == 0`. Например, для командной строки

```
dc 150/1.1934
```

ее аргументы имеют следующие значения:



Получить доступ к аргументам командной строки несложно. Вопрос в том, как их использовать с минимальными изменениями в готовой программе. Для этого предлагается читать из командной строки так же, как мы читали входные данные из входного потока. Неудивительно, что *поток для чтения из строки называется `istringstream`*. К сожалению, отсутствует способ заставить `cin` ссылаться на поток `istringstream`. Поэтому мы должны заставить функции ввода нашей программы-калькулятора работать с `istringstream`. Более того, мы должны их заставить работать либо с `istringstream`, либо с `cin`, в зависимости от конкретного вида командной строки.

Простейшим решением будет определить глобальный указатель `input` и настроить его на нужный поток, а также заставить все процедуры ввода использовать его:

```

istream* input ;                // указатель на поток ввода
int main (int argc, char* argv[])
{
    switch (argc)
    {
        case 1:
            input= &cin;
            break;
        case 2:
            input=new istringstream (argv[1]) ;
            break;
        default:
            error ("too many arguments") ;
            return 1;
    }

    table ["pi"] =3.1415926535897932385;
    table ["e"] =2.7182818284590452354;

    while (*input)
    {
        get_token () ;
        if (curr_tok==END) break;
        if (curr_tok==PRINT) continue;
        cout<<expr (false) <<' \n' ;
    }

    if (input!=&cin) delete input;

    return no_of_errors;
}

```

Поток *istringstream* является потоком ввода, который читает из строки, переданной в качестве аргумента (§21.5.3). По достижению конца читаемой строки поведение потока *istringstream* точно такое же, как у остальных потоков ввода (§3.6, §21.3.3). Чтобы использовать в программе *istringstream*, нужно директивой *#include* препроцессора включить заголовочный файл *<sstream>*.

Было бы совсем нетрудно заставить функцию *main()* читать большее число аргументов командной строки, но это представляется ненужным, так как несколько выражений можно передать одним аргументом:

```
dc "rate=1.1934;150/rate;19.75/rate;217/rate"
```

Я применил двойные кавычки, поскольку ; (точка с запятой) на моей UNIX системе служит разделителем команд. Другие операционные системы имеют свои соглашения по передаче аргументов программам при их запуске.

Нельзя назвать наше решение элегантным, поскольку все процедуры ввода приходится переделывать под использование **input* вместо *cin*, с тем чтобы они могли читать из разных источников. Этих изменений можно было избежать, если бы я заранее предвидел эту ситуацию и применил что-нибудь вроде *input* с самого начала. Более общий и полезный подход состоит в том, что источник ввода должен быть параметром для модуля калькулятора. Фундаментальная проблема данной программы-калькулятора состоит в том, что так называемый «калькулятор» реализуется

в ней просто набором функций и данных. Нет ни модуля (§2.4), ни объекта (§2.5.2), представляющих калькулятор в явном виде. Если бы я разрабатывал модуль калькулятора или тип калькулятора, то, естественно, задумался бы над его параметрами (§8.5[3], §10.6[16]).

6.1.8. Замечания о стиле

Программистам, не знакомым с ассоциативными массивами, применение библиотечного типа *map* для представления таблицы символов может показаться нарочитым высокоуровневым трюкачеством. Это не так. И стандартная, и другие библиотеки для того и создаются, чтобы ими пользовались. Часто библиотеки разрабатываются и реализуются с такой тщательностью, какую отдельный программист не может себе позволить в процессе написания участка кода, применимого лишь в одной программе.

Взглянув на нашу программу-калькулятор, особенно на ее первую версию, можно заметить, что в ней мало традиционно низкоуровневого кода в C-стиле. Необходимость низкоуровневых трюков была устранена за счет применения библиотечных классов *ostream*, *string* и *map* (§3.4, §3.5, §3.7.4, глава 17).

Обратите внимание, как редко встречаются арифметические операции, циклы и даже присваивания. Так и должно обстоять дело в программах, не управляющих непосредственно аппаратурой и не реализующих низкоуровневых абстракций.

6.2. Обзор операций языка C++

В данном разделе дается обзор выражений и операций языка C++, а также примеры на эту тему. Каждая операция сопровождается названием и примером выражения, использующего эту операцию. Ниже в таблице операций *class_name* означает имя класса; *member* — имя члена класса; *object* — выражение, дающее объект класса; *pointer* — выражение, дающее указатель; *expr* — выражение; *lvalue* — выражение, обозначающее неконстантный объект; *type* — имя типа в самом общем смысле (вместе с *, () и т.д.), если помещено в круглые скобки, а иначе есть ограничения (§A.5).

Синтаксис выражений не зависит от типа операндов. Однако смысл операций, указанный ниже, имеет место лишь для встроенных типов (§4.1.1). Смысл операций над пользовательскими типами данных можно задавать самостоятельно (§2.5.2, глава 11).

Операции	
разрешение области видимости разрешение области видимости глобально глобально	<i>class-name</i> :: <i>member</i> <i>namespace-name</i> :: <i>member</i> :: <i>name</i> :: <i>qualified-name</i>
выбор члена выбор члена доступ по индексу вызов функции	<i>object</i> . <i>member</i> <i>pointer</i> -> <i>member</i> <i>pointer</i> [<i>expr</i>] <i>expr</i> (<i>expr-list</i>)

Операции	
конструирование значения постфиксный инкремент постфиксный декремент идентификация типа идентификация типа во время выполнения преобразование с проверкой во время выполнения преобразование с проверкой во время компиляции преобразование без проверки константное преобразование	<i>type</i> (<i>expr-list</i>) <i>lvalue</i> ++ <i>lvalue</i> -- <i>typeid</i> (<i>type</i>) <i>typeid</i> (<i>expr</i>) <i>dynamic_cast</i> < <i>type</i> > (<i>expr</i>) <i>static_cast</i> < <i>type</i> > (<i>expr</i>) <i>reinterpret_cast</i> < <i>type</i> > (<i>expr</i>) <i>const_cast</i> < <i>type</i> > (<i>expr</i>)
размер объекта размер типа префиксный инкремент префиксный декремент дополнение отрицание унарный минус унарный плюс адрес разыменование создать (выделить память) создать (выделить память и инициализировать) создать (разместить) создать (разместить и инициализировать) уничтожить (освободить память) уничтожить массив приведение (преобразование типа)	<i>sizeof</i> <i>expr</i> <i>sizeof</i> (<i>type</i>) ++ <i>lvalue</i> -- <i>lvalue</i> ~ <i>lvalue</i> ! <i>expr</i> - <i>expr</i> + <i>expr</i> & <i>lvalue</i> * <i>expr</i> <i>new</i> <i>type</i> <i>new</i> <i>type</i> (<i>expr-list</i>) <i>new</i> (<i>expr-list</i>) <i>type</i> <i>new</i> (<i>expr-list</i>) <i>type</i> (<i>expr-list</i>) <i>delete</i> <i>pointer</i> <i>delete</i> [] <i>pointer</i> (<i>type</i>) <i>expr</i>
выбор члена выбор члена	<i>object</i> . * <i>pointer-to-member</i> <i>pointer</i> -> * <i>pointer-to-member</i>
умножение деление остаток от деления (деление по модулю)	<i>expr</i> * <i>expr</i> <i>expr</i> / <i>expr</i> <i>expr</i> % <i>expr</i>
сложение (плюс) вычитание (минус)	<i>expr</i> + <i>expr</i> <i>expr</i> - <i>expr</i>
сдвиг влево сдвиг вправо	<i>expr</i> << <i>expr</i> <i>expr</i> >> <i>expr</i>
меньше меньше или равно больше больше или равно	<i>expr</i> < <i>expr</i> <i>expr</i> <= <i>expr</i> <i>expr</i> > <i>expr</i> <i>expr</i> >= <i>expr</i>
равно не равно	<i>expr</i> == <i>expr</i> <i>expr</i> != <i>expr</i>
побитовое И (AND)	<i>expr</i> & <i>expr</i>
побитовое исключающее ИЛИ (OR)	<i>expr</i> ^ <i>expr</i>
побитовое ИЛИ (OR)	<i>expr</i> <i>expr</i>
логическое И (AND)	<i>expr</i> && <i>expr</i>

Операции	
логическое ИЛИ (OR)	<i>expr</i> <i>expr</i>
условное выражение	<i>expr</i> ? <i>expr</i> : <i>expr</i>
простое присваивание	<i>lvalue</i> = <i>expr</i>
умножение и присваивание	<i>lvalue</i> *= <i>expr</i>
деление и присваивание	<i>lvalue</i> /= <i>expr</i>
остаток и присваивание	<i>lvalue</i> %= <i>expr</i>
сложение и присваивание	<i>lvalue</i> += <i>expr</i>
вычитание и присваивание	<i>lvalue</i> -= <i>expr</i>
сдвиг влево и присваивание	<i>lvalue</i> <<= <i>expr</i>
сдвиг вправо и присваивание	<i>lvalue</i> >>= <i>expr</i>
И и присваивание	<i>lvalue</i> &= <i>expr</i>
ИЛИ и присваивание	<i>lvalue</i> = <i>expr</i>
исключающее ИЛИ и присваивание	<i>lvalue</i> ^= <i>expr</i>
генерация исключения	throw <i>expr</i>
запятая (следование)	<i>expr</i> , <i>expr</i>

В каждой секции данной таблицы сосредоточены операции с одинаковым *приоритетом* (*precedence*). Операции из вышестоящих секций имеют приоритет, более высокий по отношению к операциям, расположенным в нижележащих секциях. Например, *a+b*c* означает именно *a+(b*c)*, а не *(a+b)*c*, так как операция *** имеет более высокий приоритет, чем операция *+*. Аналогично, выражение **p++* означает **(p++)*, а не *(*p)++*.

Префиксные операции и операции присваивания *правоассоциативны* (*right-associative*); все остальные операции — *левоассоциативны* (*left-associative*). Например, *a=b=c* означает *a=(b=c)*, в то время как *a+b+c* есть *(a+b)+c*.

Несколько грамматических правил невозможно выразить в терминах приоритета (называемого, также, силой связывания) и ассоциативности. Например, *a=b<c?d:e:f=g* означает *a=((b<c)?(d=e):(f=g))*, но для подтверждения этого факта нужно обратиться к сводке грамматических правил (§A.5).

Грамматические правила применяются после того, как из отдельных символов построены лексемы (§A.3), причем лексемы строятся из потенциально наиболее длинных последовательностей. Например, *&&* означает единственную операцию, а не две операции *&*; выражение *a+++1* означает *(a++)+1*.

6.2.1. Результаты операций

Результирующий тип арифметических операций определяется набором правил, известных как «*стандартные арифметические преобразования*» (§C.6.3). Их общая цель сводится к тому, чтобы получить тип «наибольшего» операнда. Например, если у бинарной операции один из операндов имеет тип с плавающей запятой, то вычисления выполняются в арифметике чисел с плавающей запятой и, соответственно, таков и тип результата этой операции. Если у бинарной операции один из операндов имеет тип *long*, то вычисления выполняются в арифметике длинных целых, и возвращается результат типа *long*. Операнды «меньших» чем *int* типов (такие как *bool* или *char*) преобразуются к *int* перед выполнением операции.

Операции сравнения `==`, `<=` и т.д. *вырабатывают результат логического типа*. Смысл и тип результатов операций, определяемых пользователем, вытекают из их объявления (§11.2).

Если нет логических препятствий, то результат операции с операндом *lvalue* совпадает с *lvalue*, которое операция вырабатывает для этого операнда. Например:

```
void f(int x, int y)
{
    int j=x=y;           // значением x=y является значение x после присваивания
    int* p=&++x;          // p указывает на x
    int* q=&(x++) ;       // error: x++ не есть lvalue (это не значение, хранимое в x)
    int* pp=&(x>y?x:y) ;  // адрес целого с большим значением
}
```

Если второй и третий операнды операции `?:` являются *lvalue* одного и того же типа, то результат операции есть *lvalue* того же самого типа. В целом, такой «сохраняющий» стиль обращения с *lvalue* обеспечивает чрезвычайную гибкость операций. Особую важность это приобретает в случаях, когда требуется писать код, одинаково подходящий и эффективный как для встроенных, так и для пользовательских типов данных (например, при написании шаблонов или программ, генерирующих код C++).

Результат операции *sizeof* есть *беззнаковый интегральный тип size_t*, определенный в заголовочном файле `<stddef>`. Типом *разности указателей* служит *знаковый интегральный тип ptrdiff_t*, также определенный в файле `<stddef>`.

Конкретные реализации C++ не обязаны контролировать арифметическое переполнение и вряд ли какая из них делает это. Например:

```
void f()
{
    int i=1;
    while (0<i) i++;
    cout<< "i has become negative!" << i << '\n' ;
}
```

Эта функция попытается (в итоге) увеличить на единицу максимально возможное целое значение. Результат не определен, но в типичных случаях произойдет перескок к отрицательным значениям (на моей машине `-2147483648`). Также не определен результат деления на нуль, но обычно это заканчивается принудительным остановом программы. Стандартные исключения (§14.10) для деления на нуль и переполнений не генерируются.

6.2.2. Последовательность вычислений

Порядок вычисления подвыражений в рамках объемлющих выражений не определен. В частности, нельзя рассчитывать на то, что выражения вычисляются слева направо. Например:

```
int x=f(2)+g(3) ;      // неизвестно, что будет вызвано первым - f() или g()
```

При отсутствии ограничений на порядок вычисления выражений можно генерировать более эффективный машинный код. Это же, однако, может приводить к неопределенным результатам. Например,

```
int i=1;
v[i]=i++;           // результат не определен
```

может вычисляться как $v[1]=1$, или как $v[2]=1$, или даже еще более странным образом. Компиляторы могут предупреждать о подобных *неоднозначностях* (*ambiguities*), а могут, к сожалению, и не предупреждать.

Для операции `,` (операция запятая — *comma operator*), а также для операций `&&` (логическое «И») и `||` (логическое «ИЛИ») гарантируется, что их *левый операнд будет вычислен раньше, чем правый операнд*. Например, с помощью выражения $b=(a=2, a+1)$ переменной *b* будет присвоено значение 3. Примеры использования операций `||` и `&&` даны в §6.2.3. Для встроенных типов правый операнд операции `&&` вычисляется лишь в случае, когда левый операнд равен *true*. Для операции `||` правый операнд вычисляется только в случае, когда левый операнд равен *false*. Эти правила часто называют *редуцированной схемой вычислений* (*short-circuit evaluation*). Обратите внимание на то, что запятая как операция следования (*sequencing operator*) логически отличается от запятой, используемой в качестве разделителя аргументов функций. К примеру, в следующем коде

```
f1(v[i], i++);      // два аргумента
f2((v[i], i++));    // один аргумент
```

вызов функции *f1()* оперирует двумя аргументами, *v[i]* и *i++*, и порядок вычисления аргументов не определен. *Нельзя полагаться на определенный порядок вычисления аргументов вызова функций* — это плохой стиль программирования, приводящий к непредсказуемому поведению программы. Вызов функции *f2()* оперирует единственным аргументом — выражением с операцией следования (операцией запятая), чье значение равно *i++*.

Принудительную группировку выражений можно осуществить с помощью круглых скобок. Например, $a*b/c$ означает $(a*b)/c$, но применение круглых скобок позволяет навязать иной порядок вычислений исходного выражения — $a*(b/c)$. Для вычислений с плавающей запятой выражения $(a*b)/c$ и $a*(b/c)$ могут давать существенно разные результаты, так что компилятор обязан соблюдать указанный порядок вычисления выражений.

6.2.3. Приоритет операций

Уровни приоритетов операций и правила ассоциативности отражают естественные способы их использования. Например:

```
if(i<=0 || max<i)    // ...
```

означает «если *i* меньше или равно 0 или если *max* меньше *i*». То есть это эквивалентно следующему выражению

```
if((i<=0) || (max<i)) // ...
```

а не бессмысленному (хотя и формально корректному) выражению

```
if(i <= (0 || max) < i) // ...
```

Круглые скобки можно использовать всегда, когда у программиста есть сомнения. Чем сложнее выражения, тем чаще применяют круглые скобки. В то же время, слишком сложные выражения могут приводить к ошибкам. Поэтому, если вы чув-

стуете настоящую необходимость в круглых скобках, то возможно стоит применить вспомогательные (промежуточные) переменные и избавиться от сложных выражений.

Бывают случаи, когда приоритет операций не обеспечивает очевидности порядка вычислений. Например:

```
if(i&mask==0)    //...
```

Наложение маски (переменная *mask*) на *i* и сравнение результата с нулем является хотя и очевидной, но неправильной интерпретацией. Из-за того, что у операции `==` приоритет выше, чем у операции `&`, выражение на самом деле эквивалентно `i&(mask==0)`. К счастью, в большинстве подобных случаев компилятор выдает предупреждение о возможной ошибке. В общем, нужно применить группирующие круглые скобки:

```
if( (i&mask) ==0)    // ...
```

Следующее выражение работает не так, как ожидал бы математик:

```
if(0<=x<=99)    // ...
```

Это формально корректное выражение интерпретируется как `(0<=x) <=99`, где первое сравнение вырабатывает *true* или *false*. Эти логические значения неявно преобразуются в *1* или *0*, а затем сравниваются с *99*, что всегда дает *true*. Для проверки попадания значения *x* в диапазон *0*..*99* можно написать

```
if(0<=x && x<=99)    // ...
```

Для новичков типично ошибочное применение операции `=` (операция присваивания) вместо операции `==` (операция сравнения на равенство):

```
if(a=7)    //...
```

Это довольно естественно, поскольку во многих языках программирования знак `=` означает «равно». Но к счастью, большинство компиляторов распознают подобного рода проблемы и предупреждают программиста о возможных ошибках.

6.2.4. Побитовые логические операции

Побитовые логические операции (bitwise logical operators) `&`, `|`, `^`, `~`, `>>` и `<<` применяются к интегральным типам и перечислениям, то есть к типам *bool*, *char*, *short*, *int*, *long* (возможно, с модификатором *unsigned*). Обычные арифметические преобразования определяют тип результата (§C.6.3).

Типичным примером использования побитовых логических операций служит задача построения небольшого множества (битового вектора). Каждому элементу множества соответствует один бит беззнакового целого, так что количество элементов множества ограничивается количеством бит. Бинарная операция `&` трактуется как операция пересечения множеств, операция `|` трактуется как объединение множеств, операция `^` — как симметричная разность, и операция `~` — как дополнение. Для именования элементов такого множества можно использовать перечисление. Вот маленький пример, взятый из реализации потоков типа *ostream*:

```
enum ios_base : iostate
{
    goodbit=0, eofbit=1, failbit=2, badbit=4
};
```

Можно устанавливать и проверять состояние потока следующим образом:

```
state=goodbit;
// ...
if (state & (badbit | failbit)) // с потоком проблемы
```

Здесь дополнительные круглые скобки необходимы, так как приоритет операции `&` выше, чем приоритет операции `|`.

Достигнув конца ввода, функция может просигнализировать об этом следующим образом:

```
state |= eofbit;
```

где с помощью операции `|=` к текущему состоянию потока добавляется ***eofbit***, в то время как простое присваивание `state=eofbit` обнулило бы все остальные биты.

Флаги состояния потока доступны и клиентам (то есть вне реализации потоков). Например, клиент может посмотреть, как различаются состояния двух потоков:

```
int diff = cin.rdstate() ^ cout.rdstate(); // rdstate() возвращает состояние
```

Вычисление различий в состояниях потоков встречается не столь уж и часто. Но для других типов, базирующихся на битовых векторах, эта задача весьма актуальна. Например, важно сравнивать битовые вектора, один из которых представляет собой набор обрабатываемых прерываний, а другой — набор прерываний, ожидающих обработки.

Обратите внимание на то, что вся эта возня с битами выполняется внутри реализации потоков, а не в клиентском коде. Удобная манипуляция битами может быть и важна, но из соображений надежности, сопровождаемости, переносимости и т.д. ее лучше запрятать поглубже, на нижние уровни реализации системы. За более подробными сведениями о множествах обратитесь к описаниям таких типов стандартной библиотеки, как ***set*** (§17.4.3), ***bitset*** (§17.5.3) и ***vector<bool>*** (§16.3.11).

Удобно использовать битовые поля (§С.8.1), чтобы выполнять битовые сдвиги и маскирование с целью извлечения отдельных бит из целочисленного слова. Это, конечно, можно сделать и с помощью побитовых логических операций. Например, можно следующим образом извлечь средние 16 бит из 32-разрядного ***long***:

```
unsigned short middle(long a) {return (a>>8) & 0xffff;}
```

Не путайте побитовые логические операции с логическими операциями `&&`, `||` и `!`. Последние возвращают ***true*** или ***false*** и в основном используются в условных выражениях операторов ***if***, ***while*** или ***for*** (§6.3.2, §6.3.3). Например, `!0` (не ноль) есть ***true***, в то время как `~0` (битовое «НЕ») дает набор всех битов, равных единице, что в двоичном дополнительном коде означает `-1`.

6.2.5. Инкремент и декремент

Операция `++` выражает *инкремент* (увеличение на единицу) самым непосредственным образом, в то время как применение комбинации из сложения и присваи-

вания делает то же самое лишь косвенно. По определению, $++lvalue$ означает $lvalue+=1$, что в свою очередь означает $lvalue=lvalue+1$, если у $lvalue$ нет побочных эффектов. Выражение, возвращающее подлежащий инкременту объект, вычисляется один (и только один) раз. Операция декремента записывается как $--$. Операции $++$ и $--$ *используются как в префиксной, так и в постфиксной формах*. Значением выражения $++x$ является новое (инкрементированное) значение x . Например, $y=++x$ эквивалентно $y=(x+=1)$. Значением же выражения $x++$ является старое значение x . Например, $y=x++$ эквивалентно $y=(t=x, x+=1, t)$, где переменная t имеет тот же тип, что и x .

Как и в случае сложения и вычитания указателей, *операции $++$ и $--$ работают над указателями в единицах размера указуемых объектов*, что удобно для массивов; $p++$ заставляет указатель p настроиться на следующий элемент массива (§5.3.1).

Операции $++$ и $--$ особенно полезны при инкрементировании и декрементировании переменных в циклах. Например, копирование строк с терминальным нулем можно выполнить следующим образом:

```
void cpy (char* p, const char* q)
{
    while (*p++=*q++) ;
}
```

Язык C++ (как и язык C) либо сильно любят, либо сильно ненавидят за возможность написания *кода, ориентированного на выражения (expression-oriented coding)*. Так как

```
while (*p++=*q++) ;
```

выглядит более чем странно для программистов на большинстве языков (кроме C/C++), а на C и C++ подобного рода фрагменты встречаются отнюдь не редко, то стоит присмотреться к ним подробнее.

Сначала рассмотрим более традиционный способ копирования массива символов:

```
int length=strlen (q) ;
for (int i=0; i<=length; i++) p[i] = q[i] ;
```

Это расточительный способ работы. Длину строки с терминальным нулем легко можно определить непосредственно в процессе просмотра символов этой строки. А так получается, что мы читаем строку дважды: первый раз — чтобы определить длину строки, а второй раз для копирования ее символов. Исправляем этот недочет:

```
int i;
for (i=0; q[i] !=0; i++) p[i] = q[i] ;
p[i]=0; // терминальный нуль
```

Так как p и q — указатели, то можно избавиться от переменной i , применяемой для индексирования:

```
while (*q !=0)
{
    *p=*q;
    p++; // указывает на следующий символ
}
```

```

    ++;           // указывает на следующий символ
}
*p = 0;          // терминальный ноль

```

Поскольку постфиксные инкремент и декремент позволяют нам использовать значение и лишь потом изменить его, можно переписать цикл еще раз:

```

while (*q != 0)
{
    *p++ = *q++;
}
*p = 0;          // терминальный ноль

```

Так как значение выражения $*p++ = *q++$ есть $*q$, то цикл можно записать еще короче:

```
while ( (*p++ = *q++) != 0 ) { }
```

Здесь равенство выражения $*q$ нулю обнаруживается после того, как мы его копируем в $*p$ и инкрементируем p , так что тем самым копируется и терминальный ноль (отдельное присваивание из предыдущего варианта кода становится ненужным). Окончательное сокращение кода достигается тем, что отбрасывается пустой блок, а также явное сравнение с нулем ввиду его избыточности. В итоге, мы приходим к первоначальному варианту кода:

```
while (*p++ = *q++) ;
```

Можно ли сказать, что этот код менее читаем, чем предыдущие версии? Только не для опытных C и C++ программистов. А является ли он более эффективным? Не обязательно (если, конечно, не рассматривать версию с вызовом функции **strlen** () для отдельного вычисления длины строки). Эффективность в сильной степени будет зависеть от архитектуры компьютера и особенностей конкретного компилятора.

Самый эффективный способ копирования строк с терминальным нулем на вашей машине должна обеспечивать соответствующая стандартная библиотечная функция:

```
char* strcpy(char*, const char*); // из <string.h>
```

В более общих случаях можно применить *стандартный алгоритм copy* (§2.7.2, §18.6.1). Везде, где только можно, *пользуйтесь средствами стандартной библиотеки вместо возни с указателями и байтами*. Функции стандартной библиотеки могут быть встраиваемыми (§7.1.1) и даже реализовываться с помощью специализированных машинных инструкций. Поэтому подумайте дважды, прежде чем отдать предпочтение собственному рукотворному коду перед стандартными библиотечными средствами.

6.2.6. Свободная память

Время жизни именованных объектов определяется их областью видимости (§4.9.4). Однако часто нужны объекты, время жизни которых не зависит от области видимости, в которой они были созданы. Таковы, например, объекты, с которыми нужно работать после возврата из функции, где они были созданы. Этот тип объектов создается *операцией new*, а их уничтожение выполняется *операцией delete*. Па-

мять под эти объекты выделяется из так называемой «свободной памяти» (или из «кучи»; динамической памяти).

Посмотрим, как можно написать компилятор в стиле, который мы применили для написания программы-калькулятора (§6.1). Функции синтаксического анализа могли бы строить дерево выражений для его дальнейшего использования генератором кода:

```
struct Enode
{
    Token_value oper;
    Enode* left;
    Enode* right;
    // ...
};

Enode* expr (bool get)
{
    Enode* left=term (get) ;

    for (; ; )
        switch (curr_tok)
        {
            case PLUS:
            case MINUS:
            {
                Enode* n = new Enode;           // создать Enode в свободной памяти
                n->oper=curr_tok;
                n->left=left;
                n->right=term (true) ;
                left=n;
                break;
            }
            default:
                return left;                     // вернуть узел
        }
    }
}
```

Генератор кода использует построенные узлы дерева, а затем удаляет их:

```
void generate (Enode* n)
{
    switch (n->oper)
    {
        case PLUS:
            // ...
            delete n;                             // удалить Enode из свободной памяти
        }
    }
}
```

Объект, созданный операцией **new**, существует до тех пор, пока он не будет явным образом уничтожен операцией **delete**. Только после этого память, занимаемая объектом, освобождается и может далее снова использоваться операциями **new**. Обычно реализации C++ не гарантируют автоматического освобождения памяти

(«сборки мусора») с тем, чтобы их снова могли использовать операции *new*. Поэтому я полагаю, что объекты, созданные операцией *new*, удаляются вручную операцией *delete*. Лишь в случае наличия в конкретной реализации C++ автоматического сборщика мусора можно обойтись без операций *delete* (§С.9.1).

Применять операцию *delete* можно лишь к указателям, значения которых или установлены операцией *new*, или равны нулю. Если *delete* применяется к нулю, то не производится никаких действий.

Более специализированные версии операции *new* обсуждаются в §15.6.

6.2.6.1. Массивы

Массивы объектов также можно создавать операцией *new*. Например:

```
char* save_string (const char* p)
{
    char* s=new char [strlen (p) +1] ;
    strcpy (s,p) ;           // скопировать из p в s
    return s ;
}

int main (int argc, char* argv [])
{
    if (argc < 2) exit (1) ;
    char* p = save_string (argv [1]) ;
    // ...
    delete [] p ;
}
```

Операция *delete* применяется для удаления одиночных объектов; *delete []* используется для удаления массивов.

Чтобы корректно освободить память, выделенную операцией *new*, операции *delete* и *delete []* должны знать размер удаляемого объекта. Отсюда следует, что размер памяти, динамически выделяемой под объект стандартной реализацией операции *new*, несколько больше размера памяти, отводимого под статический объект. В типичном случае используется одно дополнительное слово для хранения размера объекта.

Так как объекты типа *vector* (§3.7.1, §16.3) ничем не хуже иных объектов, то их, естественно, можно динамически размещать в памяти операцией *new* и удалять отсюда (уничтожать) операцией *delete*. Вот пример на эту тему:

```
void f(int n)
{
    vector<int>* p = new vector<int> (n) ;    // индивидуальный объект
    int* q = new int [n] ;                  // массив
    // ...
    delete p ;
    delete [] q ;
}
```

Применять операцию *delete []* можно лишь к указателям на массивы, значения которых или установлены операцией *new*, или равны нулю. Если *delete []* применяется к нулю, то не производится никаких действий.

6.2.6.2. Исчерпание памяти

Реализации операций ***new***, ***delete***, ***new []*** и ***delete []*** используют следующие стандартные функции, прототипы которых представлены в заголовочном файле **<new>** (§19.4.5):

```
void* operator new (size_t) ;           // выделяет память для индивидуального объекта
void* operator delete (void* p) ;      // при p!=0 освобождает выделенную операцией
                                         // new память

void* operator new[] (size_t) ;        // выделяет память под массив
void* operator delete[] (void* p) ;    // при p!=0 освобождает выделенную операцией
                                         // new[] память
```

Когда операции ***new*** требуется выделить память под некоторый объект, вызывается функция ***operator new()***, которая и выделяет нужное количество байт. Аналогично, когда операции ***new []*** требуется выделить память под массив, вызывается функция ***operator new[]()***.

Стандартные реализации функций ***operator new()*** и ***operator new[]()*** не инициализируют выделяемую ими память.

Что произойдет, когда операции ***new*** не удастся найти в свободной памяти блок затребованного размера? По умолчанию в этом случае генерируется исключение ***bad_alloc*** (другие варианты смотри в §19.4.5). Например:

```
void f()
{
    try
    {
        for ( ; ; ) new char[10000] ;
    }
    catch (bad_alloc)
    {
        cerr << "Memory exhausted! \n" ;
    }
}
```

Какова бы ни была память на вашей машине, здесь рано или поздно сработает обработчик исключения ***bad_alloc***.

У нас есть возможность явно указать, что должно происходить при исчерпании памяти операцией ***new***, так как в этот момент операция ***new*** вызывает функцию, зарегистрированную с помощью стандартной функции ***set_new_handler()*** (ее прототип дан в заголовочном файле **<new>**), если, конечно, такая регистрация вообще имела место. Например:

```
void out_of_store ()
{
    cerr << "operator new failed: out of store \n" ;
    throw bad_alloc () ;
}

int main ()
{
    set_new_handler (out_of_store) ; // делаем out_of_store обработчиком нехватки памяти
```

```
for ( ; ; ) new char [ 10000 ] ;
cout << "done\n" ;
}
```

Эта программа выведет строку

operator new failed: out of store

В §14.4.5 приведена правдоподобная реализация функции **operator new** (), которая проверяет наличие зарегистрированного обработчика и генерирует исключение **bad_alloc** в случае отсутствия такового. Зарегистрированный обработчик может делать что-нибудь более умное, чем просто завершение программы. Например, обработчик может все же попытаться отыскать память, запрошенную операцией **new**, реализовав некоторый вариант сборки мусора (операция **delete** станет в таком случае ненужной). Новичок, конечно, с такой задачей вряд ли справится, но можно приобрести готовый и оттестированный обработчик от сторонних производителей (§С.9.1).

Предоставляя новый обработчик, мы контролируем ситуацию с исчерпанием памяти при обращении к операции **new**. Существуют два способа управления процессом выделения памяти. Можно либо предоставить нестандартные реализации функций **operator new** () и **operator delete** (), которые вызываются стандартной формой операции **new**, либо положиться на информацию о блоке памяти, предоставленную пользователем (§10.4.11, §19.4.5).

6.2.7. Явное приведение типов

Иногда приходится иметь дело с «сырой» памятью (*raw memory*), то есть памятью, которая предназначена для хранения объектов неизвестного компилятору типа. К примеру, распределитель памяти возвращает значение типа **void***, указывающее на выделенный блок памяти, или мы трактуем целое число как адрес устройства ввода/вывода:

```
void* malloc (size_t) ;

void f()
{
    int* p=static_cast<int*> ( malloc (100) ) ;    // память выделена под целые

    IO_device* dI=
    reinterpret_cast<IO_device*> (0Xff00) ;    // устройство по адресу 0Xff00
    // ...
}
```

Компилятор не знает тип объекта, адресуемого указателем типа **void***. Не может он и знать, корректен ли адрес **0Xff00**. Следовательно, именно программист несет всю ответственность за корректность преобразований типов. *Явное преобразование типов* (*приведение типов* — *casting*) на практике важно и применимо. Но по установившейся традиции его применение избыточно и часто служит источником ошибок.

Операция **static_cast** осуществляет преобразования между родственными типами данных, например от указателя на некоторый класс к указателю на другой класс из той же иерархии наследования, от интегрального типа к перечислению, от типа

с плавающей запятой к интегральному типу. Операция ***reinterpret_cast*** осуществляет преобразования между несвязанными типами, например от целого к указателю или от одного указателя к иному произвольному указательному типу. Различие между этими операциями позволяет компилятору немного контролировать процесс приведения типов операцией ***static_cast*** и помогает программисту следить за потенциально опасными приведениями, выполняемыми в программе с помощью ***reinterpret_cast***. Преобразования типов с помощью ***static_cast*** более-менее переносимы, а преобразования через ***reinterpret_cast*** — практически никогда. Нельзя утверждать со 100%-ой уверенностью, но все же чаще всего ***reinterpret_cast*** дает новую трактовку типа, оставляя последовательности битов аргумента неизменными. Если конечный тип преобразования не уже, чем исходный (то есть содержит не меньшее число бит), то можно еще раз применить ***reinterpret_cast*** в обратном порядке и вернуться к исходному аргументу. В общем случае, результат операции ***reinterpret_cast*** гарантированно приемлем для использования лишь тогда, когда преобразуемое значение соответствует целевому типу.

Если вы решили выполнить явное приведение типа, то сначала задумайтесь, насколько оно действительно необходимо. В C++ необходимость в явных преобразованиях типов встречается существенно реже, чем в C (§1.6) или в ранних версиях C++ (§1.6.2, §B.2.3). Во многих программах можно полностью избавиться от явного преобразования типов. В других — тщательно локализовать их внутри немногочисленных процедур. В настоящей книге реальные случаи применения явных преобразований типов встречаются лишь в §6.2.7, §7.7, §13.5, §13.6, §17.6.2.3, §15.4, §25.4.1 и §E.3.1.

Имеются также операция преобразования ***dynamic_cast***, действие которой контролируется на этапе выполнения программы (§15.4.1), и операция преобразования ***const_cast*** (§15.4.2.1), аннулирующая действие модификаторов ***const*** и ***volatile***.

Язык C++ унаследовал от C конструкцию $(T)e$, означающую любое преобразование, которое может быть выполнено комбинацией операций ***static_cast***, ***reinterpret_cast*** и ***const_cast***, для получения значения типа T из выражения e (§B.2.3). Такой стиль преобразований опаснее рассмотренных выше именованных операций приведения, ибо его труднее находить в больших программах и он не отражает точных намерений программиста, поскольку $(T)e$ может выполнять и переносимое преобразование между родственными типами данных, и непереносимое преобразование несвязанных между собой типов и снятие действия модификатора ***const*** с указателя. Без точного знания типа T и типа выражения e ничего определенного сказать нельзя.

6.2.8. Конструкторы

Для конструирования (создания) значения типа T из значения e используется функциональная форма записи $T(e)$. Например:

```
void f(double d)
{
    int i=int(d);           // усекаем d (отбрасываем дробную часть)
    complex z=complex(d);   // создаем complex из d
    //
```

Иногда конструкцию $T(e)$ называют *приведением типов в функциональном стиле* (*function-style cast*). К сожалению, для встроенного типа T запись $T(e)$ эквивалентна записи $(T)e$ (§6.2.7). Это означает, что для многих встроенных типов применение $T(e)$ небезопасно. Например, значения арифметических типов могут оказаться *усеченными* (*truncated*). Даже явные преобразования от более длинных целых типов к более коротким (например, из **long** в **char**) приводят к непереносимым, зависящим от реализации результатам. Я стараюсь применять конструкцию $T(e)$ только в четко определенных случаях: для сужающих арифметических преобразований (§C.6), для приведения целых к перечислениям (§4.8) и для конструирования объектов пользовательских типов (§2.5.2, §10.2.3).

Преобразования указателей нельзя выполнять непосредственно в виде $T(e)$. К примеру, **char*** (2) является синтаксической ошибкой. К сожалению, такую защиту против опасных приведений указателей можно обойти, используя синонимы указательных типов, вводимые с помощью **typedef** (§4.9.7).

Конструкция $T()$ используется для создания значения по умолчанию для типа T . Например:

```
void f(double d)
{
    int j=int();           // умолчательное целое значение
    complex z=complex();  // умолчательное значение для типа complex
    // ...
}
```

Для встроенных типов значением по умолчанию является нуль, преобразованный в соответствующий тип (§4.9.5). Таким образом, **int()** есть просто другая форма записи нулевого целого значения. Для пользовательского типа T , действие выражения $T()$ определяется так называемым конструктором по умолчанию (если он имеется) (§10.4.2).

Роль конструирующих выражений для встроенных типов приобретает особую важность в случае определения шаблонов, когда программист не знает, будет ли шаблонный параметр соответствовать встроенному или пользовательскому типам данных (§16.3.4, §17.4.1.2).

6.3. Обзор операторов языка C++

Ниже приведена компактная сводка операторов языка C++.

Синтаксис операторов	
<i>оператор:</i>	
	<i>объявление</i>
	{ <i>последовательность-операторов</i> _{опт} }
	try { <i>последовательность-операторов</i> _{опт} } <i>список-обработчиков</i>
	<i>выражение</i> _{опт} ;
	if (<i>условие</i>) <i>оператор</i>
	if (<i>условие</i>) <i>оператор</i> else <i>оператор</i>

Синтаксис операторов

```

switch ( условие ) оператор
while ( условие ) оператор
do оператор while ( выражение ) ;
for ( инициализирующий-оператор условиеопт; выражениеопт ) оператор
case константное выражение : оператор
default: оператор
break;
continue;
return выражениеопт;
goto идентификатор;
идентификатор: оператор

последовательность-операторов:
    оператор последовательность-операторовопт

условие :
    выражение
    спецификатор-типа объявитель = выражение

список-обработчиков:
    catch ( объявление-исключения ) { последовательность-операторовопт }
    список-обработчиков список-обработчиковопт

```

Здесь *опт* означает «optional», то есть «необязательно».

Обратите внимание на то, что объявление является оператором, и что нет никаких операторов присваивания и операторов вызова функций; *присваивания и вызов функции — это выражения, использующие соответствующие операции*. Операторы для обработки исключений, так называемые **try**-блоки (*try-blocks*), будут рассмотрены в §8.3.1.

6.3.1. Объявления как операторы

Объявление является оператором. Если только переменная не объявлена с модификатором **static**, то инициализатор выполняется всякий раз, как только поток управления проходит через соответствующее объявление (§10.4.8). Причина, по которой разрешается применять объявления всюду, где допускается оператор (и еще в некоторых местах; §6.3.2.1, §6.3.3.1), заключается в стремлении уменьшить количество ошибок, связанных с применением неинициализированных переменных, и для лучшей локализации кода. Редко когда возникают причины для объявления переменных ранее момента, когда становятся известными (доступными) их начальные значения. Например:

```

void f( vector<string>& v, int i, const char* p )
{
    if (p==0) return;

```

```

if ( $i < 0$  ||  $v.size() \leq i$ ) error ("bad index");
string  $s = v[i]$ ;
if ( $s == p$ )
{
    // ...
}
// ...
}

```

Возможность поместить объявление после некоторого количества исполняемого кода существенно для констант и вообще для стиля программирования, который можно назвать стилем однократных присваиваний, когда значения объектов после их инициализации не меняются. Для пользовательских типов с точки зрения эффективности кода также важно отложить определение объектов до момента появления приемлемого инициализатора. Например,

```

string  $s$ ; /* ... */  $s = \text{"The best is the enemy of the good."}$ ;

```

воплне может оказаться менее эффективным, чем

```

string  $s = \text{"Voltaire"}$ ;

```

Наиболее общей причиной, по которой переменные объявляются без инициализации, является необходимость использования отдельных операторов для придания им конкретных значений. Например, когда значения переменных и массивов извлекаются из потока ввода.

6.3.2. Операторы выбора (условные операторы)

Значение выражения может проверяться в операторах **if** или **switch**:

```

if (условие) оператор
if (условие) оператор else оператор
switch (условие) оператор

```

Операции сравнения

```

==    !=    <    <=    >    >=

```

возвращают логическое значение **true**, если сравнение истинно, и логическое **false** в противном случае.

Для оператора **if** выполняется первый подчиненный оператор (statement), если значение проверяемого выражения (condition) не равно нулю, или второй подчиненный оператор (если он есть) в противном случае. Отсюда следует, что *любое арифметическое выражение или выражение с указателями может использоваться в качестве проверяемого выражения (условия)*. Например, если x целое, то

```

if ( $x$ )    // ...

```

означает

```

if ( $x \neq 0$ )    // ...

```

Для указателя p , следующий код

```

if ( $p$ )    // ...

```

представляет собой прямую проверку «указывает ли p на действительный объект?», в то время как

```
if ( $p \neq 0$ ) // ...
```

делает то же самое, но косвенно, сравнивая p с нулевым значением, которое, как известно, не может быть корректным адресом объекта в памяти. Заметим, что «нулевой» указатель не на всех машинах представляется битовой последовательностью из одних нулей (§5.1.1). Все проверенные мной компиляторы для обеих форм сравнения генерировали одинаковый код.

Логические операции

```
 $\&\&$  | | !
```

чаще всего используются именно в проверяемых выражениях (условиях). *Операции $\&\&$ и $||$ вообще не вычисляют свой второй (правый) аргумент, если в этом отсутствует необходимость.* Например,

```
if ( $p \&\& I < p \rightarrow count$ ) // ...
```

сначала проверяет p на равенство нулю. Выражение $I < p \rightarrow count$ вычисляется (проверяется) только в случае, когда p не равен нулю.

Некоторые условные операторы (операторы **if**) легко могут быть заменены на условные выражения (*conditional-expressions*). Например,

```
if ( $a \leq b$ )  
     $max = b$ ;  
else  
     $max = a$ ;
```

лучше выразить следующим образом:

```
 $max = (a \leq b) ? b : a$ ;
```

Здесь круглые скобки вокруг проверяемого условия не обязательны, но мне кажется, что с ними код читается лучше.

Оператор **switch** можно заменить на эквивалентный набор операторов **if**. К примеру,

```
switch ( $val$ )  
{  
    case 1:  
         $f()$  ;  
        break ;  
    case 2:  
         $g()$  ;  
        break ;  
    default :  
         $h()$  ;  
        break ;  
}
```

можно представить и как

```
if ( $val == 1$ )  
     $f()$  ;  
else if ( $val == 2$ )
```



```

    g() ;
else
    h() ;

```

Смысл тот же самый, но первая форма (оператор **switch**) предпочтительнее, ибо явным образом отражает центральную идею о сравнении значения выражения с набором констант. Это делает оператор **switch** лучше читаемым в сложных случаях, и даже генерируемый машинный код может оказаться более эффективным. Обратите внимание на то, что каждая *case*-ветвь оператора **switch** должна специальным образом завершаться, если только вы не хотите, чтобы продолжали работать и последующие ветви. Например,

```

switch (val)
{
    case 1:
        cout<< "case 1\n" ;
    case 2:
        cout<< "case 2\n" ;
    default:
        cout<< "default: case not found\n" ;
}

```

при **val==1** выдаст (к изумлению непосвященных) следующий результат:

```

case 1
case 2
default: case not found

```

Случай, когда такой результат соответствует намерению программиста, лучше специально комментировать, чтобы легче было по отсутствию комментариев находить участки кода, в которых подобного рода эффекты не планировались, но ошибочным образом все же состоялись. Чаще всего для завершения *case*-ветвей оператора **switch** используется оператор **break**, но также применяется и оператор **return** (§6.1.1).

6.3.2.1. Объявления в условиях

Во избежание случайного неправильного использования переменных их лучше объявлять в максимально узких областях видимости. В частности, лучше откладывать определение локальной переменной до появления подходящего для нее начального значения, что автоматически предотвращает непреднамеренное использование этой переменной до инициализации.

Еlegantное применение этих идей заключается в *объявлении переменной внутри условий*. Рассмотрим пример:

```

if (double d = prim (true) )
{
    left /= d;
    break;
}

```

Здесь **d** объявляется и инициализируется, после чего проверяется в качестве условия. Область видимости переменной **d** начинается в точке ее объявления и распространяется

няется на все блоки, контролируемые условием. Например, будь в данном примере еще и *else*-ветвь оператора *if*, область видимости *d* распространялась бы на обе ветви.

Традиционной альтернативой является объявление *d* до условия. Однако это открывает лазейки для использования переменной *d* в неинициализированном состоянии, а также вне предназначенной для нее области действия:

```
double d;
// ...
d2 = d;    // oops!
// ...
if (d = prim (true) )
{
    left /= d;
    break;
}
// ...
d = 2.0;    // два разных использования d
```

В дополнение к рассмотренным преимуществам объявление переменной в условиях приводит еще и к более компактному исходному коду.

Объявление и инициализация в условиях может распространяться лишь на единственную переменную или константу.

6.3.3. Операторы цикла

Циклическое выполнение кода реализуется операторами *for*, *while* и *do*:

```
while (условие) оператор
do оператор while (выражение) ;
for (инициализирующий_оператор условиеопт ; выражениеопт) оператор
```

Каждый из перечисленных управляющих операторов обеспечивает повторное выполнение оператора *statement* (называемого управляемым оператором или телом цикла) до тех пор, пока проверяемое выражение (condition или expression) не примет значения *false* или пока программист не прервет цикл каким-либо иным способом.

Оператор *for* предназначен для стандартной организации циклов. В этом операторе переменная цикла, условие окончания и выражение для изменения переменной цикла компактно записываются в единственной строке в самом начале цикла. Это в сильнейшей степени увеличивает читаемость кода и, тем самым, уменьшает вероятность ошибок. В случае отсутствия необходимости в инициализации цикла инициализирующую часть (*for-init-statement*) можно опустить. Если опущено условие окончания, то цикл будет выполняться вечно, если только программист не предусмотрит его окончание за счет применения операторов *break*, *return*, *goto*, *throw* или менее очевидными способами, такими как вызов стандартной функции *exit()* (§9.4.1.1). Если опущено выражение для изменения переменной цикла, то эту задачу нужно выполнить в теле цикла. Если *проектируемый цикл* не вписывается в каноническую схему «ввел переменную цикла, проверил условие окончания, изменил значение переменной цикла», то лучше применить оператор *while*. Цикл *for* можно применить для записи бесконечного цикла (с отсутствием явно заданных условий окончания):

```
for ( ; ; )           // "вечно"
{
    // ...
}
```

Цикл **while** заставляет управляемый оператор исполняться до тех пор, пока условие цикла не станет *ложным*. Я предпочитаю использовать цикл **while** вместо цикла **for** в случаях, когда нет очевидной переменной цикла или когда ее модификацию лучше выполнять где-нибудь внутри тела цикла. Очевидным примером цикла без явной переменной цикла является цикл ввода:

```
While (cin>>ch) // ...
```

Мой опыт показывает, что цикл **do** скорее является источником ошибок и недоразумений, чем острой необходимостью. Единственным поводом к его применению является тот факт, что тело этого цикла всегда исполняется хотя бы один раз до самой первой проверки условия цикла. Но для надежного выполнения первой итерации все равно должно проверяться некоторое условие. Я обнаружил, что чаще, чем это можно предположить, условие корректности первого прохода цикла либо не обеспечивалось с самого начала, либо переставало соблюдаться после модификации текста программы. Также, на мой взгляд, предпочтительнее формулировать условие окончания цикла в его начале, где оно бросается в глаза и его можно проанализировать. В общем, я предпочитаю циклов **do** не применять.

6.3.3.1. Объявления в операторах цикла for

В инициализирующей части оператора **for** можно объявлять переменные. Область видимости объявленной таким образом переменной (переменных) распространяется до конца цикла. Например:

```
void f(int v[] , int max)
{
    for (int i=0; i<max; i++) v[i]=i*i;
}
```

Если конечное значение переменной цикла нужно использовать после его окончания, то переменную цикла следует объявить вне оператора цикла **for** (§6.3.4).

6.3.4. Оператор goto

В языке C++ пресловутый оператор **goto** сохранен:

```
goto идентификатор;
идентификатор: оператор
```

Оператор **goto** полезен в ряде случаев и в обычном высокоуровневом программировании, но он особо полезен, когда программу создает не человек, а другая программа: например, при автоматической генерации парсера на базе формальной грамматики. Также **goto** полезен для программ реального времени, когда нужно с высокой эффективностью реализовать выход из внутреннего цикла.

Областью действия меток (*labels*) является тело функции, в котором они находятся. Отсюда следует, что возможны переходы (по **goto**) внутрь блоков и из блоков на-

ружу. Только при этом нельзя перепрыгивать через объявления с инициализациями и непосредственно внутрь обработчиков прерываний (§8.3.1).

Важным примером использования *goto* в обычном коде является организация с его помощью выхода из вложенных циклов или операторов *switch* (оператор *break* обеспечивает выход лишь из одного уровня вложенности). Например:

```
void f()
{
    int i;
    int j;
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) if (nm[i][j] == a) goto found;
        // not found
        // ...
found:
    // nm[i][j] == a
}
```

Имеется также оператор *continue*, который *передает управление в конец тела цикла* (§6.1.5).

6.4. Комментарии и отступы

Разумное применение комментариев и согласованная система отступов могут сделать процесс чтения программы более приятным и способствовать более быстрому ее изучению и пониманию. Имеется несколько общеупотребительных стилей для отступов строк кода. Я не вижу никаких объективных причин для предпочтения какого-либо из этих стилей (хотя у меня, как и у любого программиста, есть свой привычный стиль). То же самое можно сказать и о стилях комментирования.

Вообще говоря, комментариями можно даже ухудшить читаемость программы. Компилятор существа комментариев не понимает и не может гарантировать, что комментарий:

1. Содержателен.
2. Имеет отношение к программе.
3. Не устарел.

Многие программы содержат комментарии, понять которые невозможно, которые противоречивы и даже просто неверны. Плохие комментарии хуже их отсутствия.

Если что-то можно отразить явным образом средствами языка, то так и нужно делать, а не ограничиваться простым упоминанием в комментариях. Вот примеры на эту тему:

```
// переменную v нужно проинициализировать
// переменную v должна использовать лишь функция f()
// вызовите функцию init() до вызова любой другой функции из этого файла
// вызовите функцию cleanup() в конце вашей программы
// не пользуйтесь функцией weird()
// функция f() принимает два аргумента
```

При правильном кодировании на C++ подобного рода комментарии просто не нужны. Вместо них лучше полагаться на правила компоновки (§9.2) и области видимости, на инициализацию и деинициализацию объектов классов (§10.4.1).

Если что-то ясно выражено средствами языка, не надо это повторять в комментариях. Например:

```
a=b+c;           // a принимает значение, равное b+c
count++;        // инкрементируем переменную counter
```

Такие комментарии не только не нужны, но они еще и вредны. Они увеличивают совокупный объем кода, подлежащий чтению, они зачастую затуманивают структуру программы и могут быть просто неверными. Заметим, однако, что такие комментарии интенсивно используются в учебниках, и в данной книге в том числе. Этим, помимо прочего, программы из учебников отличаются от реальных профессиональных программ.

Я предпочитаю применять комментарии в следующих случаях:

1. В начале каждого исходного файла — комментарии, поясняющие, что общего у всех представленных в файле объявлений, ссылки на литературу и другие источники, соображения по поводу дальнейшего сопровождения и т.д.
2. Комментарии к классам, шаблонам, пространствам имен.
3. Комментарии к каждой нетривиальной функции, поясняющие ее назначение, алгоритмы (если они не очевидны), и, возможно, некоторые предположения о контексте вызова.
4. Комментарии к переменным и константам из глобального или именованного пространств имен.
5. Комментарии к неочевидным или непереносимым участкам кода.
6. Редко в иных случаях.

Например:

```
// tbl.c: Реализация таблицы символов.
/*
    Исключение методом Гаусса.
    См. Ralston: "A first course ..." pg 411.
*/
// swap () предполагает раскладку стека как в SGI R6000.
*****

    Copyright (c) 1997 AT&T, Inc.
    All rights reserved

    *****/
```

Хорошо продуманные и хорошо написанные комментарии являются неотъемлемой частью хороших программ. Написание таких комментариев столь же сложно, как и написание собственно кода программы. Стоит развивать и культивировать искусство написания хороших комментариев.

Отметим, что если в функции применяются исключительно комментарии вида `//`, то целые фрагменты ее кода можно закоментировать (и быстро раскомментировать) с помощью `/* */`.

6.5. Советы

1. Предпочитайте всем библиотекам и «самодельному коду» стандартную библиотеку; §6.1.8.
2. Избегайте слишком сложных выражений; §6.2.3.
3. Используйте круглые скобки в случае сомнений по поводу приоритетов операций; §6.2.3.
4. Избегайте явных приведений типов; §6.2.7.
5. Если явное приведение типов необходимо, используйте специфические (именованные) операции приведения вместо операций в C-стиле; §6.2.7.
6. Используйте конструкции $T(e)$ только тогда, когда правила конструирования четко определены; §6.2.8.
7. Избегайте выражений с неопределенным порядком вычисления; §6.2.2.
8. Избегайте применения оператора *goto*; §6.3.4.
9. Избегайте применения циклов *do*; §6.3.3.
10. Не объявляйте переменные до того, как станут известны инициализирующие их значения; §6.3.1, §6.3.2.1, §6.3.3.1.
11. Пишите ясные и краткие комментарии; §6.4.
12. Придерживайтесь согласованного стиля отступов; §6.4.
13. Для переопределения глобальной функции *operator new*() применяйте члены классов (§15.6); §6.2.6.2.
14. При чтении ввода всегда помните о возможных «сюрпризах»; §6.1.3.

6.6. Упражнения

1. (*1) Перепишите следующий цикл *for* в виде эквивалентного *while* цикла:

```
for (i=0; i<max_length; i++)
    if (input_line[i] == ' ? ' ) quest_count++;
```

Перепишите так, чтобы проверяемой величиной был указатель и условие цикла имело вид $*p == ' ? '$.

2. (*1) Расставьте скобки в следующих выражениях:

```
a = b + c * d < 2 & 8
a & 077! = 3
a == b | a == c && c < 5
c = x! = 0
0 <= i < 7
f(1, 2) + 3
a = - 1++ b -- 5
a = b == c++
a = b = c = 0
a[4][2] * = * b ? c : * d * 2
a-b, c=d
```

3. (*2) Введите последовательность возможно разделенных пробельными символами пар (имя, значение). Имя — единственное слово, ограниченное пробельными символами. Значение формируется целым числом или числом с плавающей запятой. Вычислите и выведите сумму и среднее как для каждого отдельного имени, так и для всех имен (см. §6.1.8).
4. (*1) Напишите таблицу результатов всех побитовых логических операций (§6.2.4) для всех возможных комбинаций операндов 0 и 1.
5. (*1.5) Приведите 5 конструкций языка C++, смысл которых не определен (§C.2); (*1.5) Приведите 5 конструкций языка C++, смысл которых зависит от реализации (§C.2).
6. (*1) Приведите 10 примеров непереносимого кода на C++.
7. (*2) Напишите 5 выражений, порядок вычисления которых не определен. Выполните код, чтобы посмотреть, что при этом реально делается в разных реализациях.
8. (*1.5) Что происходит при делении на ноль в вашей системе? Что происходит при переполнении (или потере точности)?
9. (*1) Расставьте скобки в следующих выражениях:


```
*p++
*--p
++a--
(int*)p->m
*p.m
*a[i]
```
10. (*2) Напишите следующие функции: *strlen()*, которая возвращает длину строк в С-стиле; *strcpy()*, которая копирует содержимое одной С-строки в другую; *strcmp()*, которая сравнивает содержимое двух С-строк. Решите, какого типа должны быть аргументы и возвращаемые значения. Затем сравните ваши варианты со стандартными библиотечными версиями, объявленными в *<cstring>* (*<string.h>*) и рассмотренными в §20.4.1.
11. (*1) Проверьте, как компилятор реагирует на ошибки в следующем фрагменте:


```
void f(int a, int b)
{
    if(a=3)           // ...
    if(a&077==0)      // ...
    a := b+1;
}
```

Придумайте еще несколько ошибок и проверьте реакцию компилятора на них.
12. (*2) Модифицируйте программу из §6.6[3], чтобы она вычисляла и медиану.
13. (*2) Напишите функцию *cat()*, которая принимает в качестве аргументов две С-строки и возвращает конкатенированную С-строку. Используйте операцию *new* для выделения памяти под результат.
14. (*2) Напишите функцию *rev()* для реверсирования содержимого С-строки.

15. (*1.5) Что делает следующий пример?

```
void send (int* to, int* from, int count)
// Полезные комментарии умышленно удалены.
{
    int n = (count+7) / 8;
    switch (count%8)
    {
        case 0: do { *to++=*from++;
        case 7:      *to++=*from++;
        case 6:      *to++=*from++;
        case 5:      *to++=*from++;
        case 4:      *to++=*from++;
        case 3:      *to++=*from++;
        case 2:      *to++=*from++;
        case 1:      *to++=*from++;
                    } while (--n>0) ;
    }
}
```

Зачем кому-то может потребоваться подобный код?

16. (*2) Напишите функцию *atoi(const char*)*, которая принимает С-строку, содержащую цифры и возвращает соответствующее целое. Например, для *atoi("123")* должно получиться целое число *123*. Модифицируйте функцию так, чтобы помимо десятичных чисел обрабатывались также восьмеричные и шестнадцатеричные.
17. (*2) Напишите функцию *itoa(int i, char b[])*, которая формирует строковое представление *i* в *b* и возвращает *b*.
18. (*2) Наберите текст программы-калькулятора и заставьте его работать. Не экономьте время, применяя заранее приготовленный кем-то текст. Вы многому научитесь, отыскивая и исправляя «глупые мелкие ошибки».
19. (*2) Модифицируйте программу-калькулятор, чтобы она сообщала номера строк, в которых произошла ошибка.
20. (*3) Разрешите пользователю определять функции для программы-калькулятора. Подсказка: определите функцию в виде последовательности операций в порядке, в котором их ввел пользователь. Последовательность можно хранить либо в виде строки, либо как список лексем. При вызове функции читайте и выполняйте эти операции. Если допускаются аргументы у пользовательских функций, придумайте соответствующую форму записи.
21. (*1.5) Модифицируйте программу-калькулятор так, чтобы вместо статических переменных *number_value* и *string_value* использовалась структура *symbol*.
22. (*2.5) Напишите код, очищающий программу на С++ от комментариев. Пусть она читает из *cin*, удаляет комментарии вида *//* и */* */* и записывает результат в *cout*. Не заботьтесь о внешнем виде результирующей программы (это была бы более сложная задача). Не заботьтесь о корректности программ. Не забудьте про символы *//*, */* */* в комментариях, строках и символьных константах.
23. (*2) Выполните просмотр ряда программ, чтобы составить представление о применяемых стилях отступов, именования и комментирования.

Функции

*Итерация — от человека,
рекурсия — от Бога.
— Л. Питер Дойч*

Объявления и определения функций — передача аргументов — возвращаемые значения — перегрузка функций — разрешение неоднозначностей — аргументы по умолчанию — неуказанное число аргументов — указатели на функции — макросы — советы — упражнения.

7.1. Объявления функций

Обычно, сделать что-либо в C++ означает вызвать функцию. Определить функцию — значит задать способ выполнения работы. Функцию нельзя вызвать, если она не была предварительно объявлена.

Объявление функции (function declaration) предоставляет ее имя, тип возвращаемого значения (если оно есть), а также число и типы аргументов, которые требуется передавать функции при ее вызове. Например:

```
Elem* next_elem ( ) ;  
char* strcpy (char* to, const char* from.) ;  
void exit (int) ;
```

Семантика передачи аргументов идентична семантике инициализации. Производится проверка типов аргументов и при необходимости выполняется неявное приведение типов. Например:

```
double sqrt (double) ;  
double sr2=sqrt (2) ;           // вызов sqrt() с аргументом double(2)  
double sr3=sqrt ("three") ;    // error: sqrt() требует аргумент типа double
```

Значение таких проверок и преобразований типов не следует недооценивать.

Объявления функций могут содержать имена аргументов. Это полезно программисту, читающему код, но компилятор их просто игнорирует. Как упоминалось

в §4.7 тип **void** для возвращаемого значения функции фактически означает его отсутствие.

7.1.1. Определения функций

Любая вызываемая в программе функция должна быть где-то определена, причем лишь один раз. *Определение функции (function definition)* есть ее объявление плюс тело функции. Например:

```
extern void swap (int*, int*) ; // объявление
void swap (int* p, int* q)      // определение
{
    int t=*p;
    *p = *q;
    *q = t;
}
```

Все типы в определении функции и ее объявлениях *обязаны совпадать*. Однако имена аргументов совпадать не обязаны, так как они не являются частью типа.

Нередки случаи, когда в определении функции часть аргументов не используется:

```
void search (table* t, const char* key, const char*)
{
    // третий аргумент не используется
}
```

Как показано в данном примере, *неиспользуемому аргументу можно не давать имени* вообще. Обычно, неименованные аргументы в определениях функций появляются либо вследствие произведенных упрощений, либо в качестве резерва на будущее. В обоих случаях само наличие реально неиспользуемого аргумента в определении функции позволяет не трогать при этом клиентский код, вызывающий функцию.

Можно определить функцию с модификатором **inline**. Например:

```
inline int fac (int n)
{
    return (n<2) ? 1 : n*fac (n-1) ;
}
```

Такие функции называют *встраиваемыми (inline functions)*, так как модификатор **inline** указывает компилятору, что он должен пытаться осуществлять прямое встраивание кода функции **fac** () во все места, где эта функция вызывается, а не реализовывать этот код в единственном экземпляре, доступ к которому выполняется через стандартный механизм вызова. Умный компилятор может сгенерировать константу **720** на месте вызова **fac** (6) . Из-за того, что могут быть объявлены встраиваемыми рекурсивные или взаимно рекурсивные функции, гарантировать реальное встраивание кода функции в каждое место ее вызова невозможно. В зависимости от степени интеллектуальности конкретного компилятора мы можем реально получить для нашего примера и **720**, и **6*fac** (5) , и просто обычный вызов **fac** (6) .

Чтобы обеспечить возможность встраивания в случае рядового (а не сверхинтеллектуального) компилятора и компоновщика, нужно размещать определение (а не

только объявление) функции с модификатором *inline* в той же самой области видимости (§9.2). Модификатор *inline* не влияет на семантику функции. В частности, встраиваемые функции по-прежнему имеют уникальный адрес, так же как и их статические переменные (§7.1.2).

7.1.2. Статические переменные

Локальные переменные инициализируются всякий раз, как только поток исполнения достигает точки их определения. Это происходит при каждом вызове функции и каждый такой вызов располагает собственной копией локальной переменной. Если же локальная переменная объявляется с модификатором *static*, то единственный, статически размещаемый в памяти объект (§C.9) будет использоваться для представления этой переменной во всех вызовах функции. Такая переменная будет инициализироваться лишь однажды, когда поток управления первый раз достигнет точки ее определения. Например:

```
void f(int a)
{
    while (a-->0)
    {
        static int n = 0;    // инициализируется один раз
        int x=0;             // иниц-ся 'a' раз при каждом вызове f()

        cout << "n == " << n++ << ", x == " << x++ << '\n' ;
    }
}

int main ()
{
    f(3) ;
}
```

В результате будет получен следующий вывод:

```
n == 0, x == 0
n == 1, x == 0
n == 2, x == 0
```

Статические переменные наделяют функции «долговременной памятью» без необходимости применения глобальных переменных, доступ к которым из других функций способен привести к их порче (§10.2.4).

7.2. Передача аргументов

При вызове функции выделяется память под ее формальные аргументы, и каждый формальный аргумент инициализируется значением соответствующего фактического аргумента. Семантика передачи аргументов идентична семантике инициализации. В частности, типы фактических параметров проверяются относительно типов формальных параметров, и в случае необходимости выполняются либо преобразования стандартных типов, либо преобразования, определенные для пользовательских типов. Существуют специальные правила для передачи массивов

(§7.2.1), средства для передачи аргументов без проверки типов (§7.6) и средства, предназначенные для задания аргументов по умолчанию (§7.5). Рассмотрим следующую функцию:

```
void f(int val, int& ref)
{
    val++;
    ref++;
}
```

Когда осуществляется вызов *f()*, выражение *val++* инкрементирует локальную копию первого фактического аргумента, в то время как выражение *ref++* инкрементирует непосредственно второй фактический аргумент. Например,

```
void g()
{
    int i=1;
    int j=1;
    f(i, j);
}
```

увеличит *j*, но не *i*. Первый аргумент, *i*, передается *по значению* (*by value*), а второй аргумент, *j*, передается *по ссылке* (*by reference*). Как упоминалось в §5.5, функции, которые модифицируют передаваемые им по ссылке аргументы, делают программу менее ясной и их следует, чаще всего, избегать (однако, см. §21.3.2). Правда, передача параметров большого размера по ссылке намного эффективнее их передачи по значению. В таком случае следует объявлять аргументы с модификатором *const*, дабы явным образом подчеркнуть, что передача аргументов по ссылке осуществляется исключительно ради эффективности, а не для того, чтобы функция могла их модифицировать:

```
void f(const Large& arg)
{
    // здесь arg невозможно изменить без явного приведения типа
}
```

Отсутствие модификатора *const* в объявлении передаваемого по ссылке аргумента должно восприниматься как явно выраженное намерение модифицировать этот аргумент в теле функции:

```
void g(Large& arg); // предполагается, что g() может изменить arg
```

Аналогично, объявляя аргумент функции как указатель на константу, мы декларируем, что значение объекта, на который ссылается этот указатель, не будет модифицироваться в этой функции. Например:

```
int strlen(const char*); // количество символов в C-строке
char* strcpy(char* to, const char* from); // копирование C-строк
int strcmp(const char*, const char*); // сравнение C-строк
```

Практическая роль аргументов с модификатором *const* возрастает с ростом размера программы.

Следует отметить, что семантика передачи аргументов отличается от семантики присваивания. Это имеет значение для передачи аргументов с модификатором

const, для передачи аргументов по ссылке и для аргументов некоторых пользовательских типов (§10.4.4.1).

Литералы, константы и аргументы, требующие преобразования типов, могут передаваться в функции с **const**& аргументами, и не могут передаваться в функции с не-**const**& аргументами. Разрешение преобразования для аргументов, объявленных как **const T**&, гарантирует передачу любых значений типа **T** через временные объекты (в случае необходимости). Например:

```
float fsqrt (const float&); // sqrt в Fortran-стиле с аргументом-ссылкой

void g (double d)
{
    float r=fsqrt (2.0); // перелача ссылки на временную переменную со значением 2.0f
    r=fsqrt (r);          // передача ссылки на r
    r=fsqrt (d);          // перелача ссылки на временную переменную со значением float(d)
}
```

Запрещение преобразований для не-**const**& аргументов (§5.5) устраняет глупые ошибки, порождаемые созданием временных объектов. Например:

```
void update (float& i);

void g (double d, float r)
{
    update (2.0f); // error: константный аргумент
    update (r);    // передача ссылки на r
    update (d);    // error: требуется приведение типа
}
```

Будь эти вызовы разрешены, **update** () молча изменила бы временные переменные, которые тут же были бы удалены. Для программиста это было бы, скорее всего, неприятным сюрпризом.

7.2.1. Массивы в качестве аргументов

Если аргумент функции объявлен как массив, то при вызове ей передается *указатель на первый элемент массива*. Например:

```
int strlen (const char*);

void f()
{
    char v[] = "an array";
    int i = strlen (v);
    int j = strlen ("Nicholas");
}
```

Таким образом, аргумент типа **T[]** при передаче преобразуется к типу **T***. Отсюда следует, что присваивание значения элементу массива-аргумента означает изменение элемента самого массива (а не копии элемента массива). Другими словами, массивы отличаются от иных типов тем, что их нельзя передать в функцию по значению.

Так как размер массива неизвестен в вызываемой функции, то это могло бы стать неудобством, но есть способы обойти проблему. Для C-строк имеется термин-

нальный нуль, и длина строки, тем самым, легко вычисляется. Для других массивов размер можно передавать дополнительным аргументом. Например:

```
void compute1 (int* vec_ptr, int vec_size) ; // один способ
struct Vec
{
    int* ptr;
    int size;
};

void compute2 (const Vec& v) ; // другой способ
```

В качестве альтернативы вместо массивов можно использовать тип **vector** (§3.7.1, §16.3) из стандартной библиотеки.

С многомерными массивами проблем больше (§С.7), но часто вместо них можно применить массив указателей, который не требует специального обращения. Например:

```
char* day[] = { "mon", "tue", "wed", "thu", "fri", "sat", "sun" } ;
```

Но опять-таки, тип **vector** и другие типы стандартной библиотеки являются прекрасной альтернативой низкоуровневым встроенным типам — массивам и указателям.

7.3. Возвращаемое значение

Функция *должна* возвращать значение, если только ее возврат не объявлен как **void** (функция **main**() является исключением — см. §3.2). И наоборот — функция *не может* возвращать значение, если она объявлена с ключевым словом **void**. Например:

```
int f1 () {} // error: не возвращается значение
void f2 () {} // ok
int f3 () {return 1; } // ok
void f4 () {return 1; } // error: возврат в void функции
int f5 () {return; } // error: отсутствует возвращаемое значение
void f6 () {return; } // ok
```

Возвращаемое значение задается в операторе **return**. Например:

```
int fac (int n) {return (n>1) ? n*fac (n-1) : 1; }
```

Функцию, которая *вызывает саму себя*, называют *рекурсивной* (*recursive*).

Функция может содержать более одного оператора **return**:

```
int fac2 (int n)
{
    if (n>1) return n*fac2 (n-1) ;
    return 1;
}
```

Как и семантика передачи аргументов, *семантика возврата значения из функции идентична семантике инициализации*. Оператор **return** инициализирует *неименованную переменную*, имеющую тип возврата функции. Тип выражения, который содер-

жится в операторе `return`, сопоставляется с объявленным типом возврата функции, и при необходимости выполняются стандартные преобразования типов, или преобразования, определенные пользователем. Например:

```
double f()
{
    return 1;    // 1 неявно преобразуется в double(1)
}
```

Каждый раз при вызове функции выделяется память под ее аргументы и локальные (автоматические) переменные. После возврата из функции эта память считается свободной и может использоваться повторно. Поэтому указатель на локальную переменную возвращать не следует: значение, на которое он указывает, может измениться неожиданно:

```
int* fp() { int local=1; /* ... */ return &local; } // плохо
```

Эта ошибка встречается реже эквивалентной ошибки с возвратом ссылок:

```
int& fr() {int local=1; /* ... */ return local; } // плохо
```

К счастью, компилятор обычно предупреждает, что осуществляется возврат ссылки на локальную переменную.

Функция, объявленная с ключевым словом **void**, не может возвращать значений. Поэтому в теле **void**-функций можно использовать операторы **return** с выражениями вызова других **void**-функций. Например:

```
void g(int* p) ;
void h(int* p) { /* ... */ return g(p) ; } // ок: возвращает "никакое значение"
```

Подобного вида операторы `return` широко применяются при написании шаблонных функций, в которых тип возвращаемого значения является параметром шаблона (§18.4.4.2).

7.4. Перегрузка имен функций

Чаще всего, разным функциям дают разные имена. Однако когда концептуально одинаковые функции выполняют одинаковую работу над объектами разных типов, то удобно дать им одинаковые имена. Это называют *перегрузкой имен функций* (*overloading*). Такая техника всегда применялась для встроенных операций, например, одно и то же имя (обозначение) операции сложения, `+`, используется для сложения целых чисел, чисел с плавающей запятой и указательных типов. Эта идея легко распространяется на функции, определяемые пользователем. Например:

```
void print(int) ; // печать целого
void print(const char*) ; // печать C-строки
```

С точки зрения компилятора у этих функций есть только одна общая черта — имя. Конечно, предполагается, что такие функции должны быть похожи по смыслу, но сам язык не накладывает здесь никаких ограничений, так что в этом вопросе он программисту и не помогает, и не мешает. Таким образом, перегрузка имен функций является просто удобным и наглядным способом именования, особенно для

функций с *расхожими именами*, такими как *sqrt*, *print* или *open*. Когда выбор имени важен с точки зрения семантики, возможность перегрузки имен функций становится ценной вдвойне. Это имеет место, например, для операций $+$, $*$, $<<$ и т.д., для конструкторов (§11.7) и в обобщенном программировании (§2.7.2, глава 18). Когда вызывается функция $f()$, компилятор должен решить, какая именно из функций с этим именем должна быть выбрана. Для выбора компилятор сравнивает типы фактических аргументов вызова с типом формальных параметров всех функций с именем f . Идея состоит в том, чтобы вызвать ту функцию, чьи аргументы подходят больше всего, или выдать ошибку компиляции в случае невозможности такого выбора. Например:

```
void print (double) ;
void print (long) ;

void f()
{
    print (1L) ;    // print(long)
    print (1.0) ;  // print(double)
    print (1) ;    // error: неоднозначность: print(long(1)) или print(double(1))?
}
```

Поиск наиболее подходящей для вызова функции из имеющегося набора перегруженных функций выполняется выбором единственного наилучшего соответствия между типами выражений для аргументов вызова и типами формальных параметров этих функций. Чтобы формализовать понятие наилучшего соответствия, следующие критерии применяются в указанном порядке:

1. Точное совпадение типов, при котором или вообще не нужно выполнять преобразований типов, или только самые тривиальные (имя массива к указателю на первый элемент, имя функции к указателю на функцию, тип T к типу *const T*).
2. Совпадение после «продвижения типов вверх»: для интегральных типов это продвижения *bool* в *int*, *char* в *int*, *short* в *int* и их *unsigned* аналоги (§C.6.1), и продвижение *float* в *double*.
3. Совпадение после стандартных преобразований типов: например, *int* в *double*, *double* в *int*, *double* в *long double*, указатели на производные типы в указатели на базовые типы (§12.2), T^* в *void** (§5.6), *int* в *unsigned int* (§C.6).
4. Совпадение после преобразований типов, определяемых пользователем (§11.4).
5. Совпадение типов из-за *многоточий* ($\dots$ — *ellipsis*) в объявлении функции (§7.6).

Если выявляются два совпадения одного и того же наивысшего уровня, то вызов функции считается *неоднозначным* (*ambiguous*) и отвергается компилятором. Сформулированные *критерии разрешения перегрузки* (*resolution rules*) основаны на соответствующих правилах языков C и C++ для встроенных числовых типов данных (§C.6). Например:

```
void print (int) ;
void print (const char*) ;
void print (double) ;
void print (long) ;
void print (char) ;
```



```

void h(char c, int i, short s, float f)
{
    print(c);           // точное соответствие:      print(char)
    print(i);           // точное соответствие:      print(int)
    print(s);           // интегральное продвижение:  print(int)
    print(f);           // продвижение от float к double: print(double)
    print('a');          // точное соответствие:      print(char)
    print(49);           // точное соответствие:      print(int)
    print(0);            // точное соответствие:      print(int)
    print("a");          // точное соответствие:      print(const char*)
}

```

Для вызова `print(0)` выбирается вариант `print(int)`, поскольку `0` имеет тип `int`. А для вызова `print('a')` выбирается `print(char)`, ибо `'a'` имеет тип `char` (§4.3.1). Причина раздельного рассмотрения преобразований и продвижений заключается в предпочтении безопасных продвижений, таких как `char` в `int`, небезопасным преобразованиям вроде `int` в `char`.

Результат разрешения перегрузки функций не зависит от порядка объявления функций в программе.

Разрешение перегрузки функций базируется на довольно-таки сложном наборе правил, так что иногда программисту выбор компилятора покажется неочевидным. Здесь уместен вопрос, ради чего все это делается? Рассмотрим альтернативу перегрузки функций. Часто приходится выполнять похожие действия над объектами разных типов. Без перегрузки мы в таких случаях вынуждены определять несколько функций с разными именами:

```

void print_int(int);
void print_char(char);
void print_string(const char*);

void g(int i, char c, const char* p, double d)
{
    print_int(i);        // ok
    print_char(c);        // ok
    print_string(p);      // ok
    print_int(c);         // ok? вызывается print_int(int(c))
    print_char(i);        // ok? вызывается print_char(char(i))
    print_string(i);      // error
    print_int(d);         // ok? вызывается print_int(int(d))
}

```

В итоге нужно помнить все функции и не ошибаться в выборе правильного варианта. Это утомительно, препятствует обобщенному программированию (§2.7.2) и фокусирует внимание на относительно низкоуровневых аспектах типов данных. В отсутствие перегрузки к аргументам вызова функций применяются все стандартные преобразования типов. Это может приводить к ошибкам. Так, в последнем примере из четырех ошибочных вызовов компилятором обнаруживается лишь один. Перегрузка же функций повышает шансы компилятора на то, что неправильные аргументы вызова будут им обнаружены и отвергнуты.

7.4.1. Перегрузка и возвращаемые типы

Возвращаемые типы не участвуют в разрешении перегрузки. Это сделано для обеспечения независимости выбора перегруженных операций (§11.2.1, §11.2.4) и вызова функций от окружающего контекста. Рассмотрим пример:

```
float sqrt(float);
double sqrt(double);

void f(double da, float fla)
{
    float fl = sqrt(da);    // вызывается sqrt(double)
    double d = sqrt(da);    // вызывается sqrt(double)
    fl = sqrt(fla);         // вызывается sqrt(float)
    d = sqrt(fla);          // вызывается sqrt(float)
}
```

Если возвращаемые типы принимать во внимание, то уже нельзя по одному лишь виду вызова `sqrt()` решить, о каком варианте функции идет речь.

7.4.2. Перегрузка и области видимости

Перегрузка имен функций имеет место только в *одной и той же области видимости*. Например:

```
void f(int);

void g()
{
    void f(double);
    f(1);                // вызывается f(double)
}
```

Очевидно, что `f(int)` была бы идеальным соответствием вызову `f(1)`, но в текущей области видимости объявлен лишь вариант `f(double)`. В подобных случаях можно вводить и удалять локальные объявления ради достижения желаемого результата. Но как всегда, намеренное сокрытие может быть полезным, а непреднамеренное сокрытие оказаться неприятным сюрпризом. Когда требуется распространить перегрузку поверх классовых областей видимости (§15.2.2) и областей видимости, связанных с пространствами имен (§8.2.9.2), нужно использовать либо *объявления using*, либо *директивы using* (§8.2.2, §8.2.3). См. также §8.2.6.

7.4.3. Явное разрешение неоднозначностей

Объявление слишком малого (или слишком большого) количества перегруженных вариантов функции может приводить к неоднозначностям. Например:

```
void f1(char);
void f1(long);

void f2(char*);
void f2(int*);
```

```
void k (int i)
{
    f1 (i);           // неоднозначность: f1(char) или f1(long)
    f2 (0);           // неоднозначность: f2(char*) или f2(int*)
}
```

В таких случаях следует рассмотреть весь набор перегруженных вариантов функции как единое целое и решить, насколько он логичен с точки зрения семантики функции. Часто разрешить неоднозначность можно простым добавлением еще одного варианта функции. Например, добавляя

```
inline void f1 (int n) { f1 (long (n)) ; }
```

мы добиваемся разрешения неоднозначности $f(i)$ в пользу более широкого типа *long int*.

Для разрешения неоднозначности в конкретном вызове можно воспользоваться явным преобразованием типа. Например:

```
f2 (static_cast<int*> (0)) ;
```

Однако это не более, чем паллиатив — в каждом новом вызове все надо повторять заново.

Некоторых новичков в программировании на C++ раздражают сообщения компилятора о неоднозначностях. Более опытные программисты ценят эти сообщения об ошибках, ибо видят в них своевременные предупреждения об ошибках в проектировании.

7.4.4. Разрешение в случае нескольких аргументов

В рамках имеющихся правил разрешения перегрузки можно быть уверенным в том, что в случаях, когда точность или эффективность вычислений различаются существенно для разных типов данных, выбран будет самый простой вариант функции (алгоритма вычислений). Например:

```
int pow (int, int) ;
double pow (double, double) ;
complex pow (double, complex) ;
complex pow (complex, int) ;
complex pow (complex, double) ;
complex pow (complex, complex) ;

void k (complex z)
{
    int i = pow (2, 2) ;           // вызов pow(int,int)
    double d = pow (2.0, 2.0) ;    // вызов pow(double,double)
    complex z22 = pow (2, z) ;     // вызов pow(double,complex)
    complex z33 = pow (z, 2) ;     // вызов pow(complex,int)
    complex z44 = pow (z, z) ;     // вызов pow(complex,complex)
}
```

В процессе выбора среди перегруженных функций с двумя и более аргументами на основе правил из §7.4 отбираются функции с наилучшими соответствиями по каждому аргументу. Вызывается в итоге та из них, у которой для одного аргумента

соответствие наилучшее, а требующиеся для других аргументов преобразования не хуже необходимых преобразований у остальных функций. Если такой функции не находится, то вызов отвергается как неоднозначный. Например:

```
void g ()
{
    double d = pow (2.0, 2) ;    // error: pow(int(2.0),2) или pow(2.0,double(2))?
}
```

Здесь вызов неоднозначен, потому что фактический аргумент **2.0** наилучшим образом соответствует варианту **pow (double, double)**, а аргумент **2** наилучшим образом соответствует **pow (int, int)**.

7.5. Аргументы по умолчанию

Функции общего назначения обычно имеют больше аргументов, чем это необходимо в простых случаях. Например, функции для конструирования классовых объектов (классовые конструкторы — см. §10.2.3) часто для гибкости обеспечивают несколько возможностей. Рассмотрим функцию, предназначенную для печати целого. Решение о предоставлении пользователю возможности выбрать основание счисления для выводимых чисел кажется вполне разумным, но в большинстве случаев числа печатаются в десятичном виде. Например, программа

```
void print (int value, int base =10) ;

void f()
{
    print (31) ;
    print (31, 10) ;
    print (31, 16) ;
    print (31, 2) ;
}
```

выводит следующую последовательность:

```
31 31 1f 1111
```

Эффекта от применения аргумента по умолчанию можно добиться и перегрузкой:

```
void print (int value, int base) ;
inline void print (int value) { print (value, 10) ; }
```

Однако перегрузка менее четко отражает тот факт, что нужна-то единственная функция печати плюс укороченный вариант ее вызова для наиболее типичного случая.

Тип умолчательного аргумента проверяется в месте объявления функции и вычисляется в месте ее вызова. Умолчательными значениями могут снабжаться только аргументы, стоящие в самом конце списка аргументов. Например:

```
int f(int, int=0, char*=0) ;    // ok
int g(int=0, int=0, char*) ;    // error
int h(int=0, int, char*=0) ;    // error
```

Обратите внимание на то, что пробел между * и = обязателен (*= означает операцию присваивания; §6.2):

```
int nasty (char*=0) ;           // синтаксическая ошибка
```

Аргумент по умолчанию нельзя ни повторять, ни изменять в той же самой области видимости. Например:

```
void f (int x=7) ;
void f (int=7) ;           // error: нельзя повторять аргумент по умолчанию
void f (int=8) ;           // error: другое значение умолчательного аргумента

void g ()
{
    void f (int x=9) ;       // ok: это объявление скрывает внешние объявления
    // ...
}
```

Объявление имени во вложенной области видимости, скрывающее объявление того же имени в объемлющей области видимости, чревато ошибками.

7.6. Неуказанное число аргументов

Для некоторых функций бывает невозможно заранее указать количество и типы всех аргументов вызова. Объявления таких функций содержат список аргументов, завершающийся *многоточием* (*ellipsis*, . . .), означающим, что «могут быть еще аргументы». Например:

```
int printf(const char* . . .) ;
```

Это объявление означает, что при вызове стандартной библиотечной функции **printf()** (§21.8) должен быть указан хотя бы один фактический аргумент типа **char***, но также могут быть (а могут и не быть) и иные аргументы. Например:

```
printf("Hello, world!\n") ;
printf("My name is %s %s\n", first_name, second_name) ;
printf("%d + %d=%d\n", 2, 3, 5) ;
```

В процессе интерпретации списка фактических аргументов вызова такие функции должны опираться на информацию, недоступную компилятору. Для функции **printf()** первым аргументом служит так называемая *управляющая строка* (*format string*), содержащая специальные последовательности символов, которые и позволяют функции **printf()** корректно обрабатывать последующие аргументы; **%s** означает «ожидается аргумент типа **char***», а **%d** означает, что «ожидается аргумент типа **int**». Компилятор, в общем случае, этого знать не может и не может гарантировать, что ожидаемые дополнительные аргументы будут на месте, или что их тип будет корректным. Например, программа

```
#include <stdio.h>

int main ()
{
    printf("My name is %s %s\n", 2) ;
}
```

скомпилируется и (в лучшем случае) выдаст что-нибудь странное (попробуйте на практике!).

Ясно, что если аргумент не был объявлен, то компилятор не имеет необходимой информации для проверки типов фактических аргументов и их стандартных преобразований. В таких случаях *char* или *short* передаются как *int*, а *float* передается как *double*. Не факт, что программист ожидает именно этого.

Хорошо спроектированная программа нуждается лишь в минимальном количестве функций, типы аргументов которых определены не полностью. Чаще всего, вместо функций с неопределенными аргументами можно использовать перегрузку функций и аргументы по умолчанию, что гарантирует надежную проверку типов. Функциями с многоточием следует пользоваться лишь тогда, когда и количество аргументов не определено, и их типы неизвестны. Типичнейшим примером таких функций служат функции библиотеки языка С, разработанные до того, как появились их альтернативы на языке С++:

```
int sprintf(FILE*, const char* ...); // из <stdio>
int execl(const char* ...); // из UNIX заголовоч. файла
```

Набор стандартных макросов для доступа к неспецифицированным аргументам этих функций определен в файле <stdarg>. Напишем функцию *error*() для вывода сообщений об ошибках. У этой функции неопределенное число аргументов, из которых первый имеет тип *int* и означает степень серьезности ошибки, а остальные аргументы — строки. Основная идея состоит в том, чтобы составлять сообщение об ошибке из отдельных слов, передаваемых с помощью строковых аргументов. Список строковых аргументов должен оканчиваться нулевым указателем на *char*:

```
extern void error(int ...);
extern char* itoa(int, char[]); // см. §6.6[17]

const char* Null_cp=0;

int main(int argc, char* argv[])
{
    switch(argc)
    {
        case 1:
            error(0, argv[0], Null_cp);
            break;
        case 2:
            error(0, argv[0], argv[1], Null_cp);
            break;
        default:
            char buffer[8];
            error(1, argv[0], "with", itoa(argc-1, buffer), "arguments", Null_cp);
    }
    // ...
}
```

Функция *itoa*() возвращает строковое представление для своего целого аргумента.

Отметим, что использование целого значения 0 для терминирования списка строк не переносимо, ибо на разных системах целочисленный нуль и нулевой ука-

затель могут иметь разные представления. Это иллюстрирует те дополнительные тонкости и сложности, с которыми приходится иметь дело программисту, как только из-за применения многоточия пропадает проверка типов компилятором.

Определим функцию вывода сообщений об ошибках следующим образом:

```
void error (int severity . . . ) // За "severity" следует список элементов
                                // типа char* с терминальным нулем
{
    va_list ap;
    va_start (ap, severity) ;

    for ( ; ; )
    {
        char* p = va_arg (ap, char*) ;
        if (p == 0) break;
        cerr << p << ' ' ;
    }

    va_end (ap) ;

    cerr << '\n' ;
    if (severity) exit (severity) ;
}
```

Сначала создается переменная типа *va_list* и инициализируется вызовом *va_start()*. Макрос *va_start* в качестве аргументов берет имя указанной переменной и имя последнего формального параметра нашей функции *error()*. Макрос *va_arg()* последовательно извлекает неименованные фактические аргументы. При каждом вызове программист должен указывать ожидаемый тип; макрос *va_arg()* полагает, что тип переданного аргумента в точности соответствует указанному программистом, но не может это проверить. Перед выходом из функции, использовавшей *va_start()*, нужно вызвать *va_end()*, так как *va_start()* может модифицировать стек таким образом, что нормальный выход из функции становится невозможным. Вызов *va_end()* устраняет эти модификации.

7.7. Указатели на функции

С функциями можно выполнить лишь два вида работ: *вызвать* или *вычислить их адрес*. Указатель, которому присвоен адрес функции, можно далее использовать для вызова этой функции. Например:

```
void error (string s) { /* ... */ }
void (*efct) (string) ; // указатель на функцию
void f()
{
    efct=&error; // efct указывает на функцию error
    efct ("error") ; // вызов error через efct
}
```

Компилятор легко обнаруживает, что *efct* является указателем на функцию, и вызывает функцию по адресу, содержащемуся в этом указателе. При этом *разыменование*

указателя на функцию (то есть применение операции `*`) не обязательно. Также не обязательно использовать явным образом операцию `&` для получения адреса функции:

```
void (*f1) (string) = &error;    // ok
void (*f2) (string) = error;      // ok; тот же смысл, что и &error

void g ()
{
    f1 ("Vasa");                  // ok
    (*f1) ("Mary Rose");          // тоже ok
}
```

Для указателей на функции надо объявлять типы аргументов так же, как и для самой функции. Присваивание указателей на функции друг другу допустимо лишь при полном совпадении типов функций. Например:

```
void (*pf) (string);              // указатель на void(string)
void f1 (string);                  // void(string)
int f2 (string);                   // int(string)
void f3 (int*);                    // void(int*)

void f()
{
    Pf = &ff1;                     // ok
    pf = &ff2;                       // error: не тот возвращаемый тип
    pf = &ff3;                       // error: не тот тип аргумента

    Pf ("Hera");                    // ok
    pf (1);                          // error: не тот тип аргумента

    int i=p ("Zeus");               // error: void присваивается переменной типа int
}
```

Правила передачи аргументов при вызове функций через указатели те же самые, что и при непосредственном вызове функций.

Часто бывает удобным один раз определить синонимичное имя для типов указателей на функции, чтобы избежать многократного использования достаточно неочевидного синтаксиса их объявления. Вот пример из заголовочного файла системы UNIX:

```
typedef void (*SIG_TYP) (int);      // из <signal.h>
typedef void (*SIG_ARG_TYP) (int);
SIG_TYP signal (int, SIG_ARG_TYP);
```

Часто бывают полезными массивы указателей на функции. Например, система меню в моем графическом редакторе, реализована через массивы указателей на функции, каждая из которых выполняет соответствующее действие. В целом это решение нельзя здесь рассмотреть в деталях, но вот его основная идея:

```
typedef void (*PF) ();

// команды редактирования:
PF edit_ops [] = { &cut, &paste, &copy, &search };

// файловые операции:
PF file_ops [] = { &open, &append, &close, &write };
```


Теперь можно определять и инициализировать указатели, ответственные за выполнение действий, иницилируемых при помощи меню и кнопок мыши:

```
PF* button2 = edit_ops;
PF* button3 = file_ops;
```

В полной реализации с каждым пунктом меню нужно связывать больше информации. Например, нужно где-то хранить строку с текстом данного пункта меню. Во время работы роль кнопок мыши может изменяться в зависимости от контекста. Такие изменения реализуются (частично) изменением значений указателей, относящихся к кнопкам. При выборе конкретной кнопкой (например, кнопкой 2) некоторого пункта меню (например, пункта 3) выполняется соответствующая операция:

```
button2[2] (); // вызов 3-ей функции из button2
```

Чтобы по достоинству оценить всю мощь указателей на функции, нужно попробовать написать подобный код без их помощи, и без помощи их еще более полезных родственников — виртуальных функций (§12.2.6). Меню можно модифицировать во время выполнения программы, просто добавляя новые функции (обработчики) в таблицу операций данного меню. Также легко создавать новые меню динамически (во время выполнения программы).

Указатели на функции можно применить для реализации простой формы *полиморфных процедур*, то есть процедур, применимых к объектам разных типов:

```
typedef int (*CFT) (const void*, const void*);

void ssort(void* base, size_t n, size_t sz, CFT cmp)
/*
    Сортировка n элементов вектора base в возрастающем порядке с использованием
    функции сравнения, указуемой с помощью стр.
    Элементы имеют размер sz.
    Shell sort (Knuth, Vol3, pg84)
*/
{
    for (int gap=n/2; 0<gap; gap/=2)
        for (int i=gap; i<n; i++)
            for (int j=i-gap; 0<=j; j-=gap)
            {
                char* b=static_cast<char*>(base); // обязательное приведение типа
                char* pj=b+j*sz; // &base[j]
                char* pig=b+(j+gap)*sz // &base[j+gap]

                if (cmp (pig, pj) < 0) // обменять base[j] и base[j+gap];
                {
                    for (int k=0; k<sz; k++)
                    {
                        char temp =pj[k];
                        pj[k] = pig[k];
                        pig[k] = temp;
                    }
                }
            }
    }
else
```

```

        break;
    }
}

```

Процедура `ssort()` не знает типы объектов, которые она сортирует, а только число элементов (размер массива), размер каждого элемента и функцию, используемую для сравнения элементов. Тип процедуры `ssort()` намеренно выбран таким же, как у стандартной сортирующей процедуры `qsort()` из библиотеки языка C. В реальных программах применяются `qsort()`, алгоритм `sort()` стандартной библиотеки C++ (§18.7.1), или специализированные процедуры сортировки. Рассмотренный стиль кодирования типичен для языка C, но его нельзя назвать наиболее подходящим способом формулирования алгоритмов на языке C++ (§13.3, §13.5.2).

Разработанная нами процедура `ssort()` применима для сортировки табличной информации:

```

struct User
{
    char* name;
    char* id;
    int dept;
};

User heads[] = { "Ritchie D. M",    "dmr",    11271,
                  "Sethi R. ",      "ravi",    11272,
                  "Szymanski T. G.", "tgs",     11273,
                  "Schryer N. L.",   "nls",     11274,
                  "Schryer N. L.",   "nls",     11275,
                  "Kernighan B. W.", "bwk",     11276 };

void print_id (User* v, int n)
{
    for (int i=0; i<n; i++)
        cout<< v[i].name<< '\t'<< v[i].id<< '\t' <<v[i].dept<< '\n';
}

```

Нужно еще определить подходящую функцию сравнения. Такая функция должна возвращать отрицательное значение, когда первый аргумент меньше второго, нуль в случае равенства аргументов, и положительное число в остальных случаях:

```

int cmp1 (const void* p, const void* q)           // сравнение имен
{
    return strcmp ( static_cast<const User*>(p)->name,
                   static_cast<const User*>(q)->name );
}

int cmp2 (const void* p, const void* q)           // сравнение номеров отделов
{
    return static_cast<const User*>(p)->dept -
           static_cast<const User*>(q)->dept;
}

```

Саму сортировку и вывод ее результатов выполним в функции `main()`:

```

int main ()
{
    cout<<"Heads in alphabetical order:\n";
}

```

```

    ssort (heads, 6, sizeof (User) , cmp1) ;
    print_id (heads, 6) ;
    cout<< '\n' ;

    cout<< "Heads in order of department number: \n" ;
    ssort (heads, 6, sizeof (User) , cmp2) ;
    print_id (heads, 6) ;
}

```

Указателям на функции можно присваивать адреса перегруженных функций. В таких случаях тип указателя используется для выбора нужного варианта перегруженной функции. Например:

```

void f (int) ;
int f (char) ;

void (*pf1) (int) = &f;           // void f(int)
int (*pf2) (char) = &f;           // int f(char)
void (*pf3) (char) = &f;           // error: нет функции void f(char)

```

Функции вызываются через указатели на функции исключительно с правильными типами аргументов и возвращаемых значений. Когда производится присваивание указателям на функции или выполняется их инициализация, *никаких неявных преобразований типов аргументов или типов возвращаемых значений не производится*. Это означает, что функция

```

int cmp3 (const mytype* , const mytype* ) ;

```

не подходит в качестве аргумента для *ssort*() . Причина в том, что в противном случае была бы нарушена гарантия вызова *cmp3*() с аргументами типа *const mytype** (см. также §9.2.5).

7.8. Макросы

Макросы очень полезны в языке C, но в языке C++ они используются гораздо реже. Самое первое правило для макросов: не используйте их без крайней необходимости. Почти что каждый макрос свидетельствует о наличии слабых мест в языке, программе или программисте. Так как из-за макросов текст программы изменяется до того, как его увидит компилятор, то создаются лишние проблемы для многих инструментов программирования. Если вы применяете макросы, приготовьтесь получать меньшую пользу от отладчиков, генераторов перекрестных ссылок и профилировщиков. Если все же вам нужно применять макросы, внимательно прочитайте руководство по вашей реализации препроцессора C++ и не прибегайте к уж слишком хитрым конструкциям. Также следуйте общепринятому соглашению об именовании макросов с помощью исключительно заглавных букв. Синтаксис макросов описан в §A.11.

Простейший макрос определяется следующим образом:

```

#define NAME rest of line

```

Всюду, где встречается лексема *NAME*, она заменяется на *rest of line*. Например:

```

named = NAME

```

превратится в

```
named = rest of line
```

Можно определять макросы с аргументами. Например:

```
#define MAC (x, y) argument1: x argument2: y
```

В местах, где применяется макрос **MAC**, должны также присутствовать и два строковых аргумента. Они заменят *x* и *y* при макроподстановке. Например,

```
expanded = MAC (foo bar, yuk yuk)
```

превратится в

```
expanded = argument1: foo bar argument2: yuk yuk
```

Имена макросов *перегружать* нельзя. Также нельзя использовать в них *рекурсивные вызовы* (препроцессор с ними не справится):

```
#define PRINT (a, b) cout<< (a) << (b)
```

```
#define PRINT (a, b, c) cout<< (a) << (b) << (c) /*беда?: нет перегрузки, переопределение*/
```

```
#define FAC (n) (n>1) ? n*FAC (n-1) : 1 /*беда: рекурсивное макро*/
```

Макросы связаны лишь с текстовой обработкой и они мало что знают о синтаксисе языка C++, и вообще ничего не знают о его типах и областях видимости. Компилятор видит программный текст после макроподстановки, так что ошибки в макросах обнаруживаются лишь как последствия, а вовсе не в определениях макросов. Все это приводит к весьма туманным сообщениям об ошибках.

Вот макросы, внушающие доверие:

```
#define CASE break; case
```

```
#define FOREVER for ( ; ; )
```

А вот примеры совершенно ненужных макросов:

```
#define PI 3.141593
```

```
#define BEGIN {
```

```
#define END }
```

Следующие макросы опасны:

```
#define SQUARE (a) a*a
```

```
#define INCR_xx (xx) ++
```

Чтобы убедиться в их опасности, попробуйте применить их:

```
int xx = 0; // глобальный счетчик

void f()
{
    int xx = 0; // локальная переменная
    int y = SQUARE (xx+2) ; // y=xx+2*xx+2; то есть y=xx+(2*xx)+2
    INCR_xx; // инкрементирует локальную xx
}
```

Если уж вам действительно нужны макросы, применяйте операцию разрешения области видимости :: при ссылке на глобальные имена (§4.9.4) и везде, где только можно, заключайте в круглые скобки имена их аргументов. Например:

```
#define MIN(a,b) ((a)<(b))?(a):(b)
```

Если вы написали довольно сложный макрос, так что требуются комментарии к нему, применяйте комментарии вида `/* */`, ибо часто в состав инструментов программирования на C++ входит препроцессор языка C, а он ничего не знает о комментариях вида `//`. Например:

```
#define M2(a) something(a) /* полезный комментарий */
```

При помощи макросов вы можете создать свой собственный язык. Даже если вы сами предпочтете такой «улучшенный» язык простому C++, другим программистам он будет непонятен. Более того, препроцессор C — это очень простой макропроцессор. Поэтому, когда вы захотите создать что-нибудь нетривиальное, то окажется, что либо это невозможно, либо неоправданно трудоемко. Механизмы *const*, *inline*, *template*, *enum* и *namespace* являются альтернативой традиционному использованию препроцессорных конструкций. Например:

```
const int answer = 42;
template<class T> inline T min(T a, T b) {return (a<b)?a:b;}
```

При помощи макросов можно создавать новые имена — новую строку можно составить из двух строк при помощи операции препроцессора `##`. Например:

```
#define NAME2(a,b) a##b
int NAME2(hack,cah)();
```

превратится в

```
int hackcah();
```

что и достанется компилятору для чтения.

Директива препроцессора

```
#undef X
```

гарантирует, что более не существует макроса с именем *X* независимо от того, существовал ли он до этой директивы, или нет. Такой прием позволяет на всякий случай защититься от нежелательных макросов, так как в точности узнать их действие на фрагмент кода бывает нелегко.

7.8.1. Условная компиляция

Одного случая применения макросов избежать практически невозможно. Директива компилятора *#ifdef identifier* заставляет опустить последующую часть кода, пока не встретится директива *#endif*. Например:

```
int f(int a
#ifdef arg_two
, int b
#endif
);
```

превратится для компилятора в

```
int f(int a
```

если не определен макрос *arg_two*. Данный код способен запутать автоматические инструменты разработки, так как они рассчитывают на разумное поведение программиста.

Но в большинстве случаев характер применения *#ifdef* менее эксцентричный, чем в рассмотренном примере, и при наличии определенных ограничений эта директива приносит мало вреда. См. также §9.3.3.

Следует аккуратно выбирать имена макросов, используемых в директиве *#ifdef*, чтобы они не вступали в противоречие с обычными идентификаторами. Например:

```
struct Call_info
{
    Node* arg_one;
    Node* arg_two;
    // ...
};
```

Этот невинно выглядящий код вызовет недоразумения, как только кто-нибудь определит следующий макрос:

```
#define arg_two x
```

К сожалению, многие стандартные заголовочные файлы содержат множество ненужных и опасных макросов.

7.9. Советы

1. Относитесь с подозрением к **неконстантным** аргументам, передаваемым по ссылке; если вы хотите, чтобы функция модифицировала аргументы, используйте указатели и возвращаемое значение; §5.5.
2. Применяйте **константные** ссылки, если вы хотите минимизировать копирование аргументов; §5.5.
3. Используйте **const** активно и последовательно; §7.2.
4. Избегайте макросов; §7.8.
5. Избегайте функций с неуказанным числом аргументов; §7.6.
6. Не возвращайте указатели или ссылки на локальные переменные; §7.3.
7. Применяйте перегрузку функций в случаях, когда концептуально одинаковая работа выполняется над данными разных типов; §7.4.
8. В случае перегрузки функций с целыми аргументами реализуйте полный набор вариантов для устранения неоднозначностей; §7.4.3.
9. Обдумывая вопрос о применении указателей на функций, рассмотрите возможность их замены на виртуальные функции (§2.5.5) или шаблоны (§2.7.2) в качестве лучших альтернатив; §7.7.
10. Если вам нужны макросы, применяйте для них безобразно выглядящие имена из заглавных букв; §7.8.

7.10. Упражнения

- (*1) Напишите следующие объявления: функция с аргументами «указатель на символ» и «ссылка на целое», не имеющая возврата; указатель на такую функцию; функция с таким указателем в качестве аргумента; функция, возвращающая такой указатель. Напишите определение функции, принимающей такой указатель в качестве аргумента и возвращающей его же. Подсказка: воспользуйтесь *typedef*.

- (*1) Что означает следующее? Для чего это может потребоваться?

```
typedef int (&rifi) (int, int) ;
```

- (*1.5) Напишите программу типа «Hello, world!», которая берет имя (*name*) из командной строки и выводит «Hello, *name*!». Модифицируйте программу так, чтобы она могла брать произвольное количество имен в качестве аргументов и приветствовать всех по этим именам.
- (*1.5) Напишите программу, которая читает произвольное количество файлов, чьи имена задаются в командной строке, и выводит их последовательно в *cout*. Поскольку эта программа соединяет содержимое своих аргументов для формирования вывода, можете назвать ее *cat*.
- (*2) Преобразуйте небольшую программу на языке C в соответствующую программу на C++. Переделайте заголовочные файлы таким образом, чтобы в них объявлялись все вызываемые в программе функции, и чтобы были указаны типы всех их аргументов. Где можно, замените все директивы *#define* на *enum*, *const* или *inline*. Удалите все объявления *extern* из .c-файлов и в случае необходимости преобразуйте определения функций в синтаксисе языка C в соответствующие определения в синтаксисе языка C++. Замените вызовы функций *malloc()* и *free()* на операции *new* и *delete*. Удалите явные преобразования типов, в которых нет необходимости.
- (*2) Реализуйте функцию *ssort()* (§7.7) с помощью более эффективного алгоритма сортировки. Подсказка: *qsort()*.
- (*2.5) Имеется структура *Tnode*:

```
struct Tnode
{
    string word;
    int count;
    Tnode* left;
    Tnode* right ;
};
```

Напишите функцию для внедрения новых слов в дерево с узлами типа *Tnode*. Напишите функцию для вывода такого дерева. Напишите функцию для вывода слов из такого дерева в алфавитном порядке. Модифицируйте структуру *Tnode* так, чтобы она содержала указатель на произвольно длинное слово, хранящееся в массиве символов, память под который выделяется операцией *new*. Модифицируйте все функции, чтобы они работали с новым вариантом структуры *Tnode*.

8. (*2.5) Напишите функцию, инвертирующую (транспонирующую) двумерный массив. Подсказка: §С.7.
9. (*2) Напишите программу шифрования, которая читает из *cin* и пишет закодированные символы в *cout*. Можете применить следующую простую схему шифрования: символ *c* кодируется выражением $c^{\text{key}[i]}$, где *key* — строка, передаваемая в качестве аргумента командной строки. Программа циклически использует символы из строки *key* до исчерпания ввода. Повторное шифрование по этой же формуле восстанавливает исходный текст. Если нет входного текста или *key* есть нулевая строка, шифрование не выполняется.
10. (*3.5) Напишите программу, помогающую без знания ключа дешифровать текст, закодированный программой из предыдущего упражнения. Подсказка: David Kahn: *The Codebreakers*, Macmillan, 1967, New York, pp. 207-213.
11. (*3) Напишите функцию *error()*, принимающую строку в стиле функции *printf()*, содержащую *%s*, *%c* и *%d*, и произвольное число других аргументов. Не используйте функцию *printf()*. См. §21.8. Используйте *<csdarg>*.
12. (*1) Как бы вы выбирали имена для типов указателей на функции, которые определяются с помощью *typedef*?
13. (*2) Просмотрите несколько программ, обращая внимание на множество стилей именования. Как используются заглавные буквы? Как используется символ подчеркивания? Когда используются короткие имена вроде *i* или *x*?
14. (*1) Что плохого в следующих макросах?

```
#define PI = 3.141593;
#define MAX(a, b) a>b?a:b
#define fac(a) (a)*fac((a)-1)
```
15. (*3) Напишите простой макропроцессор, позволяющий определять и расширять макросы (как это делает препроцессор языка C). Читайте из *cin* и выводите в *cout*. Вначале ограничьтесь макросами без аргументов. Подсказка: программа-калькулятор (§6.1) содержит таблицу символов и лексический анализатор, которыми вы можете воспользоваться.
16. (*2) Реализуйте функцию *print()* из §7.5.
17. (*2) Добавьте функции, такие как *sqrt()*, *log()* и *sin()*, к программе-калькулятору из §6.1. Подсказка: заранее определите эти имена и вызывайте функции с помощью массива указателей на функции. Не забывайте проверять фактические аргументы вызова.
18. (*1) Напишите функцию вычисления факториала, не использующую рекурсию. См. §11.14[6].
19. (*2) Напишите функции, которые добавляют день, месяц, год к заданной дате (структура *Date* из §5.9[13]). Напишите функцию, вычисляющую день недели для заданного значения *Date*. Напишите функцию, вычисляющую значение *Date* для первого понедельника после заданного *Date*.

Пространства имен и исключения

*Год 787! От Рождества Христова?
— Монти Пайтон*

*Нет столь общих правил, что не допускали бы исключений.
— Роберт Бартон*

Модульность — интерфейсы и исключения — пространства имен — **using** — **using namespace** — разрешение конфликтов имен — поиск имен — композиция пространств имен — псевдонимы пространств имен — пространства имен и код на С — исключения — **throw** и **catch** — исключения и структура программы — советы — упражнения.

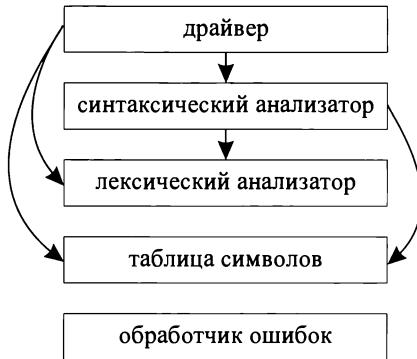
8.1. Разбиение на модули и интерфейсы.

Любая реальная программа состоит из нескольких отдельных частей. Например, даже столь простая программа, как «Hello, world!», включает по крайней мере две части: пользовательский код, требующий выполнить вывод строки **Hello, world!**, и система ввода-вывода, которая и осуществляет этот вывод.

Снова рассмотрим программу-калькулятор из §6.1. Ее можно рассматривать как совокупность пяти частей:

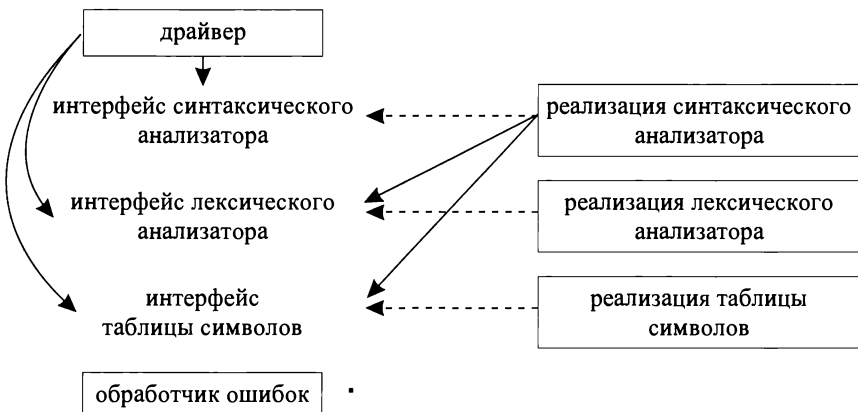
1. Парсера, выполняющего синтаксический анализ
2. Лексического анализатора, составляющего лексемы из символов
3. Таблицы символов, содержащей пары (строка, значение)
4. Управляющей части, содержащей функцию **main** ()
5. Обработчика ошибок

Это можно отобразить графически:



Здесь стрелками обозначено отношение «использует». Чтобы упростить рисунок, я не стал на нем отображать тот факт, что все части используют обработчик ошибок. По сути, калькулятор состоит лишь из трех частей, а управляющая часть («драйвер») и обработчик ошибок введены в него для полноты реализации.

Когда один модуль использует другой модуль, ему нет никакой необходимости знать все детали устройства последнего. В идеале, большая часть внутреннего устройства модуля неизвестна его клиентам. Мы проводим четкое различие между устройством модуля и его интерфейсом. Например, парсер обращается напрямую лишь к интерфейсу лексического анализатора. А лексический анализатор реализует все анонсируемые им посредством интерфейса сервисы. Это можно изобразить графически:



Пунктирные линии означают «реализует». Я считаю, что эта схема отражает реальную структуру программы, а задача программиста сводится к тому, чтобы точно отразить ее в коде. В этом случае код будет простым, эффективным, понятным, удобным для сопровождения и т.д., потому что он напрямую соответствует фундаментальным основам нашего проекта.

Последующие разделы данной главы показывают, как сделать простой и понятной логическую структуру программы калькулятора, а в §9.3 рассматривается вопрос, как физически организовать исходный текст программы, чтобы он наилучшим образом соответствовал этой структуре. Калькулятор — это крошечная программа, для которой в реальных условиях я не стал бы столь интенсивно использовать пространства имен и отдельную компиляцию (§2.4.1, §9.1). Все это используется для демонстрации приемов, позволяющих писать большие программы так, чтобы потом не утонуть в них. В реальных программах каждый модуль, представленный отдельным пространством имен, часто содержит сотни функций, классов, шаблонов и т.п.

Ради демонстрации различных практических приемов и языковых средств я намеренно провожу разбиение программы калькулятора на модули в несколько этапов. В «реальной жизни» программы вряд ли разрабатываются в такой последовательности. Опытный программист может сразу начать с почти готового проектного решения. Тем не менее, в процессе многолетней эксплуатации программы и ее модернизации не исключена и кардинальная переделка ее структуры.

Обработка ошибок проходит сквозной линией через всю структуру программы. Разбивая программу на модули, или (наоборот) собирая ее из модулей, мы должны заботиться о минимизации зависимости между модулями, проистекающей из обработки ошибок. В языке C++ обнаружение ошибок (сообщение о них) и их обработку можно четко разделить с помощью механизма исключений. Поэтому рассмотрение пространств имен в качестве модулей (§8.2) дополняется рассмотрением исключений, помогающим еще более улучшить модульность программы (§8.3).

В данной и следующей главах рассматривается лишь небольшая часть из всего множества вопросов, связанных с модульностью программ. Некоторые аспекты модульности можно было бы дополнительно рассмотреть на примере параллельно работающих и взаимодействующих между собой процессов. Модульность можно рассматривать и в смысле отдельных адресных пространств, между которыми осуществляется передача данных. Все эти аспекты модульности довольно независимы и ортогональны друг другу. Самое интересное, что разбиение на модули почти всегда выполняется довольно просто. Намного сложнее — обеспечить безопасное, удобное и эффективное взаимодействие модулей между собой.

8.2. Пространства имен

Пространства имен являются механизмом логического группирования программных объектов. Если некоторые объявления согласно определенному критерию логически близки друг другу, то для отражения этого факта их можно поместить в одно и то же пространство имен. Например, все объявления из программы калькулятора (§6.1.1), относящиеся к синтаксическому анализатору (парсеру), можно поместить в одно пространство имен *Parser*:

```
namespace Parser
{
    double expr (bool) ;
    double prim (bool get) { /* ... */ }
    double term (bool get) { /* ... */ }
    double expr (bool get) { /* ... */ }
}
```

Функция *expr*() должна быть сначала объявлена, а определена потом из-за необходимости как-то разорвать порочный круг зависимостей, рассмотренный §6.1.1.

Часть программы калькулятора, отвечающая за обработку ввода, также может быть помещена в свое собственное пространство имен:

```
namespace Lexer
{
    enum Token_value
    {
        NAME,          NUMBER,          END,
        PLUS='+',      MINUS='-',      MUL='*',      DIV='/',
        PRINT=';',      ASSIGN='=',      LP='(',      RP=')'
    };

    Token_value curr_tok;
    double number_value;
    string string_value;
    Token_value get_token() { /* ... */ }
}
```

Применение пространств имен наглядно показывает, что именно лексический и синтаксический анализаторы предлагают своим пользователям. Однако если бы я включил в них полные определения функций, то смысл пространств имен уже не проступал бы столь отчетливо. Когда в пространства имен реального размера включают тела функций, приходится просматривать множество страниц кода (или пролистывать множество экранов монитора), чтобы узнать, какие же собственно предоставляются сервисы, то есть каков интерфейс модуля.

Альтернативой раздельного определения интерфейсов является применение программных средств, автоматически извлекающих интерфейсы из модуля реализации. Я не считаю это хорошим решением, потому что: определение интерфейсов является существенной частью процесса проектирования (§23.4.3.4), модуль может предоставлять разные интерфейсы разным пользователям, и к тому же разработка интерфейсов часто ведется задолго до конкретизации деталей реализации.

Вот новый вариант пространства имен *Parser*, в котором интерфейс явным образом отделен от реализации:

```
namespace Parser
{
    double prim (bool);
    double term (bool);
    double expr (bool);
}

double Parser::prim (bool get) { /* ... */ }
double Parser::term (bool get) { /* ... */ }
double Parser::expr (bool get) { /* ... */ }
```

Обратите внимание на то, что теперь каждая функция имеет ровно одно объявление, и одно определение. Пользователям нужно ознакомиться с интерфейсом, содержащим лишь объявления. А реализация может быть помещена в какое-нибудь другое место, куда пользователю заглядывать нет необходимости.

Как продемонстрировано в последнем примере, элемент (*member-name*) пространства имен (*namespace-name*) может быть в нем лишь объявлен, а определен позднее в нотации *namespace-name* : *member-name*.

Члены (элементы) пространства имен объявляются следующим образом:

```
namespace namespace-name
{
    // объявления и определения
}
```

Нельзя объявлять новый элемент пространства имен вне этого определения (не поможет и явная квалификация имени):

```
void Parser : : logical (bool) ;      // error: нет никакого logical() в Parser
```

Идея состоит в том, чтобы все элементы пространства имен легко обнаруживались в одном определении, и чтобы можно было легко отлавливать ошибки, связанные, например, с описками или несоответствием типов. Например:

```
double Parser : : trem (bool) ;      // error: нет никакого trem() в Parser
double Parser : : prim (int) ;      // error: Parser : : prim() принимает аргумент bool
```

Пространство имен формирует свою собственную область видимости. Таким образом, пространство имен является одновременно и фундаментальной, и достаточно простой концепцией. Чем больше размер программы, тем большую пользу приносят пространства имен, помогая четко разделять программу на логические части. Обычные локальные и глобальные области видимости, а также классы, формируют свои пространства имен (§C.10.3).

В идеале, каждая программная сущность должна принадлежать некоторой четко ограниченной логической единице («модулю»). Поэтому любое объявление в нетривиальной программе должно в идеале помещаться в пространство имен, собственное имя которого должно отражать его логическую роль в программе. Исключение составляет функция *main* (), которая должна оставаться глобальной, чтобы исполнительная система всегда могла отыскать стартовую функцию (§8.3.3).

8.2.1. Квалифицированные имена

Пространство имен является отдельной областью видимости. Обычные правила для областей видимости распространяются и на пространства имен, так что если имя объявлено ранее в том же самом пространстве имен (или в охватывающей области видимости), то его можно использовать без проблем. Имена же из других пространств имен требуют дополнительной квалификации. Например:

```
double Parser : : term (bool get)    // нужна квалификация Parser : :
{
    double left = prim (get) ;        // нет нужды в квалификации
    for ( ; ; )
        switch (Lexer : : curr_tok)  // нужна квалификация Lexer : :
        {
            case Lexer : : MUL :      // нужна квалификация Lexer :
```

```

    left*=prim (true) ;      // нет нужды в квалификации
    // ...
}
// ...
}

```

Квалификатор **Parser** указывает, что функция **term()** объявлена именно в пространстве имен **Parser**, а не где-то еще (например, в глобальной области видимости). Так как **term()** является членом **Parser**, нет необходимости квалифицировать иной член этого же пространства имен — **prim()**. А вот если убрать квалификатор **Lexer**, то имя **curr_tok** будет считаться необъявленным, поскольку имена из пространства имен **Lexer** не входят в область видимости пространства имен **Parser**.

8.2.2. Объявления using

Когда имя часто используется вне пределов своего пространства имен, может быть утомительно то и дело дополнительно квалифицировать его. Рассмотрим следующий пример:

```

double Parser::prim (bool get)    // обработка первичных выражений
{
    if (get) Lexer::get_token ();
    switch (Lexer::curr_tok)
    {
        case Lexer::NUMBER:      // константа с плавающей запятой
            Lexer::get_token ();
            return Lexer::number_value;
        case Lexer::NAME:
        {
            double& v=table [Lexer::string_value] ;
            if (Lexer::get_token () ==Lexer::ASSIGN) v=expr (true) ;
            return v;
        }
        case Lexer::MINUS:        // унарный минус
            return -prim (true) ;
        case Lexer::LP:
        {
            double e = expr (true) ;
            if (Lexer::curr_tok != Lexer::RP) return Error::error (" expected") ;
            Lexer::get_token ();    // пропустить скобку ')'
            return e;
        }
        case Lexer::END:
            return 1;
        default:
            return Error::error ("primary expected") ;
    }
}

```

Назойливое повторение квалификатора **Lexer** утомительно и отвлекает внимание. Такую избыточность можно устранить с помощью объявления *using* (*using-decla-*

ration), которое позволяет однократно указать для текущей области видимости, что имя **get_token** взято из пространства имен **Lexer**. Например:

```
double Parser::prim (bool get)    // обработка первичных выражений
{
    using Lexer::get_token;        // использовать get_token из Lexer
    using Lexer::curr_tok;         // использовать curr_tok из Lexer
    using Error::error;           // использовать error из Error

    if (get) get_token ();
    switch (curr_tok)
    {
        case Lexer::NUMBER:        // константа с плавающей запятой
            get_token ();
            return Lexer::number_value;
        case Lexer::NAME:
        {
            double& v = table [Lexer::string_value];
            if (get_token () == Lexer::ASSIGN) v = expr (true);
            return v;
        }
        case Lexer::MINUS:          // унарный минус
            return -prim (true);
        case Lexer::LP:
        {
            double e=expr (true);
            if (curr_tok != Lexer::RP) return error (" expected");
            get_token ();           // пропустить скобку ')'
            return e;
        }
        case Lexer::END:
            return 1;
        default:
            return error ("primary expected");
    }
}
```

Объявление **using** вводит локальный синоним.

Полезно локальные синонимы делать настолько локальными, насколько это возможно, чтобы избежать конфликта имен. Однако все функции синтаксического анализатора используют одни и те же наборы имен из других модулей. Поэтому мы можем поместить объявления **using** непосредственно внутрь определения пространства имен **Parser**:

```
namespace Parser
{
    double prim (bool);
    double term (bool);
    double expr (bool);
    using Lexer::get_token;        // использовать get_token из Lexer
    using Lexer::curr_tok;         // использовать curr_tok из Lexer
    using Error::error;           // использовать error из Error
}
```

Это позволяет упростить код функций из пространства имен **Parser** практически до их первоначального состояния (§6.1.1).

```
double Parser::term (bool get)    // умножение и деление
{
    double left=prim (get) ;

    for ( ; )
        switch (curr_tok)
        {
            case Lexer::MUL:
                left*= prim (true) ;
                break;
            case Lexer::DIV:
                if (double d = prim (true) )
                {
                    left /= d;
                    break;
                }
                return error ("divide by 0") ;
            default:
                return left;
        }
}
```

Я мог бы с помощью объявлений *using* внести в пространство имен **Parser** также и имена лексем из **Lexer**. Но я оставил их полную запись с явными квалификаторами как напоминание о зависимости между **Parser** и **Lexer**.

8.2.3. Директивы using

А что если бы мы захотели упростить функции из пространства имен **Parser** в *точности* до их первоначальных версий? Это было бы вполне разумной задачей для большой программы, которая конвертируется из предыдущей версии с малой модульностью.

Директивы *using* (*using-directive*) позволяют сделать имена из некоторых пространств имен столь же доступными, как если бы они объявлялись вне этих пространств имен (§8.2.8). Например:

```
namespace Parser
{
    double prim (bool) ;
    double term (bool) ;
    double expr (bool) ;

    using namespace Lexer;    // сделать все имена из Lexer доступными
    using namespace Error;    // сделать все имена из Error доступными
}
```

Это позволяет нам написать функции из пространства имен **Parser** точно так же, как мы это делали ранее (§6.1.1):


```
double Parser : : term (bool get)    // умножение и деление
{
    double left = prim (get) ;
    for ( ; ; )
        switch (curr_tok)
        {
            case MUL :
                left *= prim (true) ;
                break ;
            case DIV :
                if (double d = prim (true) )
                {
                    left /= d ;
                    break ;
                }
                return error ("divide by 0") ;
            default :
                return left ;
        }
}
```

Директивы *using*, примененные глобально, являются удобным инструментом для переделки готового кода (§8.2.9), а в иных случаях их лучше избегать. Эти же директивы, примененные внутри определений пространств имен, являются средством их композиции (§8.2.8). В теле функций (и только там) *директивы using* можно смело применять с целью улучшения внешнего вида кода (§8.3.3.1).

8.2.4. Множественные интерфейсы

Ясно, что данное выше определение пространства имен *Parser* не является интерфейсом, предназначенным для пользователей синтаксического анализатора (*парсера*). Скорее, это набор объявлений, необходимый для удобной разработки функций синтаксического анализатора. А интерфейс для пользователя должен быть существенно более простым:

```
namespace Parser
{
    double expr (bool) ;
}
```

К счастью, можно определить два *пространства имен Parser*, чтобы использовать каждое из них там, где нужно. В совокупности эти пространства имен обеспечивают две вещи:

1. Общую среду для функций парсера
2. Внешний интерфейс для пользователей парсера

Так, управляющая функция *main* () должна видеть лишь

```
namespace Parser                // интерфейс для пользователей
{
    double expr (bool) ;
```

А функции, в совокупности реализующие синтаксический анализатор, должны видеть пространство имен, составляющее интерфейс разработки калькулятора:

```
namespace Parser                                // интерфейс для разработчиков
{
    double prim (bool) ;
    double term (bool) ;
    double expr (bool) ;

    using Lexer : get_token ;                   // использовать get_token из Lexer
    using Lexer : curr_tok ;                    // использовать curr_tok из Lexer
    using Error : error ;                       // использовать error из Error
}
```

или в графическом виде:



Здесь стрелки означают отношение «полагается на указанный интерфейс».

Parser' — это «малый» интерфейс для пользователей. Имя **Parser'** (читается «Парсер штрих») не является идентификатором C++; оно выбрано намеренно с целью подчеркнуть, что в программе для этого интерфейса отдельного уникального имени нет. Это не приводит к путанице, так как программисты привыкли давать разные имена для принципиально разных интерфейсов, и потому что физическая организация программы (§9.3.2) естественным образом обеспечивает изоляцию имен (в файлах).

Интерфейс для разработчиков шире такового для пользователей. Если бы это был интерфейс разработки реальной программы большого размера, то он еще и менялся бы чаще интерфейса для пользователей. Важно, что пользователи модуля (в нашем случае функция **main()**, использующая модуль **Parser**) изолированы от таких изменений.

В принципе, нет необходимости в двух пространствах имен для отражения двух различных интерфейсов, но при желании это можно делать. Вообще, разработка интерфейса является одной из важнейших задач проектирования, в которой можно многое приобрести, и многое потерять. Есть смысл обсудить, чего собственно мы хотим достигнуть, и есть ли альтернативные пути достижения того же самого.

Предложенная нами схема решения является самой простой из всех возможных, а часто и самой лучшей. Ее основным недостатком является совпадение имен двух интерфейсов при отсутствии у компилятора достаточной информации для контроля их согласованности между собой. И даже в этом случае компилятор отслеживает многие ошибки. Более того, заметную долю ошибок, пропущенных компилятором, отлавливает компоновщик.

Предложенное решение используется далее при обсуждении физического разбиения на модули (§9.3), и я его рекомендую, если нет дополнительных логических ограничений (см. также §8.2.7).

8.2.4.1. Альтернативы интерфейсам

Главная цель применения интерфейсов — уменьшение зависимостей между частями программы. Минимально достаточные интерфейсы приводят к программам, которые легко понять, которые хорошо скрывают внутренние данные, которые легко модернизировать, и которые даже компилируются быстрее.

Следует иметь в виду, что в отношении зависимостей и компиляторам, и программистам часто свойственен упрощенный взгляд на проблему: «Если определение видимо в точке *X*, то любой код в точке *X* зависит от содержимого этого определения». В общем случае, однако, дела обстоят не столь плохо, поскольку на самом деле большая часть кода не зависит от большей части определений. К примеру, в нашем случае мы имеем следующее:

```
namespace Parser                // интерфейс для реализации
{
    // ...
    double expr (bool) ;
    // ...
}

int main ( )
{
    // ...
    Parser : : expr (false) ;
    // ...
}
```

Здесь функция **main ()** зависит только от **Parser : : expr ()**, но чтобы выявить этот факт, нужно время, умственные усилия, практическая проверка гипотезы и т.д. Как следствие, в реальных программах больших размеров и компиляторы, и программисты перестраховываются и считают, что если зависимость может быть, то она действительно имеет место. Надо сказать, этот подход разумный.

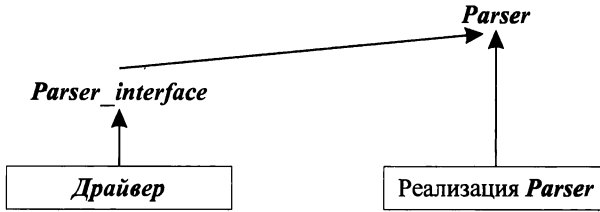
Таким образом, желательно организовать программу так, чтобы уменьшить набор потенциальных зависимостей до набора реальных зависимостей.

Сначала пробуем очевидное: определяем пользовательский интерфейс к парсеру в терминах имеющегося интерфейса разработчика:

```
namespace Parser                // интерфейс для реализации
{
    // ...
    double expr (bool) ;
    // ...
}

namespace Parser_interface { // интерфейс для пользователей
{
    using Parser : : expr ;
}
```

Ясно, что пользователи **Parser_interface** зависят только (причем косвенно) от **Parser : : expr ()**. Однако на первый (поверхностный) взгляд имеет место следующая схема зависимостей:



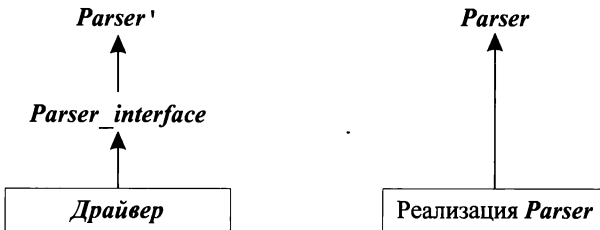
Теперь уже кажется, что пользователь (управляющая программа) зависит от любых изменений в **Parser**, от которых мы его хотели бы изолировать. Даже такая кажущаяся зависимость нежелательна, и мы явным образом ограничиваем **Parser_interface** лишь имеющей отношение к пользователям частью интерфейса **Parser** (ранее мы это называли **Parser'**):

```

namespace Parser           // интерфейс для пользователей
{
    double expr (bool) ;
}

namespace Parser_interface // отдельно поименованный интерфейс для пользователей
{
    using Parser : : expr ;
}
  
```

или графически:



Для проверки согласованности **Parser** и **Parser'** между собой мы снова полагаемся на совокупность всех средств компиляции и компоновки, а не на работу одного лишь компилятора над одной единицей компиляции. Настоящее решение отличается от такового из §8.2.4 лишь наличием дополнительного пространства имен **Parser_interface**. При желании мы могли бы явным образом объявить функцию **expr()** непосредственно в **Parser_interface**:

```

namespace Parser_interface
{
    double expr (bool) ;
}
  
```

Теперь нет необходимости держать **Parser** в области видимости в момент определения **Parser_interface**. Это нужно лишь в момент определения функции **Parser_interface::expr()**:

```
double Parser_interface : : expr (bool get)
{
    return Parser : : expr (get) ;
}
```

Последний вариант можно отобразить графически следующим образом:



Все зависимости теперь сведены к минимуму, все конкретизировано и именовано надлежащим образом. Однако в большинстве реальных практических случаев, с которыми мне приходится сталкиваться, это решение чрезмерно сложно и выходит за пределы необходимого.

8.2.5. Как избежать конфликта имен

Пространства имен предназначены для отражения внутренней структуры кода. Простейшей целью применения пространств имен является разграничение кода, написанного одним программистом, от кода, написанного кем-то еще. Такое разграничение кода имеет огромное практическое значение.

Действительно, на базе одной лишь глобальной области видимости неоправданно сложно составлять программу из отдельных частей. Проблема состоит в том, что предположительно отдельные части могут определять одни и те же имена. При объединении частей в программу происходит конфликт имен. Рассмотрим пример:

```
// my.h:
char f(char) ;
int f(int) ;
class String { /* ... */ };

// your.h:
char f(char) ;
double f(double) ;
class String { /* ... */ };
```

При наличии таких определений трудно составить единую программу, используя одновременно и *my.h*, и *your.h*. Очевидное решение состоит в том, чтобы включить каждый из наборов объявлений в его собственное пространство имен:

```
namespace My
{
    char f(char) ;
    int f(int) ;
    class String { /* ... */ };
```

```
namespace our
{
    char f(char) ;
    double f(double) ;
    class String { /* ... */ };
}
```

Теперь можно использовать объявления из *My* и *Your* либо при помощи явной квалификации (§8.2.1), либо применением объявлений *using* (§8.2.2) или директив *using* (§8.2.3).

8.2.5.1. Неименованные пространства имен

Часто бывает полезно обернуть набор объявлений объемлющим пространством имен просто ради того, чтобы избежать потенциального конфликта имен. В таком случае главная цель состоит в локализации имен, а не в предоставлении интерфейса к пользовательскому коду. Например:

```
#include "header.h"
namespace Mine
{
    int a;
    void f() { /* ... */ }
    int g () { /* ... */ }
}
```

Так как мы не планируем делать имя *Mine* доступным вне локального контекста, выбор имени становится определенной обузой, к тому же отягощенной возможностью конфликта с другими глобальными именами. В этом случае имя пространства имен можно просто опустить:

```
#include "header.h"
namespace
{
    int a;
    void f() { /* ... */ }
    int g () { /* ... */ }
}
```

Доступ к членам *неименованного пространства имен* (*unnamed namespace*) осуществляется в локальном контексте без квалификации, так что предыдущее объявление эквивалентно следующему объявлению

```
namespace $$$
{
    int a;
    void f() { /* ... */ }
    int g () { /* ... */ }
}
using namespace $$$;
```

плюс соответствующая директива *using*, а \$\$\$ — некоторое уникальное для текущей области видимости имя.

Неименованные пространства имен в разных единицах компиляции разные по определению, так что, как и задумано, невозможно обратиться к члену неименованного пространства имен из другой единицы компиляции.

8.2.6. Поиск имен

Чаще всего функция с аргументом типа *T* определяется в том же пространстве имен, что и тип *T*. Поэтому, если функцию не удастся найти в контексте, где она используется, поиск продолжается в пространстве имен ее аргументов. Например:

```
namespace Chrono
{
    class Date { /* ... */ };
    bool operator==(const Date&, const std::string&);
    std::string format(const Date&); // выдать строковое представление
    // ...
}

void f(Chrono::Date d, int i)
{
    std::string s = format(d);           // Chrono::format()
    std::string t = format(i);           // error: нет format() в области видимости
}
```

Это правило экономит усилия программиста, так как не требует ввода квалифицированных имен, и в то же время не засоряет область видимости так, как это делает директива *using* (§8.2.3). Рассмотренное правило особо полезно в случае операндов перегружаемых операций (§11.2.4) и аргументов шаблонов (§C.13.8.4), где явная квалификация может быть особо обременительной.

Важно, однако, чтобы при этом само пространство имен находилось в области видимости, а функция была объявлена до ее использования.

Естественно, что функция может иметь аргументы из нескольких пространств имен. Например:

```
void f(Chrono::Date d, std::string s)
{
    if(d == s)
    {
        // ...
    }
    else if(d == "August 4, 1914")
    {
        // ...
    }
}
```

В таком случае, функция ищется в текущей (по отношению к ее вызову) области видимости и в пространствах имен ее аргументов (включая классы аргументов и их базовые классы). После этого ко всем найденным функциям применяются обычные правила разрешения перегрузки (§7.4). Например, для *d==s* выполняется поиск *operator==* в области видимости, окружающей *f()*, в пространстве имен *std* (где определяется операция *==* для стандартного библиотечного типа *string*) и в простран-

стве имен **Chrono**. При этом находится операция **std::operator==()**, но у нее нет аргумента типа **Date**, так что используется **Chrono::operator==()**, имеющая такой аргумент. См. также §11.2.4.

Когда член класса вызывает именованную функцию, остальные члены этого класса и его базовых классов имеют предпочтение перед функциями, выбираемыми на основании типов аргументов; ситуация же с перегруженными операциями иная (§11.2.1, §11.2.4).

8.2.7. Псевдонимы пространств имен

Если давать пространствам имен короткие имена, то они могут войти в конфликт друг с другом:

```
namespace A      //слишком коротко (может войти в конфликт)
{
    // ...
}

A::String s1 = "Grig";
A::String s2 = "Nielsen";
```

В то же время, длинные имена использовать в реальном коде не очень удобно:

```
namespace American_Telephone_and_Telegraph // слишком длинное
{
    // ...
}

American_Telephone_and_Telegraph::String s3 = "Grieg";
American_Telephone_and_Telegraph::String s4 = "Nielsen";
```

Дилемма разрешается с помощью коротких *псевдонимов (aliases)* для длинных имен:

```
// используем псевдонимы:
namespace ATT = American_Telephone_and_Telegraph;
ATT::String s3 = "Grieg";
ATT::String s4 = "Nielsen";
```

С помощью псевдонимов удобно ссылаться на большой набор объявлений («библиотеку»), точно указывая библиотеку в единственной строчке кода. Например:

```
namespace Lib = Foundation_library_v2r11;
// ...
Lib::set s;
Lib::String s5 = "Sibelius";
```

Это кардинально упрощает проблему смены версии библиотеки. Непосредственно применяя **Lib** вместо **Foundation_library_v2r11**, мы можем легко перейти к версии **"v3r02"** изменением одной лишь инициализации псевдонима **Lib** и перекомпиляцией. Перекомпиляция может выявить возможные рассогласования на уровне исходного кода. В то же время, избыточное применение псевдонимов затрудняет понимание кода и ведет к путанице.

8.2.8. Композиция пространств имен

Часто возникает необходимость скомбинировать интерфейс из ряда существующих интерфейсов. Например:

```
namespace His_string
{
    class String { /* ... */ };
    String operator+ (const String&, const String&);
    String operator+ (const String&, const char*);
    void fill (char);
    // ...
}

namespace Her_vector
{
    template<class T> class Vector { /* ... */ };
    // ...
}

namespace My_lib
{
    using namespace His_string;
    using namespace Her_vector;
    void my_fct (Strin&);
}
```

После этого мы можем писать программу в терминах **My_lib**:

```
void f()
{
    My_lib::String s = "Byron";      // находим My_lib::His_string::String
    // ...
}

using namespace My_lib;

void g (Vector<String>& vs)
{
    // ...
    my_fct (vs [5]);
    // ...
}
```

Если явно квалифицированное имя (такое как **My_lib::String**) не объявляется в указанном пространстве имен, то компилятор ищет его в пространствах имен, указанных через директивы **using** (например, в **His_string**).

Истинное (конечное) пространство имен требуется указывать лишь в определениях:

```
void My_lib::fill (char c)           // error: никакого fill() в My_lib нет
{
    // ...
}

void His_string::fill (char c)       // ok: fill() объявлена в His_string
```

```

{
    // ...
}
void My_lib : : my_fct (String& v)      // ok; String есть My_lib : : String, в
                                       // смысле His_String : : String
{
    // ...
}

```

В идеале, пространство имен должно:

1. Отражать логически связанный набор элементов.
2. Ограничивать доступ пользователей к не предназначенным для них элементам.
3. Не требовать излишних усилий при использовании.

Техника композиции пространств имен, рассматриваемая в текущем и последующих разделах, совместно с препроцессорными директивами **#include** (§9.2.1), обеспечивают необходимую поддержку этих требований.

8.2.8.1. Отбор отдельных элементов из пространства имен

Иногда нам нужен доступ лишь к некоторым элементам из пространства имен. Для этого можно объявить пространство имен, содержащее лишь необходимые элементы. Например, мы могли бы объявить иную версию **His_string**, содержащую только класс **String** и перегруженные операции конкатенации:

```

namespace His_string                // лишь часть из His_string
{
    class String { /* ... */ };
    String operator+ (const String&, const String&);
    String operator+ (const String&, const char*);
}

```

Однако если я не являюсь разработчиком или программистом, сопровождающим код, то такой подход быстро приведет к хаосу. Изменения в «настоящем» **His_string** никак не отражаются в данном объявлении. Отбор отдельных элементов из некоторого пространства имен следует явным образом выполнять с помощью объявлений **using**:

```

namespace My_string
{
    using His_string : : String;
    using His_string : : operator+;      // любой + из His_string
}

```

Объявление **using** вносит имя в текущую область видимости. В частности, единственное объявление **using** вносит в область видимости все имеющиеся варианты перегруженных функций.

В таком случае, если разработчик **His_string** внесет новую функцию-член в класс **String**, или добавит новый вариант перегруженной операции конкатенации, то эти изменения станут автоматически доступными пользователям **My_string**. Если же из **His_string** удаляется некоторый элемент или какой-то его элемент изменяется, то ставшее при этом некорректным применение **My_string** сразу же будет обнаружено компилятором (см. также §15.2.2).

8.2.8.2. Композиция пространств имен и отбор элементов

Если объединить композицию пространств имен (при помощи директив *using*) с отбором элементов (при помощи объявлений *using*), то будет получена гибкость, достаточная для решения большинства реальных задач. С помощью этих механизмов мы можем обеспечить доступ к определенному набору средств таким образом, чтобы надежно избегать при этом конфликта имен и неоднозначностей. Например:

```
namespace His_lib
{
    class String { /* ... */ };
    template<class T> class Vector { /* ... */ };
    // ...
}

namespace Her_lib
{
    template<class T> class Vector { /* ... */ };
    class String { /* ... */ };
    // ...
}

namespace My_lib
{
    using namespace His_lib;           // все из His_lib
    using namespace Her_lib;          // все из Her_lib

    using His_lib::String;             // разрешается в пользу His_lib
    using Her_lib::Vector;             // разрешается в пользу Her_lib
    template<class T> class List { /* ... */ }; // прочее
    // ...
}
```

Имена, объявленные явно в пространстве имен (включая те, что присутствуют в составе объявлений *using*) имеют приоритет над именами, внесенными с помощью директив *using* (см. также §C.10.1). Следовательно, пользователь *My_lib* обнаружит, что конфликты имен *String* и *Vector* разрешаются в пользу *His_lib::String* и *Her_lib::Vector*. Также по умолчанию получает приоритет *My_lib::List*, независимо от того, объявляется ли в *His_lib* или *Her_lib* класс *List* или нет.

Обычно, я предпочитаю не менять имя при включении его в новое пространство имен. В результате мне не нужно помнить два разных имени для одного и того же элемента. Тем не менее, иногда нужно или удобно ввести новое имя. Например:

```
namespace Lib2
{
    using namespace His_lib;           // все из His_lib
    using namespace Her_lib;          // все из Her_lib

    using His_lib::String;             // разрешается в пользу His_lib
    using Her_lib::Vector;             // разрешается в пользу Her_lib
    typedef Her_lib::String Her_string; // переименовываем
}
```

```
// "Переименовываем":
template<class T> class His_vec : public His_lib : Vector<T> { /* ... */ };

template<class T> class List { /* ... */ }; // прочее
// ...
}
```

В языке нет специальных средств для переименования. Приходится использовать стандартные механизмы определения новых сущностей.

8.2.9. Пространства имен и старый код

Миллионы строк кода на С и С++ используют глобальные имена и ранние версии библиотек. Как применить пространства имен, чтобы смягчить проистекающие из-за этого проблемы? Переписать существующий код возможно далеко не всегда, но к счастью, библиотеками языка С можно пользоваться так, как будто они определены в пространствах имен. С библиотеками С++ такой номер не проходит (§9.2.4), но пространства имен разработаны таким образом, что их можно было почти что безболезненно добавлять в существующий код на языке С++.

8.2.9.1. Пространства имен и язык С

Рассмотрим каноническую начальную программу на С:

```
#include <stdio.h>
int main ()
{
    printf("Hello, world!\n");
}
```

Вряд ли хорошей идеей будет явная переработка этой программы, так же как и разработка специальных версий стандартных библиотек. Поэтому правила языка для пространств имен разработаны таким образом, что можно относительно легко превратить старый код, не использующий пространств имен, в более структурированную версию, опирающуюся на пространства имен (наша программа калькулятор (§6.1) в целом демонстрирует этот подход).

В качестве одного из способов решения этой задачи, поместим объявления из старых версий файла *stdio.h* в пространство имен *std*:

```
// файл cstdio:
namespace std
{
    int printf(const char* ... );
    //...
}
```

При наличии файла *<csdio>*, можно обеспечить обратную совместимость, если в новый заголовочный файл *<stdio.h>* поместить следующий код:

```
// файл stdio.h:
#include <csdio>
using namespace std;
```

Этот вариант файла *<stdio.h>*, опирающийся на директиву *using*, позволяет успешно скомпилировать старый вариант программы «Hello, world!». К сожалению,

директива **using** вносит в глобальную область видимости вообще все имена из пространства имен **std**. Например:

```
#include <vector>      // избегаем загрязнения глобального пространства
vector v1;            // error: нет "vector" в глобальной области видимости
#include <stdio.h>      // содержит "using namespace std;"
vector v2;            // oops: теперь это работает
```

Поэтому в файле **<stdio.h>** лучше применить объявления **using**, вносящие в глобальную область видимости лишь элементы, определенные в файле **<cstdio>**:

```
// файл stdio.h:
#include <cstdio>
using std::printf;
// ...
```

Еще одно преимущество состоит в том, что объявление **using** предотвращает (случайное или намеренное) определение пользователем нестандартных версий функции **printf()** в глобальной области видимости. Вообще, я рассматриваю нелокальные директивы **using** лишь как переходное средство. В большинстве случаев код в программах, ссылающихся на имена из других пространств имен, лучше выражается при помощи явных квалификаторов и объявлений **using**.

Связь между пространствами имен и компоновкой рассматривается в §9.2.4.

8.2.9.2. Пространства имен и перегрузка

Перегрузка (§7.4) работает поверх пространств имен. Это существенный момент, помогающий мигрировать от старых версий библиотек к применению пространств имен с минимальными изменениями исходного кода. Например:

```
// старый A.h:
void f(int) ;
// ...

// старый B.h:
void f(char) ;
// ...

// старый user.c:
#include "A.h"
#include "B.h"

void g()
{
    f('a') ;           // вызывается f() из B.h
}
```

Эту программу можно переработать в версию с пространствами имен без изменения содержательной части кода:

```
// новый A.h:
namespace A
{
    void f(int) ;
    ..
```

```
// новый B.h:
namespace B
{
    void f(char) ;
    // ...
}

// новый user.c.
#include "A.h"
#include "B.h"

using namespace A;
using namespace B;

void g ()
{
    f('a') ;           // вызывается f() из B.h
}
```

Если бы мы хотели оставить файл **user.c** вообще без изменений, то в таком случае можно было бы поместить директивы **using** в заголовочные файлы.

8.2.9.3. Пространства имен открыты

Пространства имен открыты — это означает, что в них можно добавлять имена в пределах нескольких объявлений одного и того же пространства имен:

```
namespace A
{
    int f() ;           // в A присутствует f()
}

namespace A
{
    int g() ;           // в A присутствуют f() и g()
}
```

Отсюда следует, что мы можем реализовывать большие программные фрагменты в рамках одного и того же пространства имен точно так же, как ранее разрабатывались приложения и библиотеки в рамках единственного глобального пространства. Для этого нужно распределить определение пространства имен по нескольким заголовочным файлам и файлам с исходным кодом. Как показано в примере программы калькулятора (§8.2.4), открытость пространств имен позволяет предоставлять разные интерфейсы разным группам пользователей, открывая для них разные части пространства имен. Открытость также помогает при переходе от старого кода к новым его вариантам. Например, следующий код

```
// мой заголовочный файл:
void f() ;           // моя функция
// ...
#include <stdio.h>
int g() ;           // моя функция
// ...
```

может быть переписан без изменения порядка объявлений:

```
// мой заголовочный файл:
namespace Mine
{
    void f() ;           // моя функция
    // ..
}

#include <stdio.h>

namespace Mine
{
    int g() ;           // моя функция
    // ...
}
```

Разрабатывая новые программы, я предпочитаю организовывать множество мелких пространств имен (§8.2.8), нежели помещать большие фрагменты кода в единственное пространство имен. Однако при конвертировании ранних версий больших программ это не практично.

Для определения элемента, ранее объявленного в пространстве имен, лучше использовать синтаксис **Mine::**, чем повторно открывать **Mine**. Например:

```
void Mine::ff()         // error: ff() нет в Mine
{
    // ..
}
```

Компилятор легко обнаруживает продемонстрированную ошибку. Однако ввиду того, что функции могут сразу определяться в объявлении пространства имен, компилятор не сможет обнаружить эквивалентную ошибку в случае повторного открытия Mine:

```
namespace Mine          // повторное открытие Mine с целью определения функции
{
    void ff()           // oops! ff() добавляется в Mine этим определением
    {
        // ...
    }
    // ...
}
```

Действительно, откуда компилятору знать, что на самом деле вы не вводите новую функцию **ff()**.

Псевдонимы пространств имен (§8.2.7) могут применяться для квалификации имен элементов пространства при их определении. Однако эти псевдонимы нельзя использовать для повторного открытия пространств имен.

8.3. Исключения

Когда программа составляется из отдельных модулей и, особенно, когда эти модули принадлежат независимо разработанным библиотекам, обработку ошибок следует разделить на две части:

1. На выдачу сообщений об условиях возникновения ошибок, локальная обработка которых невозможна.
2. На собственно обработку ошибок, обнаруженных в любых частях программы.

Автор библиотеки в состоянии обнаруживать ошибки времени выполнения, но, как правило, не имеет понятия о том, что с ними делать. Пользователь библиотеки может знать, как поступать в той или иной ошибочной ситуации, но не может их обнаруживать (иначе это были бы исключительно ошибки в пользовательском коде).

В примере с калькулятором мы избежали этой проблемы, создав программу в виде единого целого. Обработка ошибок встраивалась там как часть единой системы. Но после того, как мы разнесли разные логические части калькулятора по разным пространствам имен, обнаружилось, что все пространства имен зависят от пространства имен **Error** (§8.2.2), и что обработка ошибок в **Error** полагается на соответствующее поведение остальных модулей в отношении возникающих в них ошибок. Теперь представим, что у нас нет возможности создавать калькулятор в виде единого целого, и что мы хотим избежать тесной связи между **Error** и другими модулями. Вместо всего этого предположим, что синтаксический анализатор (парсер) и другие модули разрабатываются в отсутствие информации о том, как управляющая программа будет обрабатывать ошибки.

Несмотря на всю простоту функции **error()**, определенная стратегия обработки ошибок в нее была заложена:

```
namespace Error
{
    int no_of_errors;

    double error(const char* s)
    {
        std::cerr<< "error: "<< s << '\n';
        no_of_errors++;
        return 1;
    }
}
```

Функция **error()** выводит сообщение об ошибке, возвращает некоторое предопределенное значение, позволяющее вызвавшему ее коду продолжить работу, а также регистрирует ошибку в переменной **no_of_errors**. При этом очень важно, что все части программы знают о существовании функции **error()**, знают как ее вызывать, и что ожидать от этой функции. К программам, составленным из независимо разработанных библиотек, невозможно предъявлять столь большие требования.

Исключения являются средством языка C++, позволяющим *отделить информирование об ошибках от их обработки*. В данном разделе исключения описаны очень кратко и только в контексте их использования в программе калькулятора. Более обстоятельное изучение исключений и случаев их применения сосредоточено в главе 14.

8.3.1. Ключевые слова **throw** и **catch**

Концепция *исключений* (*exceptions*) введена в язык для генерации сообщений об ошибках. Например:


```

struct Range_error
{
    int i;
    Range_error (int ii) {i=ii; }           // конструктор (§2.5.2, §10.2.3)
};

char to_char (int i)
{
    if (i < numeric_limits<char>::min () || numeric_limits<char>::max () < i) //см §22.2
        throw Range_error ();
    return c;
}

```

Функция `to_char()` либо осуществляет нормальный возврат значения переменной `i` типа `char`, либо генерирует исключение `Range_error`. Фундаментальная идея заключается в том, что если функция обнаруживает ошибочную ситуацию, с которой не в состоянии справиться сама, то она *генерирует исключение* (*throw exception*) некоторого типа в надежде на то, что вызывающая функция проблему так или иначе уладит. Функция, способная решить такую проблему, явным образом указывает намерение *перехватывать исключение* (*catch exception*) указанного типа. Например, для вызова функции `to_char()` и перехвата генерируемых ею исключений, нужно написать следующий код:

```

void g (int i)
{
    try
    {
        char c = to_char (i);
        // ...
    }
    catch (Range_error)
    {
        cerr<< "oops\n";
    }
}

```

Конструкция

```

catch ( /* ... */ )
{
    // ...
}

```

называется *обработчиком исключений* (*exception handler*). Эта конструкция может располагаться только сразу после блока, помеченного ключевым словом `try`, или сразу после другого обработчика исключений; `catch` является ключевым словом языка C++. В круглых скобках располагается объявление того же типа, что и объявления аргументов функций. То есть оно специфицирует тип объектов, которые могут быть перехвачены обработчиком, а также может именовать этот объект (опционально). Например, если бы мы захотели узнать значение сгенерированного в качестве исключения объекта типа `Range_error`, мы бы снабдили этот объект именем так же, как мы снабжаем именами аргументы функций. Например:

```

void h (int i)
{
    try
    {
        char c = to_char (i) ;
        // ...
    }
    catch (Range_error x)
    {
        cerr<< "oops: to_char (" << x.i << ") \n";
    }
}

```

Если код в *try-блоке* (*try-block*), или код, вызываемый из него, генерирует исключение, будут проверяться все обработчики, относящиеся к данному *try*-блоку. Если тип сгенерированного исключения совпадает с объявленным в одном из обработчиков, то выполняется код этого обработчика. Если же ни один из обработчиков не подошел по этому критерию, то все обработчики игнорируются, и в этом случае *try*-блок ведет себя так же, как и обычный блок кода. Если сгенерированное исключение не перехватывается обработчиками, то программа принудительно закрывается (§14.7).

В своей основе, обработка исключений сводится к передаче управления из вызывающей функции в специально указанный (назначенный) код. Если требуется, то дополнительная информация об ошибке может быть передана в вызывающую функцию. Программисты на С могут рассматривать механизм обработки исключений как улучшенную замену для *setjmp/longjmp* (§16.1.2). Важная тема взаимоотношений обработки исключений и классов рассматривается в главе 14.

8.3.2. Обработка нескольких исключений

В общем случае в программе могут возникать разные *ошибки на этапе выполнения* (*run-time errors*). Этим ошибкам можно сопоставить несколько типов исключений с различающимися именами. Я предпочитаю для обработки исключений использовать специально предназначенные для этого типы (чтобы предельно ясно выразить через них свои намерения). В частности, с этой целью я никогда не использую встроенные типы (вроде *int*). В большой программе не будет эффективных способов удостовериться, что такие обработчики имеют дело исключительно с предназначенными для них ошибками. То есть возможна путаница в обработке ошибок из разных источников.

Наша программа калькулятор (§6.1) должна обрабатывать два типа ошибок: синтаксические ошибки и попытки деления на нуль. Нет необходимости передавать обработчику ошибки деления на нуль какую-либо дополнительную информацию, так что этот тип ошибок может быть представлен следующим простейшим (пустым) типом:

```
struct Zero_divide { } ;
```

С другой стороны, обработчик синтаксических ошибок наверняка захочет узнать, каков характер ошибки. В этом случае мы передаем строку:

```
struct Syntax_error
{
    const char* p;
    Syntax_error(const char* q) {p = q; }
};
```

Для удобства я добавил здесь в структуру **конструктор** (§2.5.2, §10.2.3).

Пользователь синтаксического анализатора может осуществить обработку этих двух исключений, добавив обработчики обоих типов к **try**-блоку. По ситуации будет выполняться один из них. По выходе из тела обработчика управление передается коду, следующему за самым последним обработчиком в списке:

```
try
{
    //...
    expr(false) ;
    // сюда попадаем только если expr() не возбуждает исключение
    // ...
}
catch (Syntax_error)
{
    // обработка синтаксической ошибки
}
catch (Zero_divide)
{
    // делаем что-то в ответ на попытку деления на ноль
}
// сюда попадаем, если expr не вызвала исключения или если были исключения
// Syntax_error или Zero_divide, а их обработчики не изменили потока управления
// с помощью return, throw или каким-либо иным способом.
```

Список обработчиков выглядит почти как оператор **switch**, только не требуется оператор **break**. Это отличие в синтаксисе призвано подчеркнуть тот факт, что каждый обработчик образует отдельную область видимости (§4.9.4).

Функции не обязаны перехватывать все возможные типы исключений. Например, предыдущий **try**-блок не имеет обработчика потенциально возможных исключений от операций ввода. Эти исключения просто оставляются для дальнейшего поиска обработчиков (передаются «наверх» по стеку вызовов функций).

С точки зрения синтаксиса языка исключение считается обработанным сразу же после входа в обработчик. Это сделано для того, чтобы исключения, сгенерированные в теле этого обработчика относились к «вышестоящему» коду, вызвавшему соответствующий **try**-блок. Например, в следующем коде нет бесконечного цикла обработки исключений:

```
class Input_overflow { /* ... */ };

void f()
{
    try
```

```

catch (Input_overflow)
{
    // ...
    throw Input_overflow();
}
}

```

Обработчики исключений могут быть вложенными. Например:

```

class XXII { /* ... */ };

void f()
{
    // ...
    try
    {
        // ...
    }
    catch (XXII)
    {
        try
        {
            // что-нибудь нетривиальное
        }
        catch (XXII)
        {
            // код нетривиального обработчика сам "грознулся"
        }
    }
    // ...
}

```

Однако такая вложенность редко встречается в коде, написанном человеком, и вообще-то является плохим стилем.

8.3.3. Исключения в программе калькулятора

Отталкиваясь от базового механизма обработки исключений, мы можем переработать код калькулятора из §6.1 таким образом, чтобы отделить обработку ошибок времени выполнения от основной логики программы. Это приведет к такой организации кода, которая хорошо отражает структуру реальных программ, построенных из отдельных, слабо зависимых частей.

Сначала устраним функцию **error()**. Вместо этого функции парсера будут знать лишь типы, сигнализирующие об ошибках:

```

namespace Error
{
    int no_of_errors;

    struct Zero_divide
    {
        Zero_divide{ } { no_of_errors++; }
    },

```

```

struct Syntax_error
{
    const char* p ;
    Syntax_error (const char* q) { p = q; no_of_errors++; }
};

```

Парсер (синтаксический анализатор) генерирует три типа ошибок:

```

Lexer::Token_value Lexer::get_token ()
{
    using namespace std;           // для доступа к input, isalpha() и т.д. (§6.1.7)
    // ...
    default:                       // NAME, NAME =, или ошибка
        if (isalpha (ch) )
        {
            string_value = ch;
            while (input->get (ch) && isalnum (ch) ) string_value.push_back (ch) ;
            input->putback (ch) ;
            return curr_tok=NAME;
        }
        throw Error::Syntax_error ("bad token" ) ;
    }
}

double Parser::prim (bool get)    // первичные выражения
{
    // ...
    case Lexer::LP:
    {
        double e = expr (true) ;
        if (curr_tok != Lexer::RP) throw Error::Syntax_error (" ' ' ' expected" ) ;
        get_token () ;           // пропустить ')'
        return e;
    }
    case Lexer::END:
        return 1;
    default:
        throw Error::Syntax_error ("primary expected" ) ;
    }
}

```

При обнаружении ошибочной ситуации оператором **throw** генерируется исключение с передачей управления на обработки, определенный где-нибудь в вызывающем (прямо или косвенно) коде. Оператор **throw** передает обработчику также и значение. Например, следующий оператор

```
throw Syntax_error ("primary expected" ) ;
```

передает обработчику объект типа **Syntax_error**, содержащий указатель на строку **"primary expected"**.

Выявление деления на нуль не сопровождается передачей сопутствующих данных:

```
double Parser::term (bool get)    // умножение и деление
{
    // ...
    case Lexer::DIV:
        if (double d = prim (true) )
        {
            left /= d;
            break;
        }
        throw Error::Zero_divide ();
    // ...
}
```

Теперь можно писать управляющий код, который обрабатывает исключения *Syntax_error* и *Zero_divide*:

```
int main (int argc, char* argv[])
{
    // ...
    while (*input)
    {
        try
        {
            Lexer::get_token ();
            if (Lexer::curr_tok == Lexer::END) break;
            if (Lexer::curr_tok == Lexer::PRINT) continue;
            cout << Parser::expr (false) << '\n';
        }
        catch (Error::Zero_divide)
        {
            cerr << "attempt to divide by zero\n";
            if (Lexer::curr_tok != Lexer::PRINT) skip ();
        }
        catch (Error::Syntax_error e)
        {
            cerr << "syntax error: " << e.p << "\n";
            if (Lexer::curr_tok != Lexer::PRINT) skip ();
        }
    }

    if (input != &cin) delete input;
    return no_of_errors;
}
```

Во всех случаях, кроме ошибки в самом конце ограниченного лексемой PRINT выражения (т.е. концом строки или точкой с запятой), функция *main()* вызывает «восстанавливающую» функцию *skip()*. Эта функция пытается вернуть парсер в нормальное состояние, пропуская для этого символы до тех пор, пока не встретится конец строки или точка с запятой. Очевидными кандидатами на членство в пространстве имен *Driver* являются *input* и *skip()*:

```
namespace Driver
```

```
{
    std : istream* input;
    void skip () ;
}

void Driver : : skip ()
{
    while ( *input)           // пропускать символы, пока не встретятся newline-или;
    {
        char ch;
        input->get (ch) ;

        switch (ch)
        {
            case '\n' :
            case ';' :
                return ;
        }
    }
}
```

Код функции **skip**() намеренно написан на более низком уровне абстракции, чем парсер, с целью избежать проблемы генерации новых исключений парсера во время обработки текущих исключений парсера.

Возникает искушение слить воедино пространства имен **Error** и **Driver**, ибо обработка ошибок и операции управляющей программы, такие как ввод/вывод, тесно связаны. Например, подсчет числа ошибок и выдача сообщений о них вполне могли бы быть одной из задач управляющей программы. Однако лучше сохранять явное размежевание между сущностями, разными с логической точки зрения.

Функцию **main**() я не стал помещать в пространство имен **Driver**. Глобальная функция **main**() является стартовой функцией программы (§3.2); нет никакого смысла помещать ее в иное пространство имен. Будь наша программа существенно большего размера, заметная часть кода из функции **main**() была бы перемещена в отдельную функцию из пространства имен **Driver**.

8.3.3.1. Альтернативные стратегии обработки ошибок

Оригинальный код обработки ошибок был короче и элегантнее версии, использующей исключения. Однако это было достигнуто за счет более тесной связи между частями программы. Такой подход плохо масштабируется на большие программы, составленные из независимо разработанных библиотек.

Можно рассмотреть вариант с отказом от отдельной функции обработки ошибок **skip**() за счет введения переменной состояния в функции **main**(). Например:

```
int main (int argc, char* argv[])           // пример плохого стиля
{
    // ...
    bool in_error=false;

    while ( *Driver : : input)
    {
        try
```

```

{
    Lexer : : get_token () ;
    if (Lexer : : curr_tok == Lexer : : END) break;
    if (Lexer : : curr_tok == Lexer : : PRINT)
    {
        in_error = false;
        continue;
    }
    if (in_error == false) cout<<Parser : : expr (false) << '\n' ;
}
catch (Error : : Zero_divide)
{
    cerr<< "attempt to divide by zero\n";
    in_error = true;
}
catch (Error : : Syntax_error e)
{
    cerr<< "syntax error: " << e.p<< "\n";
    in_error = true;
}
}

if (Driver : : input != &std : : cin) delete Driver : : input;
return Error : : no_of_errors;
}

```

Я считаю это плохой идеей по нескольким причинам:

1. Переменные состояния служат источником путаницы и ошибок, особенно если их действие распространяется на большие участки кода. В частности, я считаю версию функции *main* () с переменной состояния *in_error* менее читабельной, чем версия с функцией *skip* () .
2. Стратегия раздельного сосуществования «нормального» кода и кода обработки ошибок в общем случае правильная.
3. Реализация кода обработки ошибок на том же уровне абстракции, что и код, порождающий ошибки, опасна; обрабатывающий ошибки код может снова породить те же самые ошибки, что вызвали текущую их обработку. Я предлагаю в качестве упражнения выяснить, как это может произойти в версии функции *main* () с *in_error* (§8.5[7]).
4. Работы требуется больше при добавлении к «нормальному» коду кода обработки ошибок, чем при добавлении отдельных процедур обработки ошибок.

Обработка исключений предназначена для решения нелокальных по своей природе проблем. Если ошибку можно обработать локально, так и следует поступать практически всегда. Например, нет никаких причин использовать исключения для обработки ошибки, связанной со слишком большим количеством аргументов командной строки:


```

int main (int argc, char* argv[])
{
    using namespace std;
    using namespace Driver;

    switch (argc)
    {
        case 1:                                     // чтение из стандартного потока
            input = &cin;
            break;
        case 2:                                     // чтение командной строки
            input = new istringstream (argv[1]) ;
            break;
        default:
            cerr<< "too many arguments\n";
            return 1;
    }
    // то же, что и раньше
}

```

Далее исключения обсуждаются в главе 14.

8.4. Советы

1. Пользуйтесь пространствами имен для отражения логической структуры; §8.2.
2. Помещайте каждое нелокальное имя, кроме **main()**, в некоторое пространство имен; §8.2.
3. Проектируйте пространства имен таким образом, чтобы пользоваться ими было удобно и чтобы не было случайного доступа к пространствам имен, не имеющим отношения к вашей задаче; §8.2.4.
4. Избегайте слишком коротких названий для пространств имен; §8.2.7.
5. При необходимости применяйте псевдонимы для слишком длинных названий пространств имен; §8.2.7.
6. Не нагружайте пользователей ваших пространств имен слишком сложными обозначениями; §8.2.2, §8.2.3.
7. Применяйте квалифицированные имена вида **Namespace::member** в определениях членов пространства имен; §8.2.8.
8. Используйте директиву **using namespace** только в переходных целях или в локальной области видимости; §8.2.9.
9. Используйте исключения для разъединения кода обработки ошибок и нормального кода; §8.3.3.
10. Для исключений применяйте пользовательские, а не встроенные типы; §8.3.2.
11. Не пользуйтесь исключениями, когда достаточно локального контролирующего кода; §8.3.3.1.

8.5. Упражнения

1. (*2.5) Напишите модуль, реализующий двусвязный список элементов типа *string* в стиле модуля *Stack* из §2.4. Проверьте его на списке названий языков программирования. Реализуйте функцию *sort()* для этого списка, и функцию, изменяющую порядок элементов списка на обратный.
2. (*2) Возьмите не слишком большую программу, которая использует по крайней мере одну библиотеку, не применяющую пространств имен. Модифицируйте программу, чтобы в ней использовалось пространство имен для этой библиотеки. Подсказка: §8.2.9.
3. (*2) Реализуйте программу калькулятор в виде модуля в стиле §2.4, используя пространства имен. Не применяйте директивы *using*. Ведите учет допущенных вами ошибок. Предложите способ, как можно избежать таких ошибок.
4. (*1) Напишите программу, которая генерирует исключение в одной функции, а перехватывает его в другой.
5. (*2) Напишите программу с функциями, вызывающими друг друга с глубиной вложенности вызовов, равной 10. Снабдите каждую функцию аргументом, сообщаемым уровень вложенности, на котором произошла генерация исключения. Заставьте функцию *main()* перехватывать исключения и выводить информацию о перехваченном исключении. Не забудьте случай, когда исключение перехватывается в той же самой функции, где оно генерируется.
6. (*2) Перепишите программу из §8.5[5] так, чтобы она определяла степень затрат на перехват исключений, сгенерированных на разных уровнях вложенности вызовов функций. Добавьте строковый объект в каждую функцию и замерьте все снова.
7. (*1) Найдите ошибку в первой версии функции *main()* из §8.3.3.1.
8. (*2) Напишите функцию, которая в зависимости от аргумента либо возвращает некоторое значение, либо генерирует исключение в виде объекта, имеющего это же значение. Замерьте разницу во времени исполнения этих двух вариантов.
9. (*2) Модифицируйте версию калькулятора из §8.5[3] так, чтобы использовались исключения. Фиксируйте свои ошибки. Предложите способ, как можно избежать таких ошибок.
10. (*2.5) Напишите функции *plus()*, *minus()*, *multiply()* и *divide()*, контролирующие переполнения (и потерю точности) и генерирующие исключения в этих случаях.
11. (*2) Модифицируйте программу калькулятора так, чтобы она использовала функции из §8.5[10].

Исходные файлы и программы

*Форма должна следовать за функцией.
— Ле Корбюзьер*

Раздельная компиляция — компоновка — заголовочные файлы — заголовочные файлы стандартной библиотеки — правило одного определения — компоновка с кодом, написанным не на C++ — компоновка и указатели на функции — использование заголовочных файлов для выражения модульности — единственный заголовочный файл — несколько заголовочных файлов — защита от повторных включений — программы — советы — упражнения.

9.1. Раздельная компиляция

Файл служит традиционной единицей хранения информации на компьютере (в файловой системе) и традиционной единицей компиляции. Хотя в принципе и существуют системы, не хранящие информацию в файлах, и в которых программист не может реализовать программу (и компилировать ее) в виде набора исходных файлов, мы все же ограничимся лишь традиционными системами, ориентированными на файлы.

Обычно невозможно расположить текст всей программы в единственном файле. В частности, большинство стандартных библиотек и операционных систем не поставляется в исходных кодах, так что их код нельзя внедрить в текст пользовательской программы. Даже оставаясь в рамках пользовательских программ (реальных размеров), и неудобно, и непрактично сосредотачивать весь код в единственном файле. Рассредоточение разных частей программы по разным файлам подчеркивает ее логическую структуру, помогает человеку понять программу, а также обеспечивает возможность компилятору отслеживать указанную структуру. Когда единицей компиляции является файл, малейшее его изменение или изменения в файлах, от которых он зависит, требует его перекомпиляции. Даже для программ скромных размеров суммарное время компиляции (и последующих перекомпиляций) заметно уменьшается при разбиении программы на несколько исходных файлов подходящих размеров.

Программист предоставляет в распоряжение компилятора *исходный файл* (*source file*). Затем этот файл препроцессируется, то есть выполняются макроподстановки (§7.8) и директивами **#include** в текст вносится содержимое заголовочных файлов (§2.4.1, §9.2.1). В результате препроцессирования образуется то, что принято называть *единицей трансляции* (*translation unit*). Содержимое единицы трансляции описывается правилами языка C++ и над ним, собственно, и работает компилятор. В настоящей книге я подчеркиваю разницу между исходным файлом и единицей трансляции только тогда, когда нужно различать, что видит программист, а что «видит» компилятор.

Чтобы раздельная компиляция могла быть реализована, программист должен предоставить объявления с информацией о типах, достаточные для того, чтобы можно было выполнить анализ единицы трансляции отдельно от остальных частей программы. Все объявления в программе, состоящей из нескольких раздельно компилируемых частей, должны согласовываться между собой точно так же, как и в программе с единственным исходным файлом. У вас должны быть программные средства, позволяющие убедиться в этом. В частности, компоновщик может обнаружить большую часть рассогласований. *Компоновщик* (*linker*) — это программа, которая связывает воедино раздельно откомпилированные части. Иногда компоновщик называют (неправильно) *загрузчиком* (*loader*). Компоновка может быть полностью завершена до загрузки программы в память компьютера для ее выполнения. С другой стороны, к программе можно добавить новый код («динамическая компоновка») уже после загрузки программы.

Организацию программы в виде нескольких исходных файлов часто называют *физической структурой* (*physical structure*) программы. Физическое разделение программы на отдельные файлы желательно выполнять в соответствии с логической структурой программы. Те же самые соображения о зависимостях, которые помогают осуществлять композицию программ из пространств имен, помогают и при разбиении программ на исходные файлы. В то же время, логическая и физическая структуры программы не обязаны быть идентичными. К примеру, может оказаться удобным разместить определения функций из пространства имен по нескольким исходным файлам, или сосредоточить в единственном файле несколько определений разных пространств имен, или разбросать определение единственного пространства имен по нескольким файлам (§8.2.4).

Далее, мы сначала рассмотрим ряд технических вопросов, связанных с компоновкой, а потом обсудим пару способов разбиения нашей программы-калькулятора (§6.1, §8.2) на отдельные файлы.

9.2. Компоновка (linkage)

Имена функций, классов, шаблонов, пространств имен и перечислений должны согласовываться для всех файлов программы, если только эти имена не объявлены явным образом как локальные.

Программист должен обеспечить правильное объявление имен классов, функций, пространств имен и т.д. в каждой единице трансляции, где они используются, и следить за тем, чтобы все объявления одних и тех же программных объектов были согласованы. Например, рассмотрим два файла:

```
// файл file1.c:
int x = 1;
int f() { /* do something */ }

// файл file2.c:
extern int x;
int f();
void g() { x = f(); }
```

Переменная *x* и функция *f()*, используемые в функции *g()* из файла *file2.c*, определены в файле *file1.c*. Ключевое слово **extern** означает, что объявления *x* и *f()* есть исключительно (и только) объявления, но не определения (§4.9). Если бы в объявлении *x* присутствовало инициализирующее выражение, то тогда **extern** было бы проигнорировано, поскольку объявление с инициализирующим выражением всегда является определением. Любой объект должен определяться в программе ровно один раз. Объявляться же он может многократно, но типы при этом должны совпадать. Например:

```
// файл file1.c:
int x = 1;
int b = 1;
extern int c;

// файл file2.c:
int x;           // означает int x = 0;
extern double b;
extern int c;
```

В этом примере имеются три ошибки: переменная *x* определена дважды, в разных объявлениях переменной *b* указаны разные типы, а переменная *c* объявлена дважды, но ни разу не определена. Ошибки такого рода (ошибки этапа компоновки) не могут быть обнаружены на этапе компиляции, ибо компилятор работает с каждым файлом отдельно. Большинство таких ошибок позже выявляются компоновщиком. Следует отметить, что переменная, определяемая без инициализатора в глобальной области видимости или в рамках некоторого пространства имен, инициализируется по умолчанию. Это не относится к локальным переменным (§4.9.5, §10.4.2) или к объектам, создаваемым в свободной памяти (в куче — §6.2.6). Например, следующий программный фрагмент содержит две ошибки:

```
// файл file1.c:
int x;
int f() { return x; }

// файл file2.c:
int x;
int g() { return f(); }
```

Вызов функции *f()* в файле *file2.c* является ошибкой, так как функция *f()* в этом файле не объявлена. Программа не будет скомпонована также еще и потому, что переменная *x* определена дважды. А вот в языке C вызов *f()* в файле *file2.c* ошибкой не является (§B.2.2).

Говорят, что имена, которые можно использовать в единицах трансляции, отличных от той, где они определены, имеют *внешнюю компоновку* (*external linkage*).

Все имена в предыдущих примерах компоновались внешним образом. Если же на имя можно сослаться лишь в единице трансляции, где оно определено, то говорят, что имя имеет *внутреннюю компоновку* (*internal linkage*).

Функции с модификатором **inline** (§7.1.1, §10.2.9) должны определяться — посредством идентичных определений (§9.2.3) — в каждой единице трансляции, где они используются (вызываются). Следовательно, следующий пример не просто демонстрирует плохой стиль программирования, он вообще недопустим:

```
// файл file1.c:
inline int f(int i) { return i; }

// файл file2.c:
inline int f(int i) { return i+1; }
```

К сожалению, такого рода ошибки с трудом обрабатываются конкретными реализациями, так что следующий пример (вполне логически корректный) — комбинация внешней компоновки и встраивания, запрещается в угоду компиляторам:

```
// файл file1.c:
extern inline int g(int i);
int h(int i) { return g(i); } // error: g() не определена в данной единице трансляции

// файл file2.c:
extern inline int g(int i) { return i+1; }
```

По умолчанию **const** (§5.4) и **typedef** (§4.9.7) подразумевают внутреннюю компоновку. В результате следующий пример является допустимым (но сбивающим с толку):

```
// файл file1.c:
typedef int T;
const int x = 7;

// файл file2.c:
typedef void T;
const int x = 8;
```

Лучше избегать определения глобальных переменных с внутренней компоновкой. Для обеспечения согласованности лучше помещать глобальные объекты с модификаторами **const** или **inline** только в заголовочные файлы (§9.2.1).

С помощью ключевого слова **extern** объектам с модификатором **const** можно навяать внешнюю компоновку:

```
// файл file1.c:
extern const int a = 77;

// файл file2.c:
extern const int a;
void g()
{
    cout<< a << '\n';
}
```

Функция **g()** выведет 77.

С помощью анонимных (неименованных) пространств имен (§8.2.5) можно превращать имена в локальные по отношению к единице компиляции. Эффект от неименованных пространств имен очень близок к внутренней компоновке. Например:

```
// файл file1.c:
namespace
{
    class X { /* ... */ };
    void f();
    int i;
    // ...
}
```

```
// файл file2.c:
class X { /* ... */ };
void f();
int i;
// ...
```

Здесь функция *f()* из *file1.c* не та же самая, что *f()* из *file2.c*. Использовать одинаковое имя, являющееся локальным для некоторой единицы трансляции, и обозначающее сущности с внешней компоновкой в других единицах трансляции — значит нарываться на неприятности.

В языке С и в более ранних версиях С++ ключевое слово **static** означает (и это может привести к путанице) «использовать внутреннюю компоновку» (§B.2.3). Не применяйте **static** иначе как внутри функций (§7.1.2) или классов (§10.2.4).

9.2.1. Заголовочные файлы

Типы во всех объявлениях одной и той же сущности должны быть согласованы. Это означает, что исходный код, подаваемый на вход компилятору и позднее собираемый в единое целое компоновщиком, должен быть согласован. Простой (хотя и несовершенный) метод достижения согласованности объявлений в разных единицах трансляции состоит в применении директив **#include header files** для внесения интерфейсной информации в исходные файлы с исполняемым кодом и/или определением данных.

Механизм директив **#include** является простым текстовым средством, позволяющим собрать воедино разные фрагменты исходного кода в единицу (файл) компиляции. Директива

```
#include "to_be_included"
```

замещает строку, содержащую **#include**, на содержимое файла *to_be_included*. Файл должен содержать код на С++, так как он поступит на обработку компилятором языка С++.

Имена стандартных библиотечных заголовочных файлов заключайте в угловые скобки < и > (а не в кавычки). Например:

```
#include <iostream>           // из стандартного каталога заголовочных файлов
#include "myheader.h"         // из текущего каталога
```

Обратите внимание на то, что пробелы внутри угловых скобок или кавычек существенны для выполнимости директивы **#include**:

```
#include < iostream > // не будет найден файл <iostream>
```

Необходимость компилировать файл каждый раз при его включении куда-либо может показаться несколько экстравагантной, но заголовочные файлы в типичном случае содержат только объявления, а не код, требующий серьезного анализа со стороны компилятора. Более того, все современные реализации обеспечивают некоторую форму *предкомпиляции* (*precompiling*) заголовочных файлов для минимизации усилий, требуемых для их повторных компиляций.

Согласно эмпирическому правилу заголовочные файлы могут содержать следующие элементы:

Именованные пространства имен	<code>namespace N { /* ... */ }</code>
Определения типов	<code>struct Point { int x, y; };</code>
Объявления шаблонов	<code>template<class T> class Z;</code>
Определения шаблонов	<code>template<class T> class V { /* ... */ }</code>
Объявления функций	<code>extern int strlen (const char*);</code>
Определения встроенных функций	<code>inline char get (char* p) { return *p++; }</code>
Объявления данных	<code>extern int a;</code>
Определения констант	<code>const float pi = 3.141593;</code>
Перечисления	<code>enum Light { red, yellow, green };</code>
Объявления имен	<code>class Matrix;</code>
Директивы включения	<code># include <algorithm></code>
Макроопределения	<code># define VERSION 12</code>
Директивы условной компиляции	<code># ifdef __cplusplus</code>
Комментарии	<code>/* проверка на конец файла */</code>

Данное эмпирическое правило не является требованием языка. Оно просто формулирует разумные способы использования заголовочных файлов (директив **#include**) для выражения физической структуры программы. С другой стороны, заголовочные файлы не должны содержать следующего:

Определения обычных функций	<code>char get (char* p) { return *p++; }</code>
Определения данных	<code>int a;</code>
Определения агрегатов	<code>short tbl[] = {1, 2, 3};</code>
Неименованные пространства имен	<code>namespace { /* ... */ }</code>
Экспортируемые определения шаблонов	<code>export template<class T> f(T t) { /* ... */ }</code>

Обычно, заголовочные файлы имеют расширение **.h**, а файлы, содержащие определения функций и/или данных, имеют расширение **.c**. Поэтому на них часто ссылаются как на "**.h** файлы" или "**.c** файлы", соответственно. Также в ходу расширения **.C**, **.cxx**, **.cpp** и **.cc**. В руководстве к вашему компилятору об этом должно быть сказано со всей определенностью.

Причина, по которой в заголовочные файлы рекомендуется включать определения простых констант, а определения агрегатов включать не рекомендуется, заключается в том, что реализациям трудно избежать репликации агрегатов в не-

скольких единицах трансляции. К тому же, более простые конструкции встречаются чаще, а потому для них важнее генерировать более качественный машинный код.

Будет разумным не переусердствовать в изобретении слишком «умных» случаев использования директив **#include**. Мои рекомендации сводятся к тому, чтобы включать лишь полные объявления и определения в глобальной области видимости, в блоках спецификации компоновки и в пространствах имен для конвертирования старого кода (§9.2.2). Как всегда, стоит избегать магических фокусов с макроопределениями. Я, например, страшно не люблю отслеживать ошибки неожиданных результатов макроподстановок из косвенно включаемых заголовочных файлов, о которых я даже, ровным счетом, ничего и не слышал.

9.2.2. Заголовочные файлы стандартной библиотеки

Средства стандартной библиотеки описаны в наборе стандартных заголовочных файлов (§16.1.2). В именах этих заголовочных файлов расширения не используются; заголовочные же они потому, что включаются в текст программы с помощью **#include** <...> (стандартные заголовки требуют угловых скобок, а не кавычек). Отсутствие суффикса **.h** не отражает способа хранения содержимого этих файлов; например, содержимое **<map>** может располагаться в файле **map.h** из стандартного каталога. Более того, вообще не требуется, чтобы стандартные заголовки хранились традиционным образом. Конкретные реализации могут по максимуму использовать свои знания о стандартной библиотеке и ее заголовках, реализуя оптимизированные способы их хранения и представления. Например, компилятор, отталкиваясь от своих знаний о встроенном характере стандартной математической библиотеки (§22.3), может трактовать директиву **#include <cmath>** просто как некий логический переключатель (тумблер), открывающий возможность применения в программе стандартных математических функций, а вовсе не как требование чтения конкретного файла.

Для каждого стандартного заголовка **<X.h>** языка C имеется соответствующий стандартный заголовок **<cX>** языка C++. Например, **#include <cstdio>** обеспечивает то же, что и **#include <stdio.h>**. В типичном случае, содержимое **stdio.h** выглядит следующим образом:

```
#ifdef __cplusplus           // только для C++ компиляторов (§9.2.4)
namespace std {             // стандарт. биб-ка определена в прост. имен std (§8 2.9)

extern "C" {                 // функции stdio имеют компоновку языка C (§9.2.4)
    #endif

    /* ... */
    int printf(const char* ...);
    /* ... */

    #ifdef __cplusplus
    }
    }
    // ...
    using std::printf;       // делает printf доступным в глобальном пространстве имен
    // ...
    #endif
```

Таким образом, реальное содержимое (объявления) для обоих языков одинаковое, а пространства имен и режимы компоновки введены так, что этот файл можно использовать из программ на C и C++.

9.2.3. Правило одного определения

Каждый класс, перечисление, шаблон и т.д. должны быть определены в программе ровно один раз.

С практической точки зрения это означает, что должно существовать единственное определение, например, класса, находящееся в каком-либо одном файле. К сожалению, языковые правила не столь просты. Например, определение класса можно сформировать через макроопределение (б-р-р!), или определение класса может быть текстуально включено с помощью директив **#include** сразу в несколько исходных файлов (§9.2.1). Еще хуже то, что файл вообще не является концепцией языков C/C++; могут существовать реализации, не использующие файлы для хранения программ.

В результате правило стандарта, гласящее о том, что определения классов, шаблонов и т.д. должны быть уникальными, на самом деле формулируется сложнее и тоньше. Это правило называют «*правилом одного определения*» («*the one-definition rule*» — сокращенно *ODR*): два определения класса, шаблона или встраиваемой функции принимаются за экземпляры одного и того же уникального определения тогда и только тогда, если:

1. Они находятся в разных единицах трансляции.
2. Они полексемно идентичны.
3. Смысл этих лексем одинаковый в обеих единицах трансляции.

Например:

```
// файл file1.c:
struct S { int a; char b; };
void f(S*);

// файл file2.c:
struct S { int a; char b; };
void f(S* p) { /* ... */ }
```

Правило ODR говорит, что данный пример корректен, и что определения *S* в обоих исходных файлах ссылаются на один и тот же класс (структуру). Конечно же, крайне неразумно дважды записывать одно и то же определение таким образом. Кто-нибудь, модифицируя файл *file2.c*, может предположить, что определение *S* из этого файла единственное в программе, и спокойно изменить его. Это приведет к трудноуловимой ошибке.

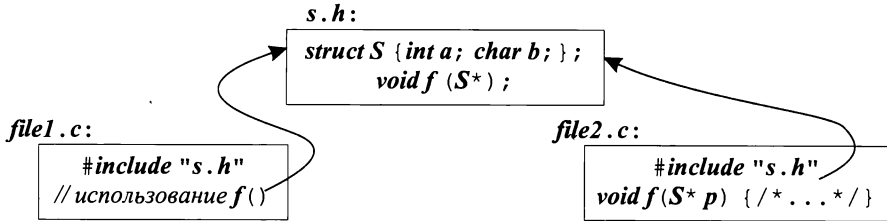
Правило ODR нацелено на то, чтобы можно было включать определение класса из единственного файлового источника в разные единицы трансляции. Например:

```
// файл s.h:
struct S { int a; char b; };
void f(S*);

// файл file1.c:
#include "s.h"
// используем f() здесь
```

```
// файл file2.c:
#include "s.h"
void f(S* p) { /* ... */ }
```

или в графической форме:



Приведем три ошибочных примера, в которых правило ODR нарушается:

```
// файл file1.c:
struct S1 {int a; char b; };
struct S1 {int a; char b; };           // error: повторное определение
```

Здесь ошибка заключается в том, что нельзя дважды определять структуру в одной и той же единице трансляции.

А в следующем примере ошибка состоит в том, что определения структуры **S2** в разных файлах содержат разные имена членов структуры.

```
// файл file1.c:
struct S2 {int a; char b; };

// файл file2.c:
struct S2 {int a; char bb; };         // error
```

В третьем ошибочном примере два определения структуры **S3** полексемно идентичны, но смысл лексемы **X** в разных файлах разный:

```
// файл file1.c:
typedef int X;
struct S3 {X a; char b; };

// файл file2.c:
typedef char X;
struct S3 {X a; char b; };           // error
```

Проверка согласованности определений классов в разных единицах трансляции выходит за пределы возможностей большинства реализаций C++. Как следствие, нарушение правила ODR влечет за собой возникновение тонких ошибок. К сожалению, размещение общих определений в заголовочных файлах с последующих их включением директивами **#include** не гарантирует отсутствия последней из представленных в примерах форм нарушения правила ODR. Локальные операторы **typedef** и макросы могут менять смысл объявлений из заголовочных файлов:

```
// файл s.h:
struct S {Point a; char b; };

// файл file1.c:
#define Point int
```

```
#include "s.h"
// ...
// файл file2.c:
class Point { /* ... */ };
#include "s.h"
// ...
```

Наилучшей защитой от этой проблемы является создание как можно более самодостаточных заголовочных файлов. Например, если бы класс **Point** был объявлен в **s.h**, ошибка была бы обнаружена.

Определение шаблонов можно включать в разные единицы трансляции до тех пор, пока выдерживается правило ODR. Кроме того, если шаблон является экспортируемым (определен в некотором файле с модификатором **export**), то в ином файле его можно использовать при наличии одного лишь объявления:

```
// файл file1.c:
export template<class T> T twice (T t) {return t+t; }

// файл file2.c:
template<class T> T twice (T t) ; // объявление
int g (int i) {return twice (i) ; }
```

Ключевое слово **export** означает «доступно из другой единицы трансляции» (§13.7).

9.2.4. Компоновка с кодом, написанном не на языке C++

Нередко программы, написанные в основном на C++, содержат фрагменты кода на других языках. И наоборот, программы на других языках используют фрагменты на C++. Добиться корректного взаимодействия таких фрагментов между собой непросто, даже для фрагментов на C++, компилируемых разными компиляторами. Действительно, разные языки и разные компиляторы для одного языка могут по-разному использовать регистры процессора для хранения аргументов, по-разному выполнять раскладку стековых фреймов, могут различаться размещением в памяти отдельных символов и строк символов, целых чисел, чисел с плавающей запятой, могут по-разному передавать имена компоновщику, могут различаться глубиной проверки типов, необходимой компоновщику. Определенную помощь оказывает явное указание *соглашений компоновки (linkage convention)* в объявлениях с модификатором **extern**. В следующем примере объявляется функция **strcpy()** стандартной библиотеки языков C/C++ и явно указывается, что компоновку нужно осуществлять по правилам языка C:

```
extern "C" char* strcpy (char*, const char*) ;
```

Эффект от такого объявления отличается от эффекта обычного объявления

```
extern char* strcpy (char*, const char*) ;
```

только соглашением о вызове функции **strcpy()**.

Директива **extern "C"** особо полезна из-за тесной связи между языками C и C++. Важно понимать, что **C** в **extern "C"** означает именно соглашение о компоновке, а не язык сам по себе. Часто, **extern "C"** применяется с целью компоновки с процедурами, написанными на языке Fortran или ассемблере, которые удовлетворяют соглашениям языка C.

Директива ***extern "C"*** специфицирует соглашение о компоновке (и только) и не влияет на семантику вызовов функции. В частности, функция, объявленная с ***extern "C"***, по-прежнему подчиняется проверке типов и правилам преобразования аргументов, принятым в C++, а не более слабым правилам языка C. Например:

```
extern "C" int f() ;

int g ()
{
    return f(1) ;           // error: неожиданный аргумент
}
```

Так как добавление ***extern "C"*** к большой группе объявлений утомительно, то предусмотрен механизм группового использования этой директивы:

```
extern "C"
{
    char* strcpy(char*, const char*) ;
    int strcmp(const char*, const char*) ;
    int strlen(const char*) ;
    // ..
}
```

Эту конструкцию часто называют *блоком спецификации компоновки (linkage block)* и в нее можно заключить все содержимое заголовочного файла языка C, чтобы сделать его применимым в программах на языке C++. Например:

```
extern "C" {
#include <string.h>
}
```

Такая техника, превращающая заголовочный файл языка C в заголовочный файл языка C++, на практике используется очень часто. Кроме того, применив еще и директивы условной компиляции (§7.8.1), можно получить заголовочный файл, одинаково пригодный для C и C++:

```
#ifdef __cplusplus
extern "C" {
#endif

char* strcpy(char*, const char*) ;
int strcmp(const char*, const char*) ;
int strlen(const char*) ;
// ...

#ifdef __cplusplus
}
#endif
```

Предопределенный макрос ***__cplusplus*** позволяет исключить конструкции C++, когда файл используется в качестве заголовочного файла C.

В блоке спецификации компоновки допускаются любые объявления:

```
extern "C" {
int g1;                // определение
extern int g2;         // объявление (но не определение)
}
```

Никакого дополнительного влияния на области видимости и время жизни переменных при этом не оказывается, так что **g1** по-прежнему является глобальной переменной, причем эта переменная определена, а не просто объявлена. Чтобы объявить переменную (а не определить ее), нужно явным образом использовать модификатор **extern**. Например:

```
extern "C" int g3;      // объявление (не определение)
```

Выглядит немного странно, но на самом деле, это просто следствие неизменности смысла объявлений с модификатором **extern** при добавлении "C" (аналогично неизменности смысла содержимого файла при заключении его в блок спецификации компоновки).

Имя, которое должно компоноваться в стиле языка C, можно объявлять в пространстве имен. Это влияет лишь на то, как имя видимо в коде C++, но не на его представление компоновщику. Функция **printf()** из пространства имен **std** служит типичным примером:

```
#include <cstdio>

void f()
{
    std::printf("Hello, ");    // ok
    printf("world!\n");        // error: нет глобальной printf()
}
```

Несмотря на обращение в виде **std::printf()**, вызовется-то по-прежнему старая добрая стандартная функция **printf()** языка C (§21.8).

Это позволяет нам заключать библиотеки с C-компоновкой в произвольные пространства имен и не засорять единственное глобальное пространство имен. Подобная гибкость отсутствует для библиотек, содержащих глобальные определения функций с компоновкой в стиле C++. Причина состоит в том, что компоновка C++-сущностей учитывает пространства имен, и объектные файлы в явном виде содержат информацию об этом (то есть используются пространства имен или нет).

9.2.5. Компоновка и указатели на функции

При смешении фрагментов кода на C и C++ часто приходится передавать указатели на функции одного языка функциям другого языка. Если обе реализации двух языков имеют общие соглашения о компоновке и вызове функций, то проблем при этом не возникает. В общем же случае на это рассчитывать нельзя, так что требуется предпринимать специальные меры, гарантирующие, что вызов функции будет осуществляться надлежащим образом.

Когда соглашение о компоновке указывается в объявлении, то его действие распространяется на все типы, имена и переменные в этом объявлении. Это делает

возможными странные на первый взгляд, но иногда важные комбинации соглашений о компоновке. Например:

```
typedef int (*FT) (const void*, const void*);           // FT имеет C++ компоновку

extern "C" {
    typedef int (*CFT) (const void*, const void*);       // CFT имеет C компоновку
    void qsort (void* p, size_t n, size_t sz, CFT cmp);   // cmp имеет C компоновку
}

void isort (void* p, size_t n, size_t sz, FT cmp);        // cmp имеет C++ компоновку
void xsort (void* p, size_t n, size_t sz, CFT cmp);       // cmp имеет C компоновку
extern "C" void ysort (void* p, size_t n, size_t sz, FT cmp); // cmp имеет C++ компоновку

int compare (const void*, const void*);                 // compare() имеет C++ компоновку
extern "C" int ccmp (const void*, const void*);          // ccmp() имеет C компоновку

void f(char* v, int sz)
{
    qsort (v, sz, 1, &compare);           // error
    qsort (v, sz, 1, &ccmp);              // ok

    isort (v, sz, 1, &compare);           // ok
    isort (v, sz, 1, &ccmp);              // error
}
```

Реализации C и C++ с одинаковыми соглашениями о вызовах могут принять случаи, отмеченные комментарием *//error*, как языковые расширения.

9.3. Применяем заголовочные файлы

Для иллюстрации практического применения заголовочных файлов я сейчас рассмотрю ряд альтернативных способов выражения физической структуры нашей программы калькулятора (§6.1, §8.2).

9.3.1. Единственный заголовочный файл

Простейшим решением проблемы разбиения программы на несколько файлов является размещение определений в подходящем числе .c файлов и помещение всех нужных для их взаимодействия объявлений типов в единственный заголовочный файл, подлежащий включению во все остальные файлы директивой **#include**. Для программы калькулятора можно использовать пять .c файлов — *lexer.c*, *parser.c*, *table.c*, *error.c* и *main.c* — для хранения функций и определений данных, и один заголовочный файл *dc.h* для хранения объявлений имен, используемых более чем в одном .c файле.

Вот возможное содержимое файла *dc.h*:

```
// файл dc.h:

namespace Error
{
    struct Zero_divide {};
    struct Syntax_error
```

```

{
    const char* p;
    Syntax_error (const char* q) {p = q; }
};
}

#include <string>

namespace Lexer
{
    enum Token_value
    {
        NAME,           NUMBER,           END,
        PLUS='+',        MINUS='-',        MUL='*',        DIV=' / ',
        PRINT=';',       ASSIGN='=',      LP=' ( ',       RP=' ) '
    };

    extern Token_value curr_tok;
    extern double number_value;
    extern std::string string_value;
    Token_value get_token ();
}

namespace Parser
{
    double prim (bool get);           // обработка первичных выражений
    double term (bool get);           // умножение и деление
    double expr (bool get);           // сложение и вычитание

    using Lexer::get_token;
    using Lexer::curr_tok;
}

#include <map>

extern std::map<std::string, double> table;

namespace Driver
{
    extern int no_of_errors;
    extern std::istream* input;
    void skip ();
}

```

Ключевое слово **extern** используется для объявления переменных, чтобы гарантировать отсутствие множественных определений при включении файла **dc.h** в разные **.c** файлы. Соответствующие определения располагаются в том или ином **.c** файле.

За вычетом реального кода файл **lexer.c** будет выглядеть следующим образом:

```

// файл lexer.c:

#include "dc.h"
#include <iostream>
#include <cctype>

```



```

Lexer : : Token_value Lexer : : curr_tok;
double Lexer : : number_value;
std : : string Lexer : : string_value;

Lexer : : Token_value Lexer : : get_token () { /* ... */ }

```

Использование заголовочных файлов гарантирует, что каждое объявление из этих файлов будет помещено в *.c* файл, содержащий соответствующее определение. Например, при компиляции *lexer.c* компилятор повстречает объявление

```

namespace Lexer                                     // из dc.h
{
    // ...
    Token_value get_token ();
}

// ...

Lexer : : Token_value Lexer : : get_token () { /* ... */ }

```

Это, в свою очередь, гарантирует, что компилятором будут выявлены все случаи рассогласования типов. Например, если бы функция *get_token* () в объявлении возвращала бы значение типа *Token_value*, а в определении — значение типа *int*, то компиляция файла *lexer.c* завершилась бы с ошибкой несоответствия типов. Если отсутствует определение, ошибку обнаружит компоновщик, а если нет объявления — это обнаружит компилятор.

Файл *parser.c* будет выглядеть следующим образом:

```

// файл parser.c:
#include "dc.h"

double Parser : : prim (bool get) { /* ... */ }
double Parser : : term (bool get) { /* ... */ }
double Parser : : expr (bool get) { /* ... */ }

```

А вот содержимое файла *table.c*:

```

// файл table.c:
#include "dc.h"

std : : map<std : : string, double> table;

```

Таблица символов является просто глобальной переменной *table* стандартного библиотечного типа *map*. В программах реальных (больших) размеров загрязнение глобального пространства имен вызовет в конце концов проблемы. Но здесь я допустил эту небрежность намеренно, чтобы появился повод предупредить о возможных проблемах.

Наконец, файл *main.c* имеет следующий вид:

```

// файл main.c:
#include "dc.h"
#include <sstream>
#include <iostream>

int Driver : : no_of_errors = 0;
std : : istream* Driver : : input = 0;

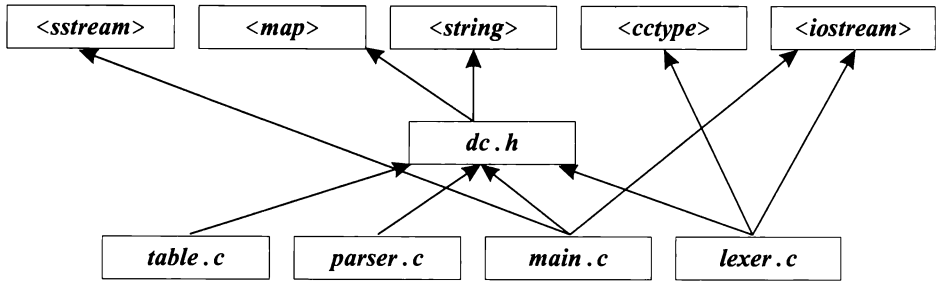
```

```
void Driver::skip() { /* ... */ }

int main (int argc, char* argv[]) { /* ... */ }
```

Функция **main()**, как стартовая функция программы, должна быть глобальной, и поэтому она не помещена ни в какое пространство имен.

Физическую структуру программы можно представить графически:



Заголовочные файлы в верхней части рисунка являются стандартными библиотечными заголовками. В большинстве случаев при анализе программ эти заголовочные файлы можно опустить, ибо про эту стабильную составляющую практически любой программы и так все известно заранее. Крошечные программы можно предельно упростить, поместив все директивы **#include** в один заголовочный файл.

Такой стиль физического сегментирования программ хорош для программ малых размеров, отдельные части которых не предполагается использовать отдельно. Если используются пространства имен, то логическая структура программы также отражается в **dc.h**. Если же пространства имен не используются, то структура становится туманной. В этом случае помогают комментарии.

Для программ больших размеров стиль сегментирования с единственным заголовочным файлом становится неработоспособным в обычных средах разработки, ориентированных на файлы. Изменения в единственном заголовочном файле вызывают перекомпиляцию всей программы, а работа с этим единственным файлом разных программистов чревата ошибками. По мере роста размеров программ их логическая структура катастрофически ухудшается, если разработка ведется без опоры на пространства имен и классы.

9.3.2. Несколько заголовочных файлов

Альтернативой является такая физическая организация программы, когда каждому логическому модулю соответствует свой заголовочный файл, содержащий все необходимые модулю объявления. Каждый **.c** файл располагает соответствующим заголовочным файлом, определяющим его интерфейс. Каждый **.c** файл директивами **#include** включает свой заголовочный файл и иные заголовочные файлы, определяющие то, что нужно модулю от других модулей. В этом случае физическая организация соответствует логической организации модуля. Интерфейс для пользователей модуля размещается в **.h** файле, интерфейс для реализации модуля — в файле с суффиксом **_impl.h**, а сама реализация модуля (функции, определения переменных и т.д.) сосредотачивается в **.c** файле. В результате модуль представляется тремя файлами. Например, интерфейс для пользователей модуля парсера располагается в файле **parser.h**:

// файл *parser.h*:

```
namespace Parser
{
    double expr (bool get) ;
}
```

Заголовочный файл реализации этого модуля помещается в заголовочный файл *parser_impl.h*:

```
// файл parser_impl.h:
#include "parser.h"
#include "error.h"
#include "lexer.h"

namespace Parser          // интерфейс для реализации
{
    double prim (bool get) ;
    double term (bool get) ;
    double expr (bool get) ;

    using Lexer: :get_token;
    using Lexer: :curr_tok;
}
```

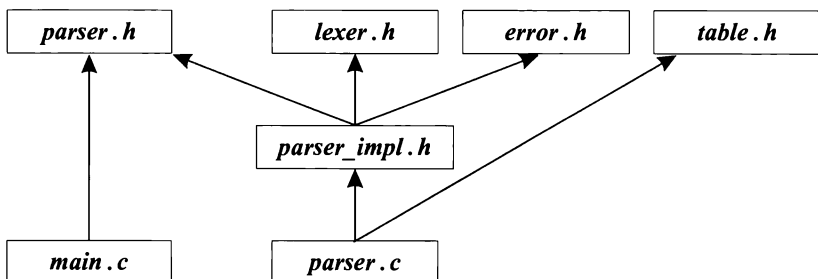
Сюда включен и заголовочный файл для пользователей *parser.h*, чтобы компилятор имел возможность проверить согласованность объявлений (§9.3.1).

Реализация парсера располагается в файле *parser.c*, включающем все необходимые ему заголовочные файлы:

```
// файл parser.c:
#include "parser_impl.h"
#include "table.h"

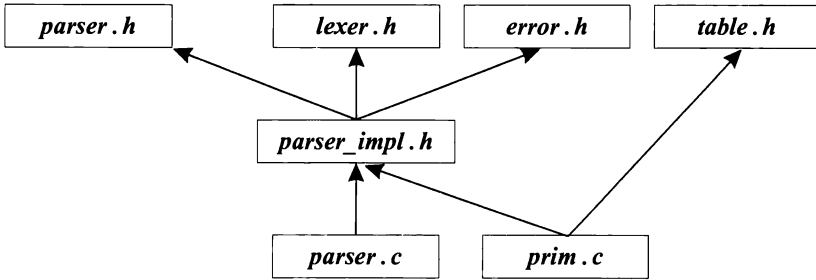
double Parser: :prim (bool get) { /* ... */ }
double Parser: :term (bool get) { /* ... */ }
double Parser: :expr (bool get) { /* ... */ }
```

В графическом виде парсер и его использование управляющим модулем выглядит так:



Как и было задумано, здесь достигнуто близкое соответствие логической структуре, описанной ранее в §8.3.2. Для ее дальнейшего упрощения можно было бы по-

местить директиву `#include "table.h"` в файл `parser_impl.h` вместо файла `parser.c`. Однако файл `table.h` не относится к общему интерфейсу функций синтаксического анализатора (парсера); скорее он нужен лишь реализации этих функций. Более точно, он нужен лишь функции `prim()`, так что если уж мы на самом деле были бы настроены минимизировать зависимости, то стоило бы функцию `prim()` выделить в отдельный `.c` файл и только в него включать `table.h`:



Столь тщательная проработка уместна, в первую очередь, для больших программ. Для них типично включение дополнительных заголовочных файлов только там, где это требуется отдельными функциями. Более того, для больших программ нередко используется несколько заголовочных файлов с суффиксом `_impl.h`, поскольку разным подмножествам функций модуля требуются разные интерфейсы реализации.

Обратите внимание на то, что применение суффикса `_impl.h` не только не является стандартом, но даже и нельзя сказать, что сильно распространенным явлением; просто лично мне нравится такой стиль именования.

Итак, для чего же все-таки стоит возиться с более сложной схемой со многими заголовочными файлами? Ведь намного проще поместить все объявления в единственный заголовочный файл `dc.h`.

Схема со *многими заголовочными файлами* отвечает масштабу модулей, во много раз больших нашего игрушечного парсера, и масштабу программ, намного больших нашего калькулятора. *Фундаментальная идея* сводится к *большей локализации*. Когда программист анализирует и модифицирует большую программу, ему важно иметь возможность *сфокусироваться на относительно небольшом фрагменте кода*. Схема организации программы со многими заголовочными файлами позволяет четко выявить, от чего зависит, например, модуль парсера, и *игнорировать остальные части программы*. Подход же с единственным заголовочным файлом заставляет нас в рамках каждого модуля просматривать все объявления и решать, какие из них необходимы. Факт состоит в том, что при этом приходится работать с кодом в условиях неполной информации, отталкиваясь от локальной перспективы. Подход же со многими заголовочными файлами позволяет нам в локальной перспективе успешно продвигаться «изнутри наружу» (from the inside out). Метод с единственным заголовком, как и любая другая организация программ с глобальным хранением информации, требует *подхода «сверху вниз»* (top-down approach) и заставляет нас бесконечно выяснять, что от чего зависит.

Более сильная локализация ведет, к тому же, и к меньшим временам компиляции. Эффект может быть весьма значительным. Я наблюдал уменьшение времени

компиляции в десятки раз из-за лучшего анализа зависимостей и лучшего применения заголовочных файлов.

9.3.2.1. Остальные модули калькулятора

Остальные модули калькулятора можно организовать аналогично модулю парсера. Однако они столь малы, что для них нет нужды в собственных *_impl.h* файлах. Такие файлы нужны лишь логическим модулям, содержащим наборы функций с общим контекстом.

Обработчик ошибок сократился до набора типов исключений, так что для него не требуется *error.c* файл:

```
// файл error.h:

namespace Error
{
    struct Zero_divide{ };
    struct Syntax_error
    {
        const char* p;
        Syntax_error(const char* q) {p = q; }
    };
}
```

Лексический анализатор обладает довольно большим и более запутанным интерфейсом:

```
// файл lexer.h:

#include <string>

namespace Lexer
{
    enum Token_value
    {
        NAME,          NUMBER,          END,
        PLUS='+',      MINUS='-',      MUL='*',      DIV='/',
        PRINT=';',      ASSIGN='=',      LP='(',      RP=')'
    };

    extern Token_value curr_tok;
    extern double number_value;
    extern std::string string_value;

    Token_value get_token();
}
```

Реализация лексического анализатора кроме заголовка *lexer.h* зависит еще от *error.h*, *<iostream>*, *driver.h* и от функций, определяющих тип символов, объявленных в *<cctype>*:

```
// файл lexer.c:

#include "lexer.h"
#include "error.h"
#include "driver.h"
```

```
#include <iostream>
#include <cctype>

Lexer::Token_value Lexer::curr_tok;
double Lexer::number_value;
std::string Lexer::string_value;

Lexer::Token_value Lexer::get_token() { /* ... */ }
```

Можно было бы выделить директиву `#include "error.h"` в отдельный файл с суффиксом `_impl.h`, но я счел это излишним для столь малой программы.

Как всегда, мы включаем файл с пользовательским интерфейсом модуля — в данном случае файл `lexer.h`, в файл реализации модуля, чтобы дать компилятору возможность проверить согласованность типов.

Причина, по которой мы вводим в программу файл `driver.h`, заключается в том, что функции `Lexer::get_token()` нужен доступ к средствам ввода:

```
// файл driver.h:

namespace Driver
{
    int no_of_errors;
    std::istream* input;
    void skip();
}
```

Таблица символов вполне самодостаточна, и стандартный заголовок `<map>` содержит все необходимое для ее эффективной реализации (в виде экземпляра шаблонного класса `map`):

```
// файл table.h:

#include <map>
#include <string>

extern std::map<std::string, double> table;
```

Поскольку каждый заголовочный файл может включаться в разные `.c` файлы, то нужно отделить объявление `table` от ее определения:

```
// файл table.c:

#include "table.h"

std::map<std::string, double> table;
```

По существу, управляющая программа зависит от всего:

```
// файл main.c:

#include "parser.h"
#include "lexer.h"
#include "error.h"
#include "table.h"
#include "driver.h"
#include <sstream>

int main (int argc, char* argv[]) { /*...*/ }
```

Для больших программ стоило бы реорганизовать структуру программы так, чтобы ее управляющая часть имела бы меньше прямых зависимостей. Часто также желательно минимизировать и код функции *main()*, поместив в нее лишь вызов управляющей функции, размещенной в своем собственном модуле. Это особенно важно для библиотечного кода, ибо тогда мы не можем полагаться на *main()* и должны быть готовы к тому, что вызовы могут последовать из самых разных функций (§9.6[8]).

9.3.2.2. Использование заголовочных файлов

Оптимальное для данной программы количество используемых заголовочных файлов зависит от многих факторов. Многие из этих факторов относятся даже не к языку C++, а к тому, как ведется работа с файлами в вашей системе. Например, если ваш редактор не допускает удобной параллельной работы со множеством файлов, то большое количество заголовочных файлов становится определенным препятствием в работе. Или, если ваш редактор на открытие и просмотр двадцати файлов по пятьдесят строк каждый требует больше времени, чем на ту же работу с единственным файлом из 1000 строк, то тут уж дважды подумаешь, прежде чем решишься применить схему организации небольшой программы в варианте со многими заголовками.

Предупредим, что дюжина собственных плюс ряд стандартных для конкретной реализации заголовков (могут исчисляться сотнями) вполне терпимы. Однако если вы сегментируете объявления в большой программе на логически минимальные заголовки (содержащие, например, объявление единственной структуры), то можете столкнуться с хаосом из сотен файлов даже для небольших проектов. Я нахожу это излишним.

Для больших проектов множественные заголовочные файлы неизбежны. В таких проектах сотни собственных (не считая стандартных) заголовков являются нормой. Настоящая проблема начинается, когда их число переваливает за тысячу. В масштабе таких проектов обсуждаемые здесь методики по-прежнему применимы, но это уже задача для Геракла. Запомните, что для настоящих больших программ единственный заголовочный файл — это не актуально. В них всегда присутствует много заголовочных файлов. Реальный выбор между двумя стилями организации заголовков переносится на меньшие части проектов.

На самом деле, обсуждаемые здесь два стиля организации заголовочных файлов вовсе не альтернативны друг другу. Скорее, они являются комплиментарными технологиями, выбор между которыми всегда стоит при разработке важных модулей, и вопрос о таком выборе возникает снова и снова по мере эволюции программной системы. Важно понимать, что один и тот же интерфейс не может быть оптимальным для всех. Обычно стоит различать интерфейс для пользователей и интерфейс для разработчиков. Дополнительно, часто в больших системах большинству пользователей предоставляется несколько упрощенный интерфейс, в то время как дополнительные возможности предоставляются пользователям-экспертам. Экспертный интерфейс («полный интерфейс») обычно директивами *#include* включает намного больше средств, чем хотел бы знать обычный пользователь системы. На практике, обычный пользовательский интерфейс получается после удаления из экспертного интерфейса как раз таких средств (и соответственно директив *#include*), которые неизвестны рядовому пользователю. Термин «рядовой пользова-

тель» не является уничижительным. В тех областях, где я не обязан быть экспертом, я предпочитаю быть рядовым пользователем — это помогает избегать ненужных трудностей.

9.3.3. Защита от повторных включений

Целью применения множественных заголовочных файлов является построение согласованных и самодостаточных модулей. С точки зрения программы многие объявления, необходимые для обеспечения самодостаточности отдельных модулей, в целом избыточны. В больших программах такая избыточность может приводить к ошибкам, ибо определения классов или встраиваемых функций могут через директивы **#include** (в том числе и вложенные) попасть в отдельные единицы трансляции более одного раза (§9.2.3).

У нас есть две возможности:

1. Реорганизовать программу для устранения избыточности, или
2. Найти способ безопасного повторного включения заголовков.

Первый подход — именно он ведет к нашей последней версии калькулятора — весьма утомителен и непрактичен в случае реально больших программ. Кроме того, избыточность ведь делает отдельные части программ более осмысленными.

Конечно, выигрыш от тщательного анализа избыточности и результирующего упрощения программы может быть значительным как с логической точки зрения, так и в плане сокращения времени компиляции. Однако такой анализ редко когда бывает полным, так что в любом случае требуется найти метод преодоления негативных последствий множественного включения заголовков. Желательно, чтобы этот метод можно было применять систематически, ибо заранее неизвестна полнота анализа, проводимого пользователем.

Традиционным решением проблемы является использование следующей комбинации директив условной компиляции — **#ifndef**, **#define** и **#endif**:

// файл *error.h*:

```
#ifndef CALC_ERROR_H
#define CALC_ERROR_H

namespace Error
{
    // ...
}

#endif                // CALC_ERROR_H
```

Содержимое заголовочного файла между **#ifndef** и **#endif** игнорируется компилятором, если макроконстанта **CALC_ERROR_H** определена. Таким образом, когда заголовочный файл *error.h* первый раз встречается при компиляции, его содержимое вычитывается полностью, а константа **CALC_ERROR_H** становится определенной (в результате действия директивы **#define**). Но если компилятор в пределах той же самой единицы трансляции встретит *error.h* еще раз, его содержимое будет проигнорировано. Этот метод, хоть он и опирается на трюкачество с макросами, реально работает и широко распространен в мире программирования на C и C++. Все стандартные заголовочные файлы используют этот метод.

Так как заголовочные файлы включаются в произвольном контексте, то следует учитывать возможность конфликта имен, из-за чего для макроконстант, контролирующих множественные включения, рекомендуется выбирать длинные и «безобразно» выглядящие имена.

Как только люди привыкают к заголовочным файлам и получают метод избавления от множественных включений, они тотчас же приступают к неумеренному наращиванию количества прямо или косвенно включаемых заголовочных файлов. Даже в реализациях с оптимизированной обработкой заголовков, такое нежелательно, ибо может сильно увеличивать время компиляции и засоряет глобальную область видимости объявлениями и макросами. Последнее может оказать непредсказуемое влияние на смысл программы. Заголовочные файлы следует использовать лишь тогда, когда в этом есть действительная необходимость.

9.4. Программы

Программа состоит из множества отдельно компилируемых единиц, собираемых воедино компоновщиком. Каждая функция, объект, тип и т.п. должен иметь уникальное определение в рамках этого множества (§4.9, §9.2.3). В программе должна быть единственная функция *main*() (§3.2). Вычислительная активность программы начинается в функции *main*() и завершается по выходу из этой функции. Возвращаемое этой функцией целое значение передается программному загрузчику в качестве результата работы программы.

Рассмотренная простая схема получает дополнительные детали в программах, содержащих глобальные переменные (§10.4.9), или в программах, генерирующих перехватываемые исключения (§14.7).

9.4.1. Инициализация нелокальных переменных

В принципе, любая переменная, определяемая вне функции (в глобальном пространстве, в именованном пространстве имен, в качестве статического члена класса), инициализируется до вызова *main*() . Порядок инициализации таких переменных соответствует порядку их определения в единице трансляции (§10.4.9). Если явные инициализаторы отсутствуют, то переменные инициализируются умолчательными значениями, характерными для их типов данных (§10.4.2). Умолчательное значение для встроенных типов и перечислений равно *нулю*. Например:

```
double x=2;           // нелокальные переменные
double y;
double sqx=sqrt(x+y);
```

В этом примере нелокальные переменные *x* и *y* инициализируются раньше, чем переменная *sqx*, так что при инициализации последней вычисляется *sqrt(2)* .

Порядок инициализации глобальных переменных, определяемых в разных единицах трансляции, стандартом языка C++ не фиксируется. Поэтому крайне неразумно строить код, зависящий от порядка инициализации глобальных переменных. Кроме того, невозможно перехватить исключение, сгенерированное инициализатором глобальной переменной (§14.7). Использование глобальных переменных вообще желательно минимизировать, в особенности тех, что требуют сложной инициализации.

В принципе, существует ряд приемов, позволяющих фиксировать порядок инициализации глобальных переменных, определенных в разных единицах трансляции. Однако они не являются ни эффективными, ни переносимыми. Например, динамические библиотеки плохо переживают зависимость от таких глобальных переменных.

Хорошей альтернативой глобальным переменным могут служить функции, возвращающие ссылку. Например:

```
int& use_count ()
{
    static int uc = 0;
    return uc;
}
```

Вызов `use_count()` действует как глобальная переменная, инициализация которой происходит при самом первом вызове (§5.5, §7.1.2). Например:

```
void f()
{
    cout<< ++use_count() ;    // увеличить значение и вывести его
    // ...
}
```

Инициализация нелокальных переменных (статически распределенных) зависит от механизма запуска программ, реализованного в конкретной системе. Гарантируется правильность работы такого механизма при вызове функции `main()`. Отсюда следует, что в общем случае невозможно обеспечить правильность инициализации нелокальных переменных в C++-коде, если сам этот код предназначен для работы в программах, написанных на иных языках программирования.

Отметим, что переменные, инициализируемые константными выражениями (§C.5) не могут зависеть от значения объектов из других единиц трансляции и не требуют инициализации на этапе выполнения. Такие переменные можно безопасно использовать во всех случаях.

9.4.1.1. Завершение выполнения программы

Можно завершить работу одним из следующих способов:

- возвратом из функции `main()`;
- вызовом `exit()`;
- вызовом `abort()`;
- генерацией неперехватываемого исключения.

Дополнительно существуют нереконструируемые, зависящие от реализации способы доведения программы до краха.

Если выход из программы осуществляется вызовом стандартной библиотечной функции `exit()`, то вызываются деструкторы для всех статически сконструированных объектов (§10.4.9, §10.2.4). В то же время, если прекращение работы программы достигается вызовом стандартной библиотечной функции `abort()`, то этого не происходит. Отсюда следует, что при использовании функции `exit()` мгновенного прекращения работы программы не происходит. Более того, вызов `exit()` из дест-

руктора вообще может привести к бесконечной рекурсии. Функция **exit()** объявляется следующим образом

```
void exit(int) ;
```

Как и у функции **main()**, возврат **exit()** адресован системе в качестве результата работы программы. Нуль означает успешное завершение.

При вызове **exit()** не будут вызваны деструкторы локальных переменных во всех функциях вверх по цепочке имевших место функциональных вызовов. Генерация и перехват исключений гарантируют корректное уничтожение локальных объектов (§14.4.7). Вызов же **exit()** не позволяет корректно отработать всем функциям из цепочки вызовов. Поэтому лучше не ломать контекст вызова функций и просто сгенерировать исключение, а вопрос о том, что делать дальше, оставить обработчикам.

В языках С и С++ стандартная библиотечная функция **atexit()** позволяет вызывать некоторый код по выходу из программы. Например:

```
void my_cleanup() ;
```

```
void somewhere() {
```

```
if(atexit(&my_cleanup)==0)
```

```
{
```

```
    // my_cleanup будет вызвана при нормальном завершении
```

```
}
```

```
else
```

```
{
```

```
    // oops: слишком много функций atexit
```

```
}
```

```
}
```

Это уже сильно напоминает автоматический вызов деструкторов глобальных переменных по выходу из программы (§10.4.9, §10.2.4). Обратите внимание, что функция-аргумент **atexit()** сама не может иметь аргументов и возвращаемого значения. От конкретной реализации зависит предельное количество функций **atexit()**; при превышении предела **atexit()** возвращает нуль. Это делает **atexit()** менее полезной, чем могло показаться на первый взгляд.

Деструкторы статически размещенных объектов (глобальных, §10.4.9; локальных в функциях с модификатором **static**, §7.1.2; определенных с модификатором **static** в классах, §10.2.4), созданных до вызова **atexit(f)**, вызываются после вызова **f()**. Деструкторы же таких объектов, созданных после вызова **atexit()**, вызываются до вызова **f()**.

Функции **exit()**, **abort()** и **atexit()** объявлены в **<cstdlib>**.

9.5. Советы

1. Используйте заголовочные файлы для представления интерфейсов и логической структуры; §9.1, §9.3.2.
2. Включайте заголовочный файл в исходный файл, реализующий объявленные функции; §9.3.1.
3. Не определяйте глобальные сущности с одинаковыми именами, но с разным (пусть и похожим) смыслом в разных единицах трансляции; §9.2.
4. Избегайте определений невстраиваемых функций в заголовочных файлах; §9.2.1.
5. Используйте директивы **#include** только в глобальной области или в пространствах имен; §9.2.1.
6. С помощью **#include** включайте только полные объявления; §9.2.1.
7. Реализуйте защиту от лишних включений; §9.3.3.
8. Закрывайте заголовочные файлы языка C в пространства имен во избежание конфликта глобальных имен; §8.2.9.1, §9.2.2.
9. Делайте заголовочные файлы самодостаточными; §9.2.3.
10. Различайте пользовательские интерфейсы и интерфейсы для разработчиков; §9.3.2.
11. Различайте обычные пользовательские интерфейсы и интерфейсы для пользователей-экспертов; §9.3.2.
12. Избегайте нелокальных объектов с инициализацией на этапе выполнения, предназначенных для использования в качестве фрагментов программ, написанных не на C++; §9.4.1.

9.6. Упражнения

1. (*2) Найдите, где в вашей системе находятся заголовочные файлы стандартной библиотеки. Просмотрите список их имен. Есть ли среди них нестандартные заголовочные файлы? Могут ли нестандартные заголовки включаться с помощью `<>`?
2. (*2) Где хранятся заголовочные файлы нестандартных «фундаментальных» библиотек вашей системы?
3. (*2.5) Напишите программу, которая читает исходные файлы на C++ и выводит имена файлов, включенных в них директивой **#include**. В результирующем списке примените отступы для наглядного показа информации о том, какие файлы включаются в тот или иной исходный файл. Опробуйте эту программу на реальных исходных файлах (и получите информацию об объеме включаемых файлов).
4. (*3) Модифицируйте программу из предыдущего упражнения, чтобы она выводила количество строк-комментариев, количество строк без комментариев, количество слов (разделенных пробельными символами) в каждом включаемом файле.

5. (*2.5) Разработайте внешнее средство, которое проверяет перед компиляцией каждый исходный файл на предмет существования повторных включений заголовков и устраняет повторы. Опробуйте это средство на практике и сравните его со способами контроля повторных включений, рассмотренных в §9.3.3. Дает ли такое средство какие-либо дополнительные преимущества на этапе выполнения на вашей системе?
6. (*3) Как в вашей системе реализуется динамическая компоновка? Какие ограничения накладываются на динамически компокуемый код? Что дополнительно требуется от кода, чтобы он мог быть скомпонован динамически?
7. (*3) Откройте и прочтите 100 файлов по 1500 символов каждый. Откройте и прочтите один файл, содержащий 150000 символов (см. пример в §21.5.1). Есть ли разница во времени выполнения задач? Каково наибольшее число файлов, которые можно одновременно открыть в вашей системе? Рассмотрите этот вопрос касательно использования включаемых файлов.
8. (*2) Модифицируйте нашу версию калькулятора так, чтобы его можно было вызывать из **main** (), либо из любой другой функции (в виде обычного функционального вызова).
9. (*2) Нарисуйте диаграмму зависимостей модулей (§9.3.2) для версии калькулятора, которая использует **error** () вместо исключений (§8.2.2).

